Exercises

Advanced Computing Architectures

Vector mode

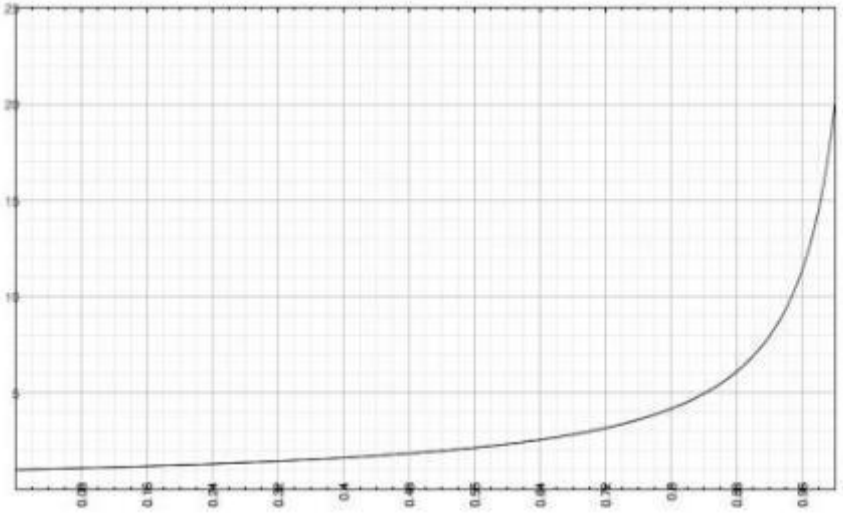
- Computation in vector mode runs 20 times faster than normal
  1. Draw a graph that plots the overall speedup as a function of the percentage of the computation performed in vector mode
  2. What percentage is needed to achieve an overall speedup of 2?
  3. What percentage is needed to achieve one-half of the maximum speedup attainable?
  4. What percentage do you need in order to achieve the same speedup that you would get if vector mode run 40 times faster and the percentage were 70%?

$$S_{overall} = \frac{1}{(1-F) + \frac{F}{S}} \xrightarrow[S \to \infty]{} \frac{1}{1-F}$$

Vector mode: solutions

1. Plot the graph of the speed-up function, using S = 20, and F going from 0 to 1
2. Let $S_{overall}$ = 2 and S = 20, then solve for F. The result is F = 53% (or use the graph above)
3. The maximum speed-up is 20 (for F = 1). To have $S_{overall}$ = 20/2 = 10, as before, we solve for F. The result is F = 95%
4. At S = 40 and F = 70% you get $S_{overall}$ = 3.15. Now set S = 20 again, and solve for F. The result is F = 71.8%.

$$F = \frac{S(1 - S_{overall})}{S_{overall}(1 - S)}$$



load/store – register/memory

- Want to replace a load/store with register/memory instructions
  - LOAD R1, 0(Rb) ADD R2,0(Rb)
  - ADD R2, R2, R1
- The register/memory solution reduces the number of instructions to be executed (it removes a load!), but is more complex, so it increases the clock period by 10%
- Assume the CPI is not affected
- Assume 36% of the instructions are load/store, of which 2/3 are loads
- How many loads should be removed in order for the performance of the two machines to be the same?
- Recall the CPU time equation
  - CPU time = CPI · IC · T_{clk}

Solution

- First off, let's compute the performance of the two machines, and set them equal to have a first equation

  $P_1 = CPI \cdot T_{CLK} \cdot IC_1$

  $P_2 = CPI \cdot T_{CLK} \cdot 1.10 \cdot IC_2$

  $P_2 / P_1 = 1$

  $IC_2 = IC_1 \cdot 10/11$
- Assume $IC_1 = 100$. Then IC_2 must be equal to 90.91. Approximate to 91.
- So we need to remove 9 instructions out of 100 instructions
- Of the 100 instructions, only $36 \cdot 2/3 = 24$ are loads
- So we need to remove 9 out of 24 loads, or 37.5% of the loads
- Finally there are going to be 24 – 9 = 15 loads, and 91 instructions
- So the loads become 15 / 91 = 16.5% of the total instructions

How many registers?

- We have three machines M_0, M_8 and M_{16}, with 0, 8 and 16 registers. All operations use three operands. Assume that
  - Access to memory is 6 cycles
  - Access to register is 1 cycle
- Given the probability of operand in register below, and the duration in cycles of each operation, compute the cycle count per operation

Machine	Registers	Operation cycles	Probability of operand in register
M_0	0	4 cycles	0.0
M_8	8	5 cycles	0.5
M_{16}	16	6 cycles	0.8

Solution

- Compute the average operation duration
  - For M_0, all references go to memory and take 6 cycles each. Including 4 cycles for the operation itself:
    - $M_0 = 4 + 3 \cdot 6 = 22$ cycles
  - For M_1, half of the references take 1 cycle, the other half 6. We compute the average access time and then add the contribution of the operation itself (5 in this case)
    - $A_1 = 0.5 \cdot 6 + 0.5 \cdot 1 = 3.5$ cycles
    - $M_1 = 5 + 3 \cdot 3.5 = 15.5$ cycles
  - For M_2, 80% of the references take 1 cycle, and 20% take 6. We compute the average access time and then add the contribution of the operation itself (6 in this case)
    - $A_2 = 0.2 \cdot 6 + 0.8 \cdot 1 = 2$ cycles
    - $M_2 = 6 + 3 \cdot 2 = 12$ cycles
- Conclusion
  - Adding registers in this case provides faster execution times, despite the added complexity which makes the operations themselves run longer

What's the CPI?

- Assume the following CPIs:
  - Alu operations: 1
  - Load/Store: 3
  - Branch: 5
- Given the table below of thousands of executed instructions, what is the optimization in number of instructions?
- What is the effective CPI for optimized and non-optimized? And the CPU time (in number of clock cycles)?

Program	Branches	Loads/stores	ALU ops
TeX unoptimzed	13	45	50
TeX optimized	12	30	41
Spice unoptimzed	6	52	59
Spice optimized	4	31	38
GCC unoptimized	28	70	81
GCC optimzed	26	53	69

What's the CPI?

- Assume the following CPIs:
  - Alu operations: 1
  - Load/Store: 3
  - Branch: 5
- Given the table below of thousands of executed instructions, what is the optimization in number of instructions?
- What is the effective CPI for optimized and non-optimized? And the CPU time (in number of clock cycles)?

13/108*5 + 45/108*3 + 50/108 = 2.31

Program	Branches	Loads/stores	ALU ops	N. instructions	Eff. CPI	CPU time
TeX unoptimzed	13	45	50	108	2.31	250
TeX optimized	12	30	41	83 (23%)	2.30	191 (24%)
Spice unoptimzed	6	52	59	117	2.09	245
Spice optimized	4	31	38	73 (38%)	2.07	151 (38%)
GCC unoptimized	28	70	81	179	2.41	431
GCC optimzed	26	53	69	148 (17%)	2.42	358 (17%)

Actual CPI

- Single issue pipeline, ideal CPI = 1
- 60% of the instructions have data dependencies
  - 25% are not fixed by forwarding and result in 2 cycles of stall
- 20% of the instructions are branches
  - They result in 3 clock cycles of delay (stalls)

Actual CPI

- Single issue pipeline, ideal CPI = 1
- 60% of the instructions have data dependencies
  - 25% are not fixed by forwarding and result in 2 cycles of stall
- 20% of the instructions are branches
  - They result in 3 clock cycles of delay (stalls)
- What is the actual CPI?
  - Actual CPI = Ideal CPI + average number of stalls per instruction
  - Actual CPI = 1 + 0.6*0.25*2 + 0.2*3
  - Actual CPI = 1 + 0.3 + 0.6 = 1.9 (clocks per instruction)

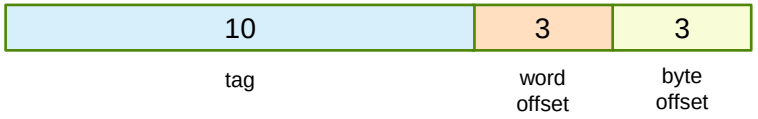
CACHES

Hit or miss?

- Consider an 8 kbyte cache made of blocks of 8 words of 64 bits (8 bytes)
  - Show how a hypothetical 16-bit address is divided for byte addressing, word addressing, index and tag for a direct mapped, a fully associative and a four-way set associative cache
- For the following sequence of addresses, determine which are hit and which are miss for the direct mapped and the 4-way set associative cache
  - 0x0120, 0x4110, 0x0130, 0x0108, 0x4120, 0x4080

Solution

- Each word is 64 bits, or 8 bytes
  - I need 3 bits for the byte offset
- Each cache line is 8 words
  - I need 3 bits for the word offset within the cache line
- For the direct mapped cache, there are 128 lines (8 kB / 64 B/w = 128)
  - I need 7 bits for the index
  - I'm left with 16 – 7 – 3 – 3 = 3 bits for the tag
- For the fully associative
  - There is no index
  - The tag is 16 – 3 – 3 = 10 bits
- For the 4-way set associative
  - There are 128 / 4 = 32 sets
  - I need 5 bits for the index
  - I'm left with 16 – 5 – 3 – 3 = 5 bits for the tag



Solution: direct mapped

- Split the address in fields (3, 7, 3, 3)
  - 0x0120 = 0000_0001_0010_0000 = 000_0000100_100_000 (**miss**)
    - Data not yet in cache, compulsory miss
  - 0x4110 = 0100_0001_0001_0000 = 010_0000100_010_000 (**miss**)
    - Same index, different tag, conflict miss
  - 0x0130 = 0000_0001_0011_0000 = 000_0000100_110_000 (**miss**)
    - Same index, different tag, conflict miss
  - 0x0108 = 0000_0001_0000_1000 = 000_0000100_001_000 (**hit**)
    - Same index, same tag, different word, hit
  - 0x4120 = 0100_0001_0010_0000 = 010_0000100_100_000 (**miss**)
    - Same index, different tag, conflict miss
  - 0x4080 = 0100_0000_1000_0000 = 010_0000010_000_000 (**miss**)
    - Different index, data not yet in cache, compulsory miss

Solution: 4-way set associative

- Split the address in fields (5, 5, 3, 3)
  - 0x0120 = 0000_0001_0010_0000 = 00000_00100_100_000 (**miss**)
    - Data not yet in cache, compulsory miss
  - 0x4110 = 0100_0001_0001_0000 = 01000_00100_010_000 (**miss**)
    - Same index, different tag, no conflict, data not yet in cache, compulsory miss
  - 0x0130 = 0000_0001_0011_0000 = 00000_00100_110_000 (**hit**)
    - Same index, same tag, different word, hit
  - 0x0108 = 0000_0001_0000_1000 = 00000_00100_001_000 (**hit**)
    - Same index, same tag, different word, hit
  - 0x4120 = 0100_0001_0010_0000 = 01000_00100_100_000 (**hit**)
    - Same index, same tag, different word, hit
  - 0x4080 = 0100_0000_1000_0000 = 01000_00010_000_000 (**miss**)
    - Different index, data not yet in cache, compulsory miss

Cache parameters

- Miss rate
  - Given in percent. Fraction of memory accesses that result in a miss
- Memory references per instruction
  - On average, how many references to memory each instruction performs
  - At least 1, since the instruction must be fetched
  - Loads and stores perform additional accesses
- Cache misses per 1000 instructions
  - The number of cache misses that occur on average every 1000 instructions
  - To convert to miss rate, must know how many memory references were performed
  - That depends on the average memory references per instruction
  - $miss\ rate = \frac{misses\ per\ 1000\ instructions}{memory\ accesses\ per\ 1000\ instructions}$
  - $miss\ rate = \frac{misses\ per\ 1000\ instructions}{1000 \bullet (memory\ references\ per\ instruction)}$

CPU performance

- Compute the impact of cache on CPU performance
  - CPI = 1.0 (without memory stalls)
  - Cache miss penalty is 200 clock cycles
  - Average miss rate is 2%
  - Average of 1.5 memory references per instruction
  - Average number of cache misses per 1000 instructions is 30
    - = 1000 * 1.5 * 2% = 30
- Compute CPU time assuming
  - Perfect cache: Cache is always a hit (miss rate = 0%)
  - Real cache: Cache as above (miss rate = 2%)
  - No cache: Cache is always a miss (miss rate = 100%)

CPU performance

- Computing the CPU time
  - CPU time = IC × (CPI + Miss rate × memory_accesses/instruction × Miss penalty) × Clock cycle time
  - IC × (1.0 + (2% × 1.5 × 200)) × Clock cycle time
  - IC × 7.00 × Clock cycle time
  - CPI increases 7 times if we include the effect of memory access!!
- CPU time with no cache (always miss)
  - CPU_time = IC × (1.0 + (100% × 1.5 × 200)) × Clock cycle time
  - IC × 301 × Clock cycle time
  - Execution time more than 40 times longer without the cache!!

To split or unify?

- Table gives misses per 1000 instructions of split and unified caches of different sizes
  - Assume 36% of the instructions are loads/stores

Size	Instruction cache	Data cache	Unified cache
8 KB	8.16	44.0	63.0
16 KB	3.82	40.9	51.0
32 KB	1.36	38.4	43.3
64 KB	0.61	36.9	39.4
128 KB	0.30	35.3	36.2
256 KB	0.02	32.6	32.9

To split or unify?

- Which is better? Two 16 KB split, or one 32 KB unified?
  - I-cache misses per 1000 instr: 3.82
  - D-cache misses per 1000 instr: 40.9
  - U-cache misses per 1000 instr: 43.3
- Compute miss rate and average memory access time
  - Assuming hit is 1 clock cycle, miss penalty is 200 clock cycles
  - Load and store hits take 1 additional clock cycle on the unified cache with one port to satisfy 2 requests (load/store and fetch)
  - Loads and stores are 36% of the instructions
  - Write through with write buffer

To split or unify?

- Compute miss rate
  - Miss rate is number of misses per number of accesses
  - We have instead the number of misses per 1000 instructions
  - Need then to divide by the actual number of memory accesses per instruction
  - (i.e., not all 1000 instruction necessarily result in a memory access)

To split or unify?

- Miss rate for instruction cache
  - 3.82 misses per 1000 instructions
  - Every instruction results in a memory access
  - Miss rate_{16KB_instr} = 3.82 / (1000 * 1.0) = 0.004
- Miss rate for data cache
  - 40.9 misses per 1000 instructions
  - Only 36% of instruction access the data cache
  - Miss rate_{16KB_data} = 40.9 / (1000 * 0.36) = 0.114

To split or unify?

- Miss $rate_{16KB_instr}$ = 3.82 / (1000 * 1.0) = 0.004
- Miss $rate_{16KB_data}$ = 40.9 / (1000 * 0.36) = 0.114

- Overall miss rate for split caches
  - Need to compute a weighted average between instruction and data cache
  - What is the fraction of instruction references?
  - If I execute 1000 instructions, I have 1000 instruction references and 1000 • 0.36 = 360 data references
  - So the instruction references are 1000 / 1360 = 74%
  - The data references are 360 / 1360 = 26%
  - Miss $rate_{split}$ = (74% * 0.004) + (26% * 0.114) = 0.0326

To split or unify?

- Miss $rate_{split}$ = (74% * 0.004) + (26% * 0.114) = 0.0326

- Miss rate for unified cache
  - 43.3 misses per 1000 instructions
  - Every instruction results in a memory access for fetch, and 36% have memory access for data
  - Miss $rate_{32KB_unif}$ = 43.3 / (1000 * (1.0 + 0.36)) = 0.0318
- Split cache has a slightly higher average miss rate than the unified cache

To split or unify?

- Average memory access time
  - AMAT = %instructions × (Hit time + Instruction miss rate × Miss penalty) + %data × (Hit time + Data miss rate × Miss penalty)
- Average memory access time split

  = 74% × (1 + 0.004 × 200) + 26% × (1 + 0.114 × 200)

  = (74% × 1.80) + (26% × 23.80) = 1.332 + 6.188 = 7.52
- Average memory access time unified

  = 74% × (1+0.0318 × 200)+26% × (1+1+0.0318 × 200)

  = (74% × 7.36) + (26% × 8.36) = 5.446+2.174 = 7.62
- Split cache has lower average memory access time because it does not have the structural hazard

CPU comparison

- Consider the following CPUs running gcc:
  - A simple MIPS two-issue static pipe running at a clock rate of 4 GHz and achieving a pipeline CPI of 0.8. This processor has a cache system that yields 0.005 misses per instruction.
  - A deeply pipelined version of a two-issue MIPS processor with slightly smaller caches and a 5 GHz clock rate. The pipeline CPI of the processor is 1.0, and the smaller caches yield 0.0055 misses per instruction on average.
  - A speculative superscalar with a 64-entry window. It achieves one-half of the ideal issue rate measured for this window size (9), i.e., it issues 4.5 instructions per clock. This processor has the smallest caches, which lead to 0.01 misses per instruction, but it hides 25% of the miss penalty on every miss by dynamic scheduling. This processor has a 2.5 GHz clock.
- Assume main memory has a 50 ns read time (miss penalty)

- Determine the relative performance (MIPS) of the three processors

Processor 1

- Specs:
  - A simple MIPS two-issue static pipe running at a clock rate of 4 GHz and achieving a pipeline CPI of 0.8. This processor has a cache system that yields 0.005 misses per instruction.
- Compute miss penalty
  - CPI penalty = misses per instruction * miss penalty
  - miss penalty = memory access time / clock cycle
- Substitute values
  - miss penalty = 50 ns / 250 ps = 200 cycles
  - CPI penalty = 0.005 * 200 = 1.0
- Compute total CPI
  - CPI = ideal CPI + CPI penalty = 0.8 + 1.0 = 1.8
- Compute instruction execution rate
  - IER = Clock rate / CPI = 4000 MHz / 1.8 = 2222 MIPS

Processor 2

- Specs:
  - A deeply pipelined version of a two-issue MIPS processor with slightly smaller caches and a 5 GHz clock rate. The pipeline CPI of the processor is 1.0, and the smaller caches yield 0.0055 misses per instruction on average.
- Compute miss penalty
  - CPI penalty = misses per instruction * miss penalty
  - miss penalty = memory access time / clock cycle
- Substitute values
  - miss penalty = 50 ns / 200 ps = 250 cycles
  - CPI penalty = 0.0055 * 250 = 1.375
- Compute total CPI
  - CPI = ideal CPI + CPI penalty = 1.0 + 1.375 = 2.375
- Compute instruction execution rate
  - IER = Clock rate / CPI = 5000 MHz / 2.375 = 2105 MIPS

Processor 3

- Specs:
  - A speculative superscalar with a 64-entry window. It achieves one-half of the ideal issue rate measured for this window size (9). This processor has the smallest caches, which lead to 0.01 misses per instruction, but it hides 25% of the miss penalty on every miss by dynamic scheduling. This processor has a 2.5 GHz clock.
- Compute miss penalty
  - CPI penalty = misses per instruction * miss penalty
  - miss penalty = memory access time / clock cycle
- Substitute values
  - miss penalty = (0.75 * 50 ns) / 400 ps = 94 cycles
  - CPI penalty = 0.01 * 94 = 0.94
- Compute total CPI
  - ideal CPI = 1 / issue rate = 1 / (9/2) = 1 / 4.5 = 0.22
  - CPI = ideal CPI + CPI penalty = 0.22 + 0.94 = 1.16
- Compute instruction execution rate
  - IER = Clock rate / CPI = 2500 MHz / 1.16 = 2155 MIPS

DYNAMIC SCHEDULING

Exploring impact of techniques

- Compute the baseline performance (in cycles, per loop iteration) of the following code sequence
  - Assume no new instruction execution is initiated until the previous instruction has completed

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

Simple stalling execution

```
Loop:   LD      F2,0(Rx)      1 + 3
        MULTD   F2,F0,F2      1 + 4
        DIVD    F8,F2,F0      1 + 10
        LD      F4,0(Ry)      1 + 3
        ADDD    F4,F0,F4      1 + 2
        ADDD    F10,F8,F2     1 + 2
        SD      F4,0(Ry)      1 + 1
        ADDI    Rx,Rx,#8      1
        ADDI    Ry,Ry,#8      1
        SUB     R20,R4,Rx     1
        BNZ     R20,Loop      1 + 1
                              ____
        cycles per loop iter  37
```

- Each instruction requires one cycle plus the extra latency

Data dependencies

- Instead of stalling everything because one functional unit is busy, detect true data dependencies
  - Assume multiple functional units are available
  - Assume in-order issue (so, do not issue an instruction if the previous is stalled due to data dependencies)
  - But do issue if previous is already executing and there is no data hazard
  - Show the code with <stall> inserted where necessary to accommodate the latencies

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

```
Loop:   LD      F2,0(Rx)        1 + 3
        <stall>
        <stall>
        <stall>
        MULTD   F2,F0,F2        1 + 4
        <stall>
        <stall>
        <stall>
        <stall>
        DIVD    F8,F2,F0        1 + 10
        LD      F4,0(Ry)        1 + 3
        <stall due to LD latency>
        <stall due to LD latency>
        <stall due to LD latency>
        ADDD    F4,F0,F4        1 + 2
        <stall due to DIVD latency>
        <stall due to DIVD latency>
        <stall due to DIVD latency>
        <stall due to DIVD latency>
        <stall due to DIVD latency>
        ADDD    F10,F8,F2       1 + 2
        SD      F4,0(Ry)        1 + 1
        ADDI    Rx,Rx,#8        1
        ADDI    Ry,Ry,#8        1
        SUB     R20,R4,Rx       1
        BNZ     R20,Loop        1 + 1
        <stall due to BNZ>
                                ____
        cycles per loop iter    27
```

- Instructions must complete before a dependent instruction can be executed
  - e.g., first LD must stall the next instruction for three cycles
  - MULTD must stall for 4
  - DIVD stalls, but other independent instructions can be executed meanwhile

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

Re-schedule the instructions

- Move instructions in the code (static scheduling) so that stalls due to data dependencies are minimized
  - Assume you have sufficiently many functional units to run independent operations in parallel
  - Show code with <stall> again, and compute the total time to execute one iteration

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

Rescheduled instructions

```
LD       F2, 0(Rx)
LD       F4, 0(Ry)
<stall>
<stall>
MULTD    F2, F0, F2
ADDD     F4, F0, F4
<stall>
<stall>
<stall>
DIVD     F8, F2, F0
SD       F4, 0(Ry)
ADDI     Rx, Rx, #8
ADDI     Ry, Ry, #8
SUB      R20, R4, Rx
<stall>
<stall>
<stall>
<stall>
<stall>
<stall>
ADDD     F10, F8, F2
BNZ      R20, Loop
<stall>
```

Total number of cycles per loop iteration: 23

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

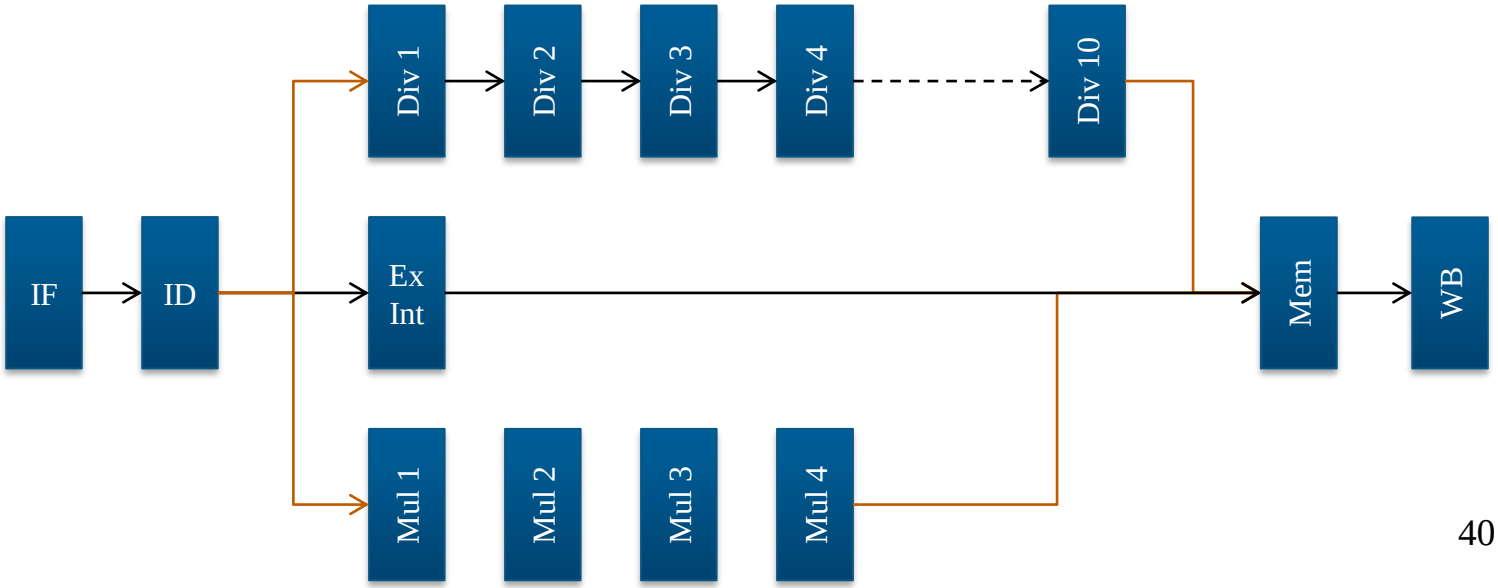
Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

Loop unrolling

- Hand-unroll two iterations of the loop in the reordered code
  - Reschedule the instructions after unrolling
  - Rename registers to avoid conflicts
  - Also, assume that all functional units are pipelined. For instance, the divider has 10 pipeline stages, and we can start a new division the clock cycle after the first

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10



Loop unrolling

```
LD       F2, 0(Rx)
LD       F12, 8(Rx)
LD       F4, 0(Ry)
LD       F14, 8(Ry)
MULTD    F2, F0, F2
MULTD    F12, F0, F12
ADDD     F4, F0, F4
ADDD     F14, F0, F14
<stall>
DIVD     F8, F2, F0
DIVD     F18, F12, F0
SD       F4, 0(Ry)
SD       F14, 8(Ry)
ADDI     Rx, Rx, #16
ADDI     Ry, Ry, #16
SUB      R20, R4, Rx
<stall>
<stall>
<stall>
<stall>
ADDD     F10, F8, F2
ADDD     F20, F18, F12
BNZ      R20, Loop
<stall>
```

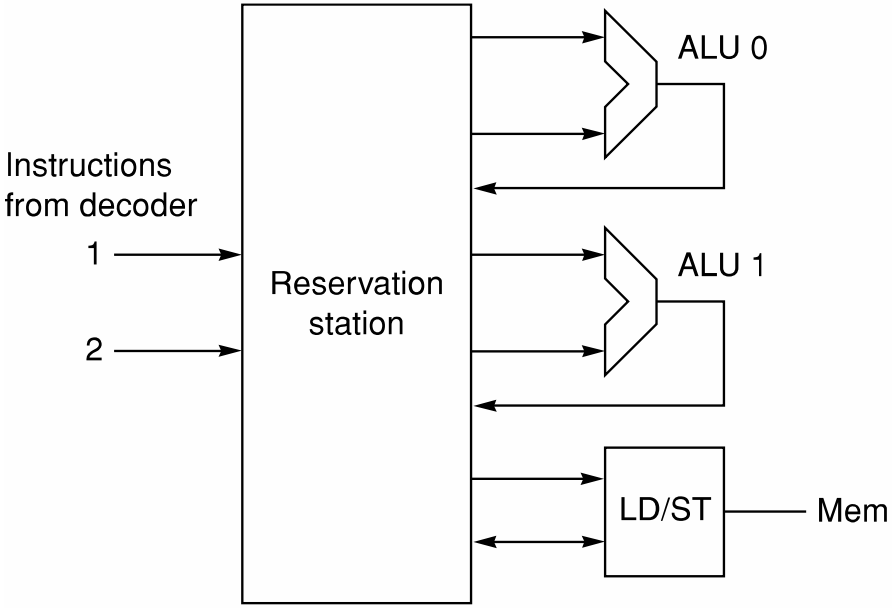
Total number of cycles per loop iteration: 24
Cycles per original loop iteration: 24/2 = 12

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

Dynamic scheduling

- Assume you are given the architecture shown in the figure
  - ALUs can do all arithmetic ops and branches
  - Can dispatch only one instruction to each FU per clock
  - **Assume all instructions are already loaded in the reservation stations, i.e., they are all ready to be sent to a functional unit**
  - Instructions are already renamed
  - Schedule the instructions out-of-order, clock by clock, to optimize performance
  - Assume result must be written into the reservation station before it is available for use (no bypassing)

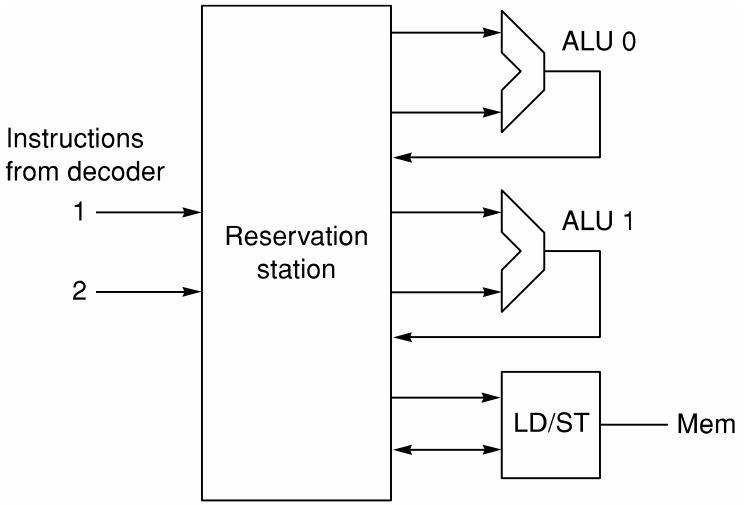


Dynamic scheduling

Clock cycle	alu0	alu1	ld/st
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
…			

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10



Dynamic scheduling

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Cycle op was dispatched to FU

Note: these ADDI's are generating Rx,y for next loop iteration, not this one.

Clock cycle	alu0	alu1	ld/st
1	ADDI Rx,Rx,#8	ADDI Ry,Ry,#8	LD F2,0(Rx)
2			LD F4,0(Ry)
3	SUB R20,R4,Rx		
4			
5	MULTD F2,F0,F2		
6		ADDD F4,F0,F4	
7			
8			
9			
10	DIVD F8,F2,F0		SD F4,0(Ry)
11			
12			
...			
20			
21	BNZ R20,Loop	ADDD F10,F8,F2	
22	(branch shadow)		

LD latency

MULTD latency

ADDD latency

DIVD latency

22 clock cycles total

Dynamic scheduling

- Do the same as before, however
  - Do not assume that all instructions are already in the reservation station
  - Instead, they are decoded and placed in the reservation station two at a time
  - Schedule the instructions for execution as before, looking only at the instructions available in the reservation station

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

0	1		2		3		4		5		...
	LD	**F2, 0(Rx)**	LD	F2, 0(Rx)	LD	F2, 0(Rx)	LD	F2, 0(Rx)	LD	F2, 0(Rx)	
	MULTD	**F2,F0,F2**	**MULTD**	**F2,F0,F2**	**MULTD**	**F2,F0,F2**	**MULTD**	**F2,F0,F2**	**MULTD**	**F2,F0,F2**	
	DIVD	F8,F2,F0	**DIVD**	**F8,F2,F0**	**DIVD**	**F8,F2,F0**	**DIVD**	**F8,F2,F0**	**DIVD**	**F8,F2,F0**	
	LD	F4, 0(Ry)	**LD**	**F4, 0(Ry)**	LD	F4, 0(Ry)	LD	F4, 0(Ry)	LD	F4, 0(Ry)	
	ADDD	F4,F0,F4	ADDD	F4,F0,F4	**ADDD**	**F4,F0,F4**	**ADDD**	**F4,F0,F4**	**ADDD**	**F4,F0,F4**	
	ADDD	F10,F8,F2	ADDD	F10,F8,F2	**ADDD**	**F10,F8,F2**	**ADDD**	**F10,F8,F2**	**ADDD**	**F10,F8,F2**	
	SD	F4,0(Ry)	SD	F4,0(Ry)	SD	F4,0(Ry)	**SD**	**F4,0(Ry)**	**SD**	**F4,0(Ry)**	
	ADDI	Rx,Rx,#8	ADDI	Rx,Rx,#8	ADDI	Rx,Rx,#8	**ADDI**	**Rx,Rx,#8**	ADDI	Rx,Rx,#8	
	ADDI	Ry,Ry,#8	ADDI	Ry,Ry,#8	ADDI	Ry,Ry,#8	ADDI	Ry,Ry,#8	**ADDI**	**Ry,Ry,#8**	
	SUB	R20,R4,Rx	SUB	R20,R4,Rx	SUB	R20,R4,Rx	SUB	R20,R4,Rx	**SUB**	**R20,R4,Rx**	
	BNZ	20,Loop	BNZ	20,Loop	BNZ	20,Loop	BNZ	20,Loop	BNZ	20,Loop	

First 2 instructions appear in RS

Candidates for dispatch

		alu0	alu1	ld/st
Clock cycle	1			LD F2,0(Rx)
	2			LD F4,0(Ry)
	3			
	4	ADDI Rx,Rx,#8		
	5	MULTD F2,F0,F2	ADDI Ry,Ry,#8	
	6	SUB R20,R4,Rx	ADDD F4,F0,F4	
	7			
	8			
	9			
	10	DIVD F8,F2,F0		SD F4,0(Ry)
	11			
	12			
	...			
	20			
	21	BNZ R20,Loop	ADDD F10,F8,F2	
	22	(Branch shadow)		

Figure L.16 Candidates for dispatch.

How to improve performance

- What performance improvement would you get by adding each of the following
  - another ALU
  - another LD/ST unit
  - Full bypassing of ALU result to following operation
  - Cutting the latency of the longest running instruction in half (hint: the DIVD)

Cycle op was dispatched to FU

Clock cycle	alu0	alu1	ld/st
1	ADDI Rx,Rx,#8	ADDI Ry,Ry,#8	LD F2,0(Rx)
2			LD F4,0(Ry)
3	SUB R20,R4,Rx		
4			
5	MULTD F2,F0,F2		
6		ADDD F4,F0,F4	
7			
8			
9			
10	DIVD F8,F2,F0		SD F4,0(Ry)
11			
12			
...			
20			
21	BNZ R20,Loop	ADDD F10,F8,F2	
22			

LD latency

MULTD latency

ADDD latency

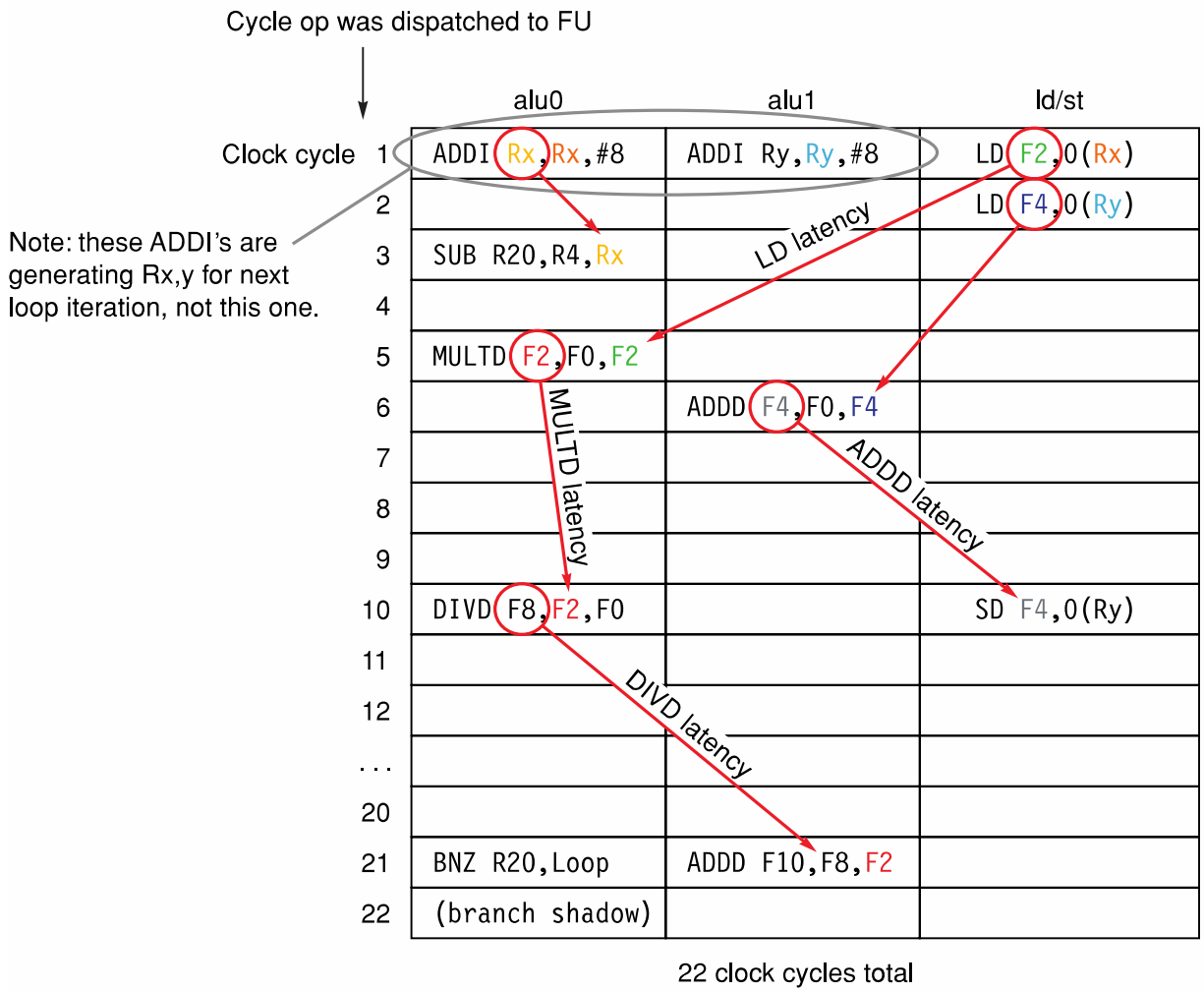
DIVD latency

21 clock cycles total

1. Another ALU: 0% improvement.
2. Another LD/ST unit: 0% improvement.
3. Full bypassing: critical path is `LD -> Mult -> Div -> ADDD`. Bypassing would save 1 cycle from latency of each, so 3 cycles total.
4. Cutting longest latency in half: divider is longest at 10 cycles. This would save 5 cycles total.

What if we unroll loop twice?

- Execution not limited by functional units



What if we unroll loop twice?

- Execution not limited by functional units
- Still executes in roughly the same number of clock cycles (fill in the blanks!)

Cycle op was dispatched to FU

Clock cycle	alu0	alu1	ld/st
1	ADDI Rx,Rx,#8	ADDI Ry,Ry,#8	LD F2,0(Rx)
2			LD F2,0(Rx)
3	SUB R20,R4,Rx		LD F4,0(Ry)
4			
5	MULTD F2,F0,F2		
6	MULTD F2,F0,F2		
7		ADDD F4,F0,F4	
8			
9			
10	DIVD F8,F2,F0		
11	DIVD F8,F2,F0		SD F4,0(Ry)
12			
. . .			
20			
21		ADDD F10,F8,F2	
22	BNZ R20,Loop	ADDD F10,F8,F2	

ADDD latency

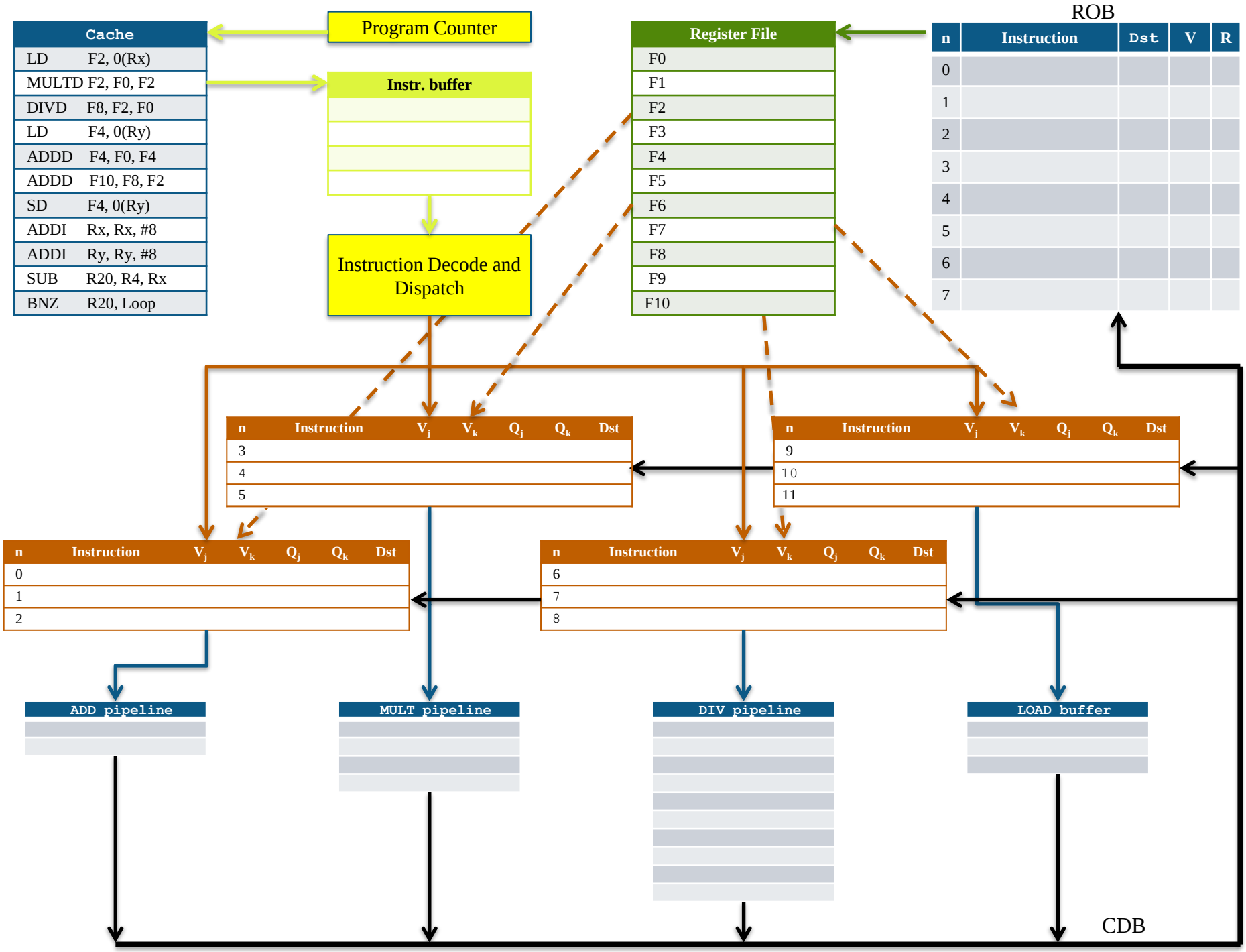
21 clock cycles total

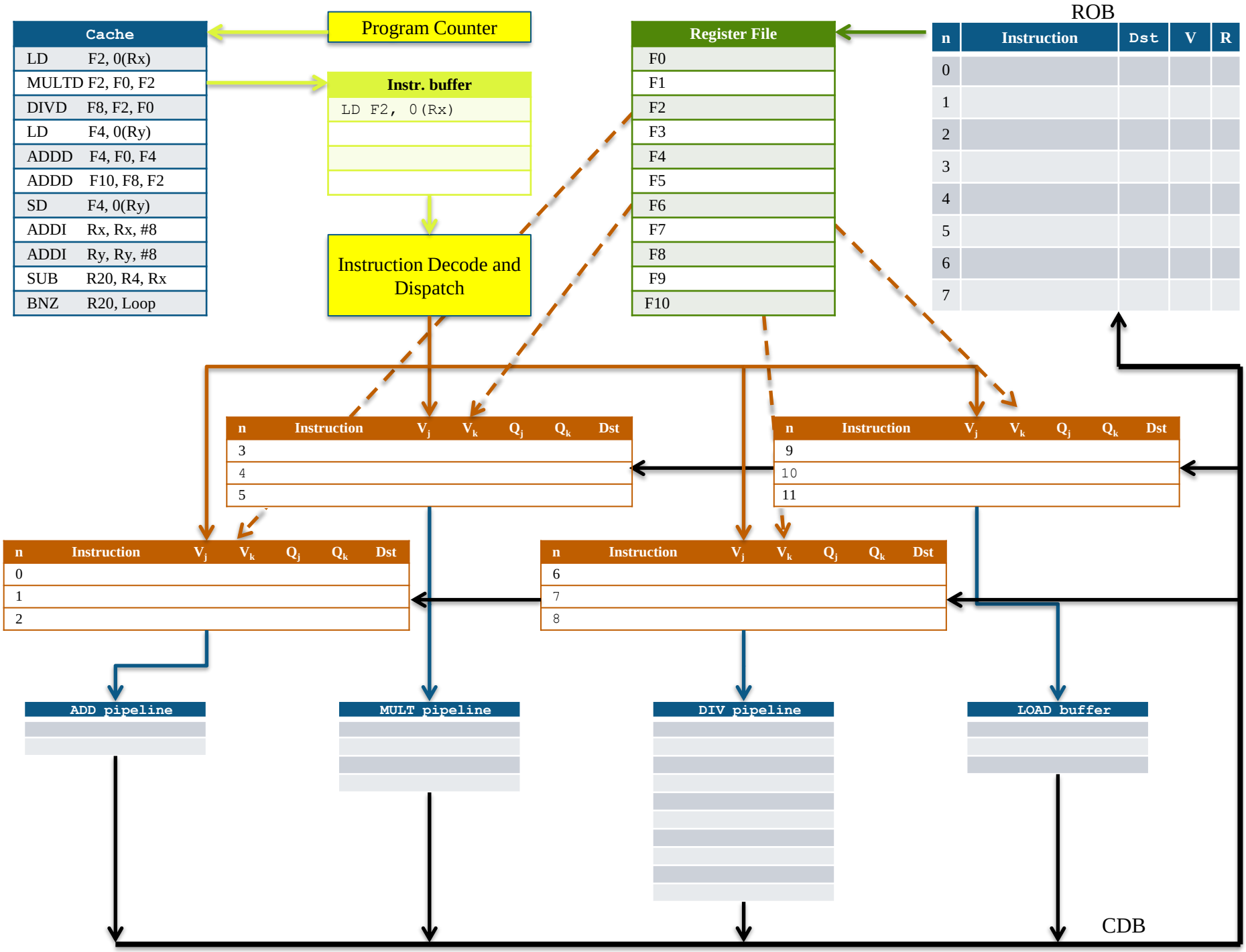
Speculation

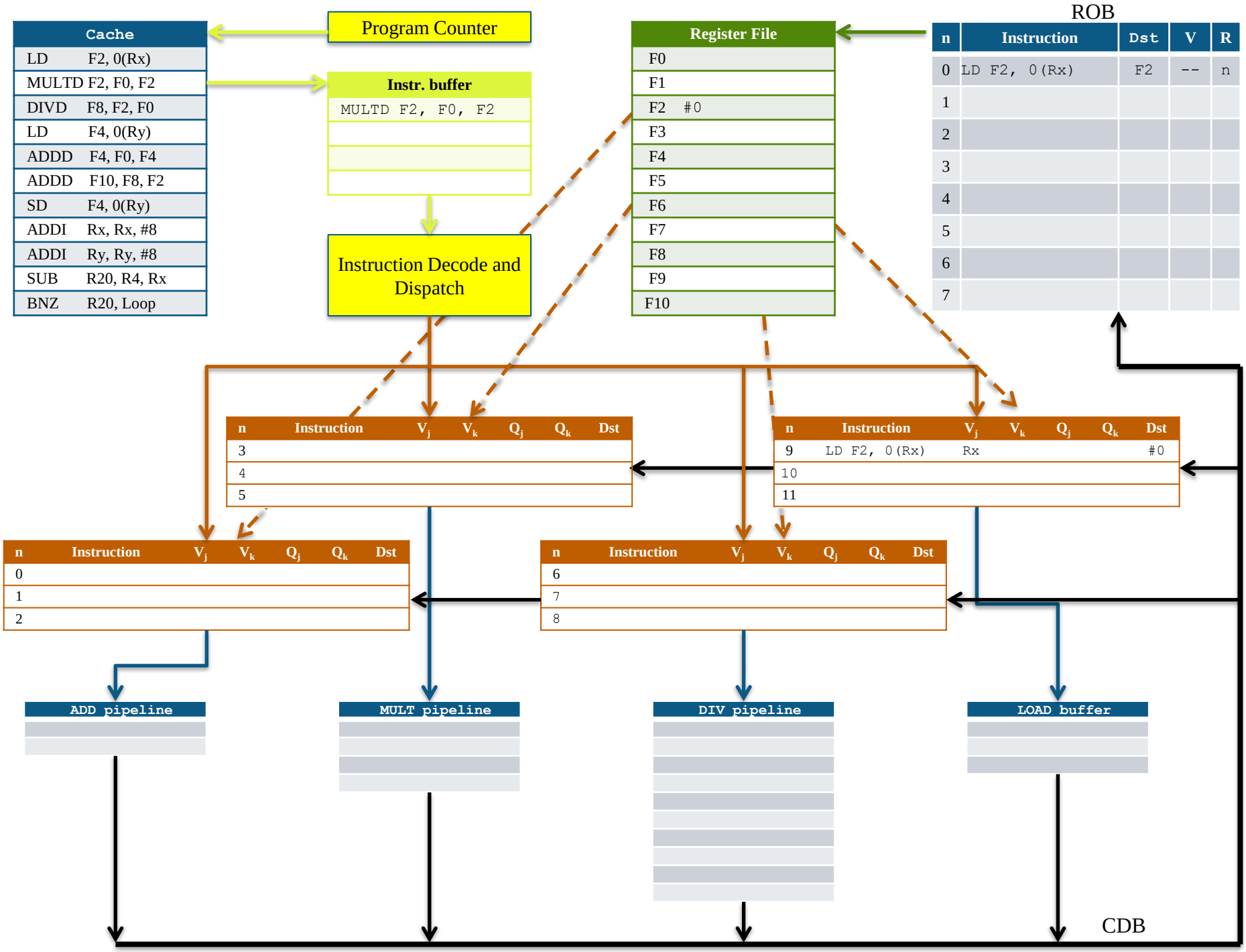
- We can execute the second iteration of the loop earlier if we are reasonably sure the branch is going to be taken
  - Run the loop speculatively assuming a single-issue Tomasulo architecture with reorder buffer and pipelined functional units
  - Decide whether to rename registers explicitly or not
  - Use the original instruction order
  - Then do the same using the rescheduled code
  - Then do the same using rescheduled loop unrolled code
  - Then do the same assuming a two-issue architecture

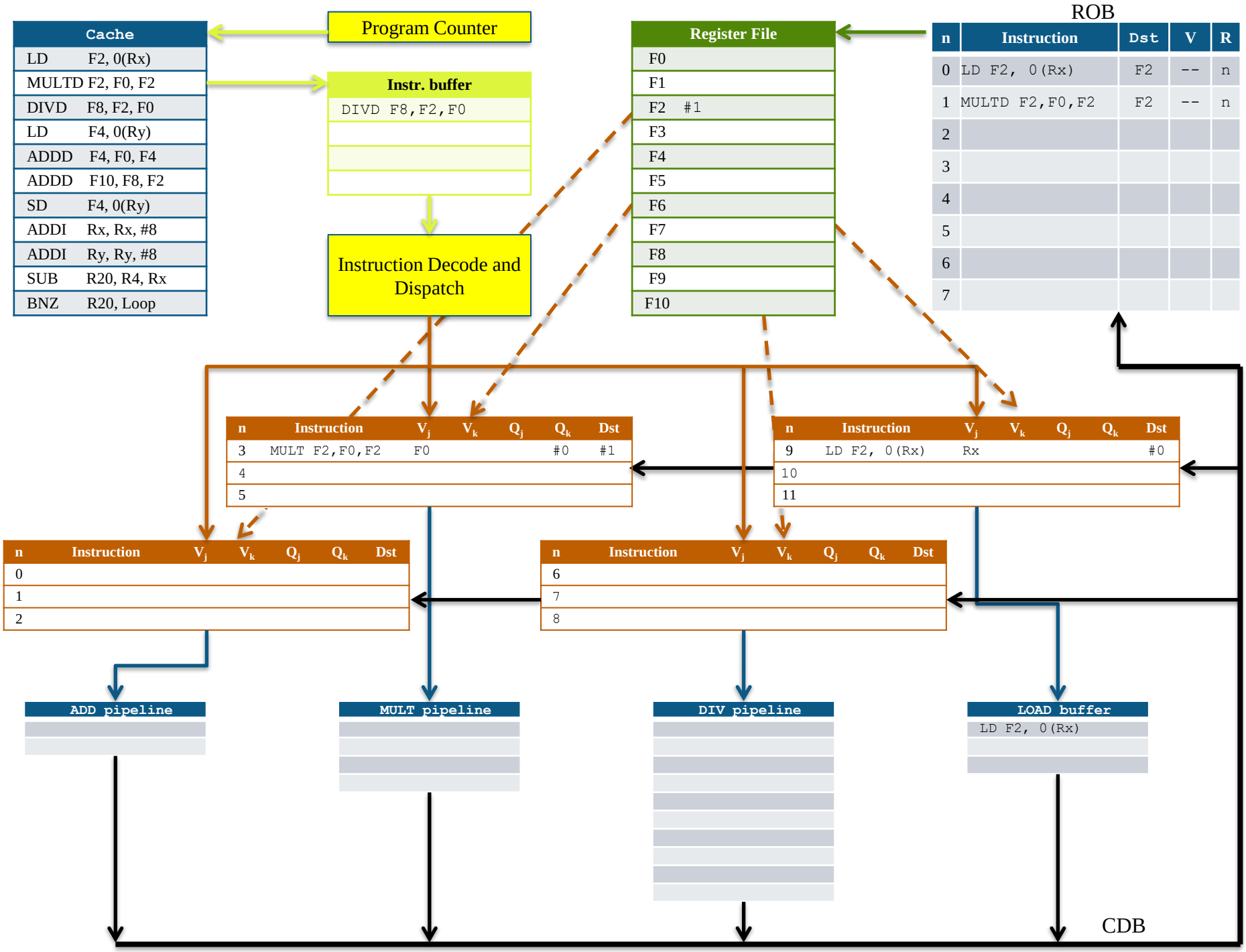
```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

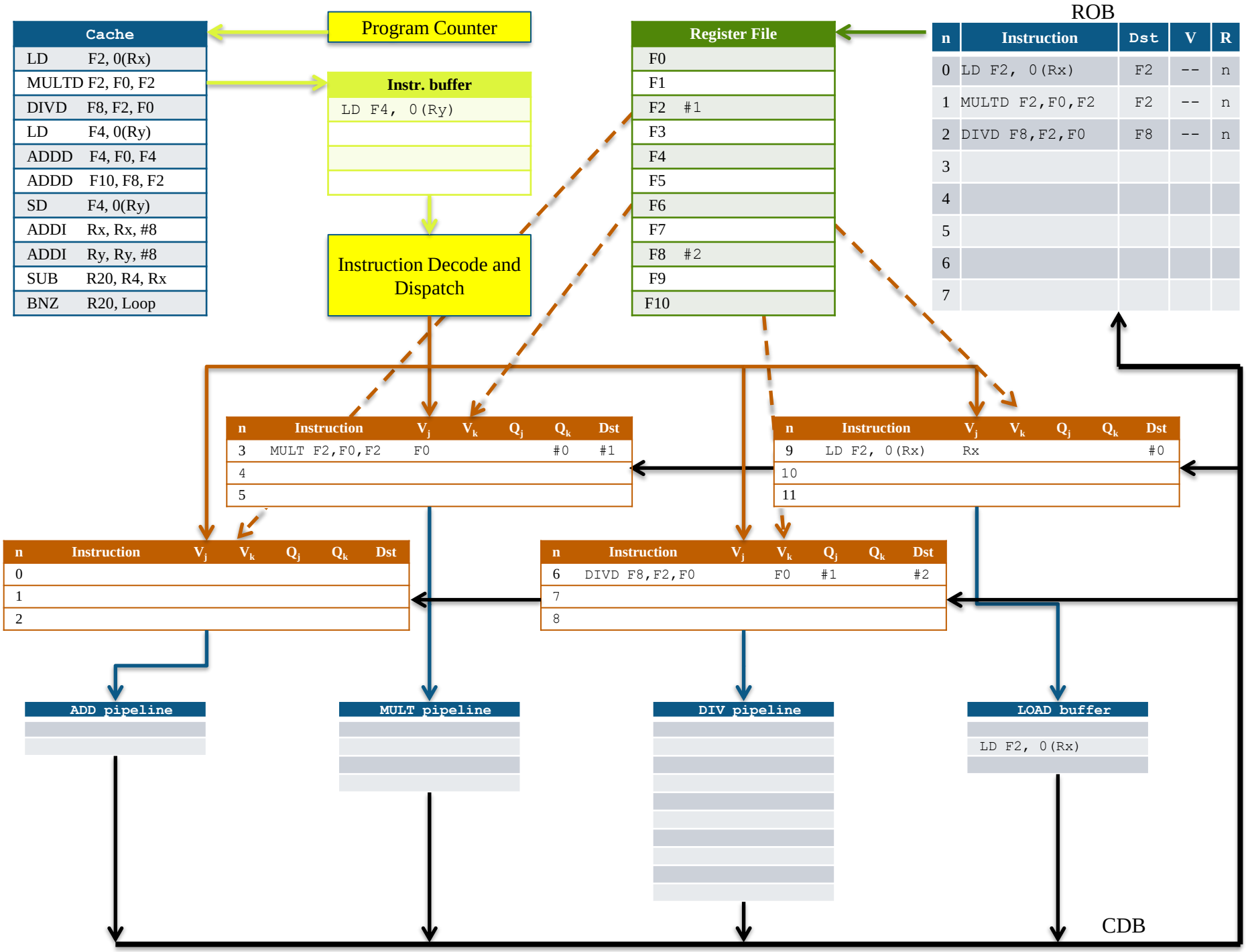
Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

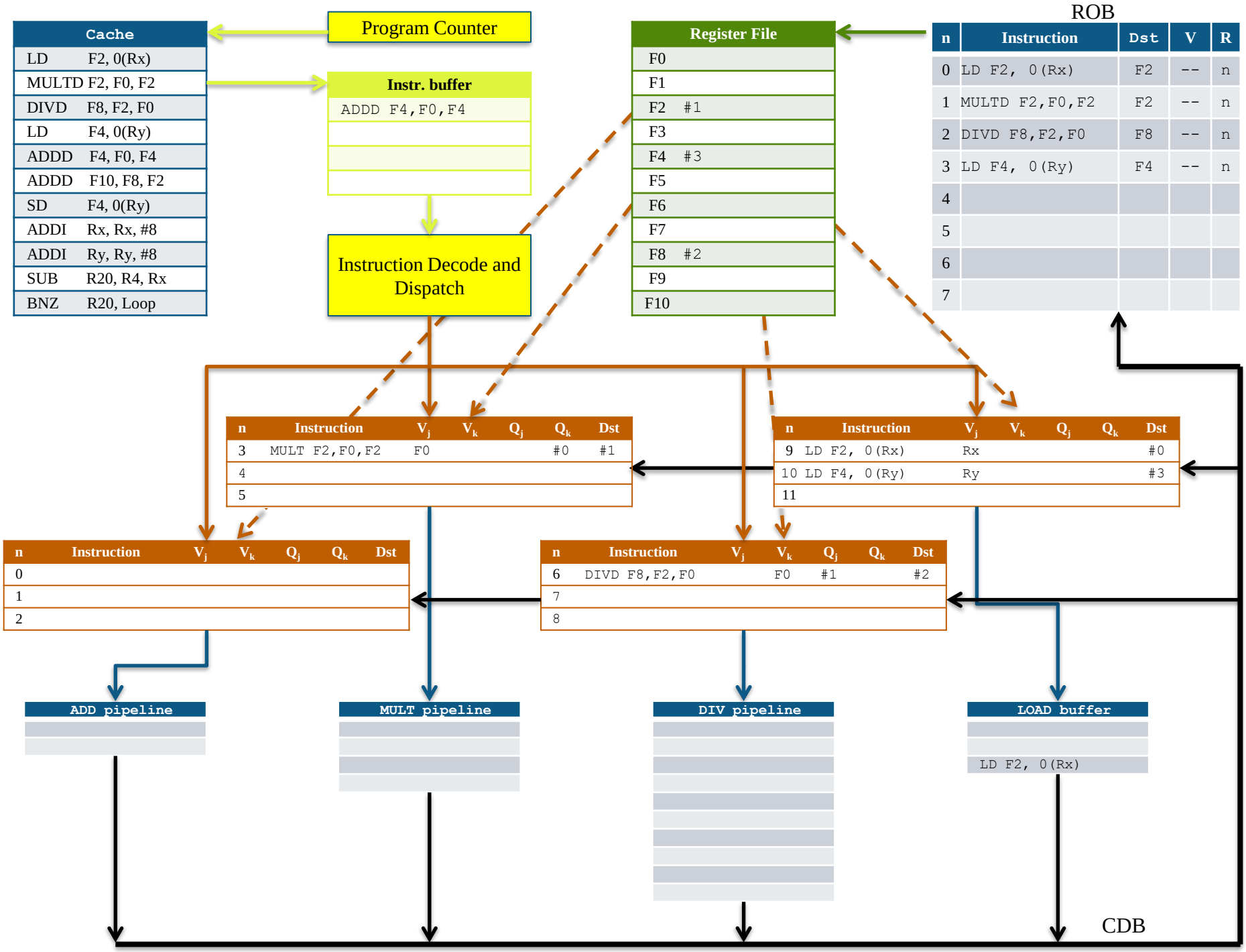

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
Instruction Decode and Dispatch
Register File
F0
F1
F2
F3
F4
F5
F6
F7
F8
F9
F10
ROB
n Instruction Dst V R
0
1
2
3
4
5
6
7
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9
10
11
n Instruction Vj Vk Qj Qk Dst
0
1
2
n Instruction Vj Vk Qj Qk Dst
6
7
8
ADD pipeline
MULT pipeline
DIV pipeline
LOAD buffer
CDB

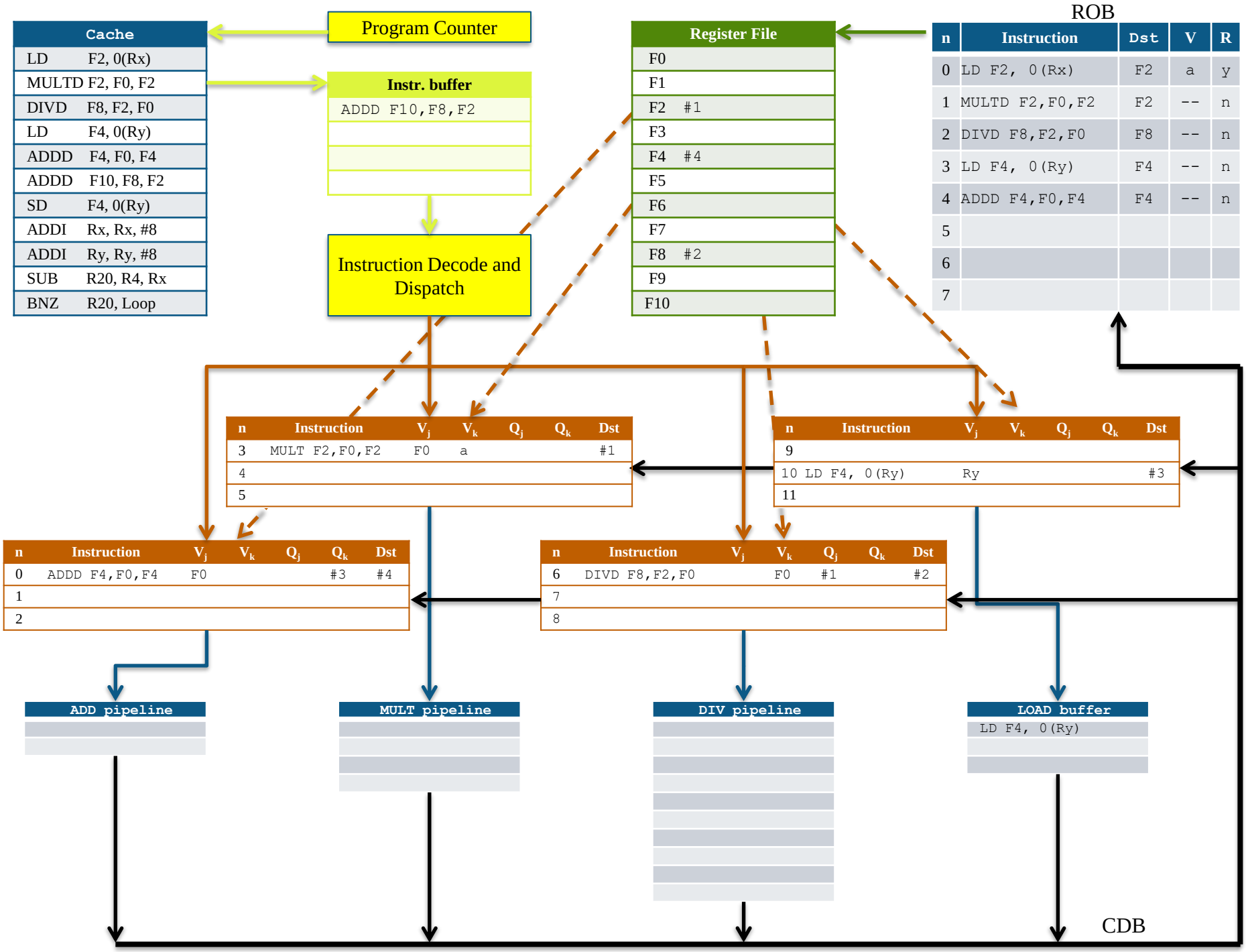

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F2, 0(Rx)
Instruction Decode and Dispatch
Register File
F0
F1
F2
F3
F4
F5
F6
F7
F8
F9
F10
ROB
n Instruction Dst V R
0
1
2
3
4
5
6
7
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9
10
11
n Instruction Vj Vk Qj Qk Dst
0
1
2
n Instruction Vj Vk Qj Qk Dst
6
7
8
ADD pipeline
MULT pipeline
DIV pipeline
LOAD buffer
CDB

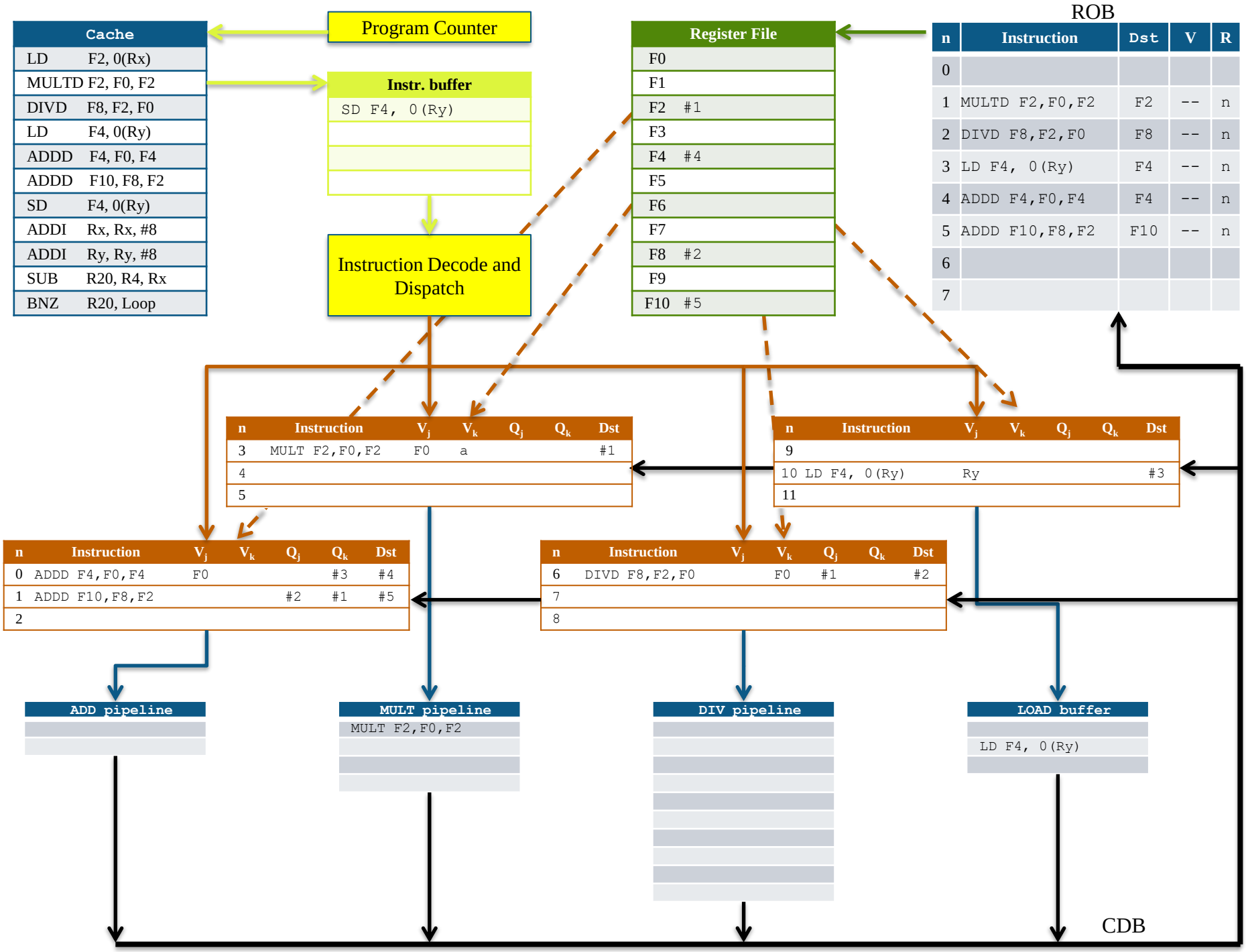

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
MULTD F2, F0, F2
Instruction Decode and Dispatch
Register File
F0
F1
F2 #0
F3
F4
F5
F6
F7
F8
F9
F10
ROB
n Instruction Dst V R
0 LD F2, 0(Rx) F2 -- n
1
2
3
4
5
6
7
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 LD F2, 0(Rx) Rx #0
10
11
n Instruction Vj Vk Qj Qk Dst
0
1
2
n Instruction Vj Vk Qj Qk Dst
6
7
8
ADD pipeline
MULT pipeline
DIV pipeline
LOAD buffer
CDB

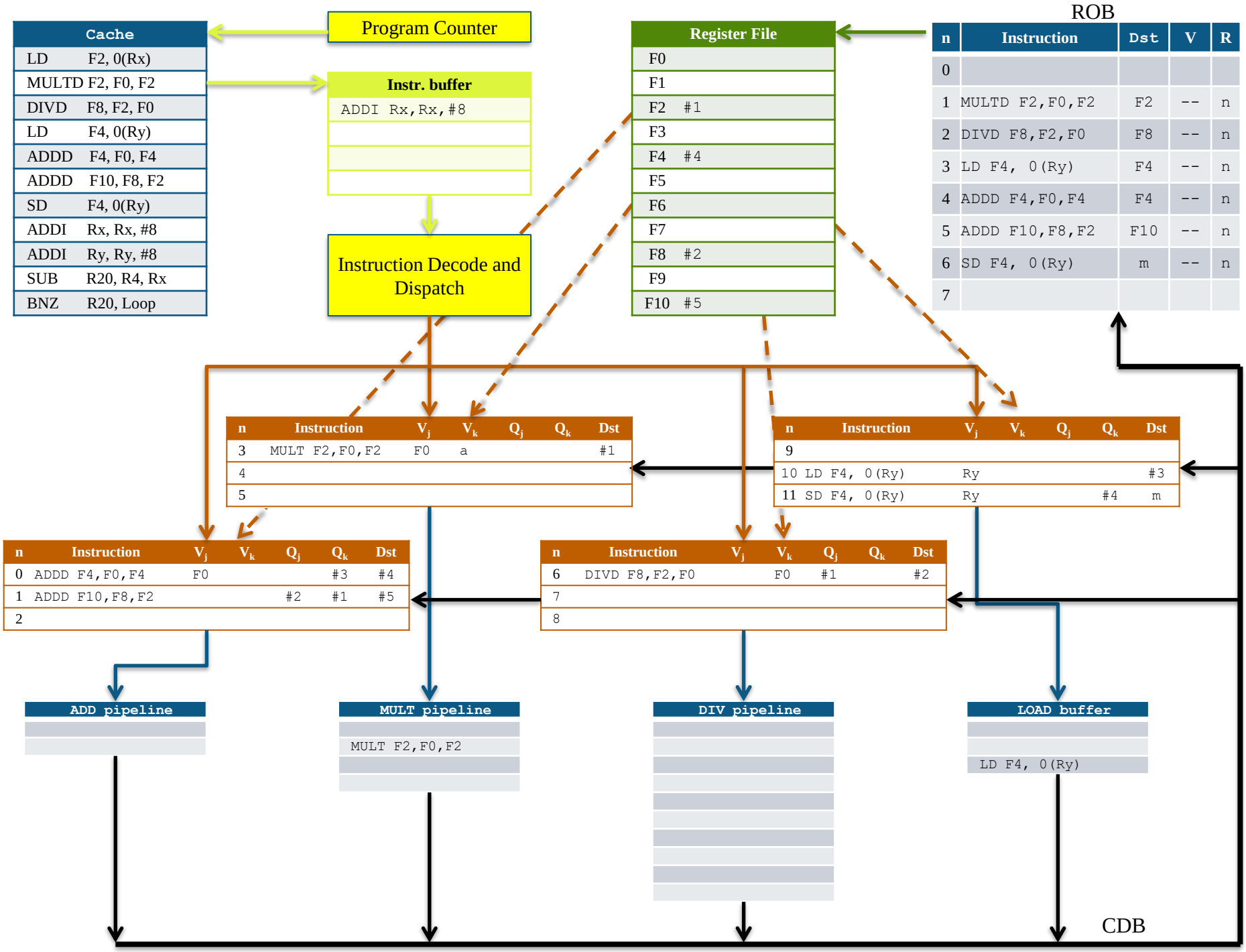

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
DIVD F8,F2,F0
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4
F5
F6
F7
F8
F9
F10
ROB
n Instruction Dst V R
0 LD F2, 0(Rx) F2 -- n
1 MULTD F2,F0,F2 F2 -- n
2
3
4
5
6
7
n Instruction Vj Vk Qj Qk Dst
3 MULT F2,F0,F2 F0 #0 #1
4
5
n Instruction Vj Vk Qj Qk Dst
9 LD F2, 0(Rx) Rx #0
10
11
n Instruction Vj Vk Qj Qk Dst
0
1
2
n Instruction Vj Vk Qj Qk Dst
6
7
8
ADD pipeline
MULT pipeline
DIV pipeline
LOAD buffer
LD F2, 0(Rx)
CDB

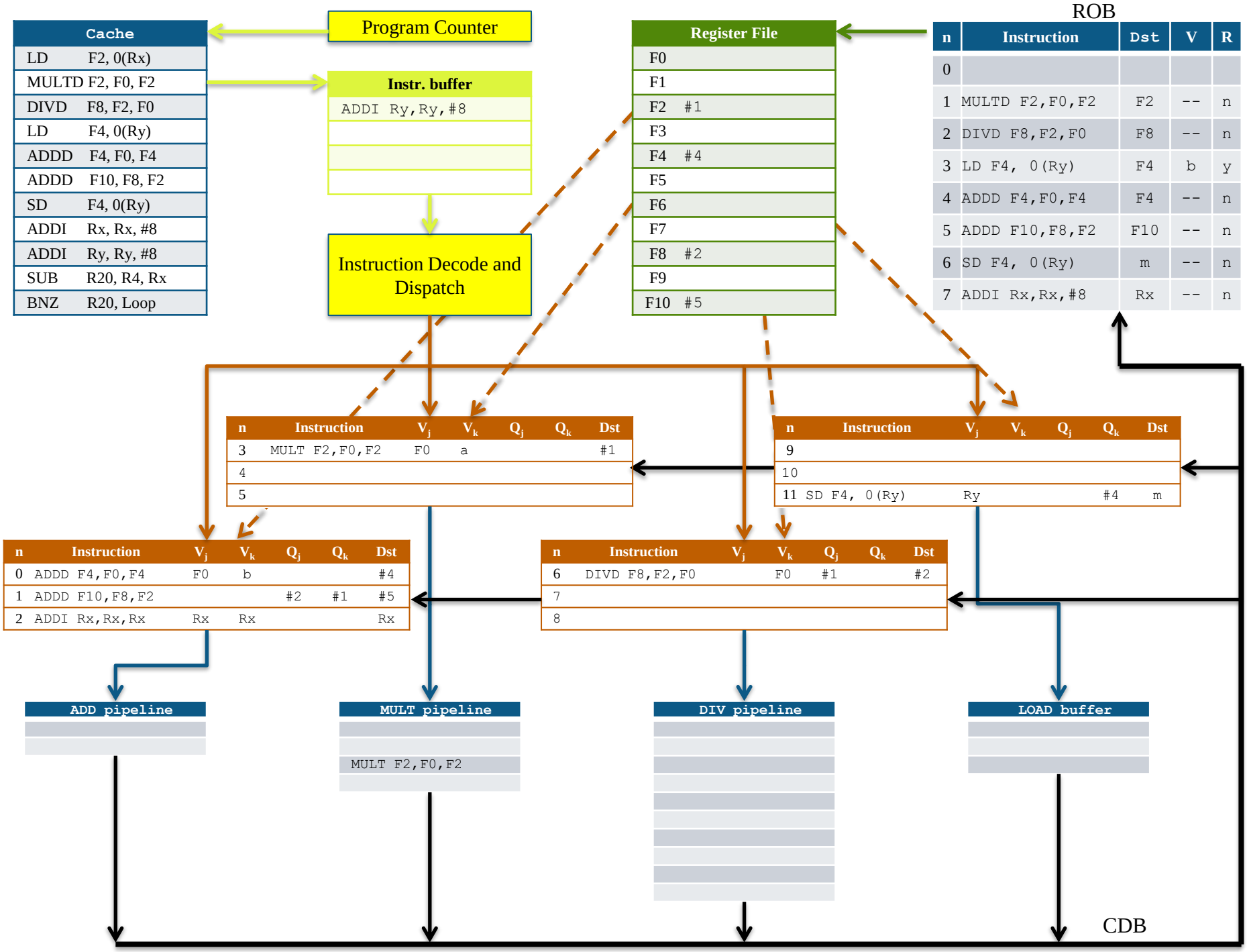

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F4, 0(Ry)
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4
F5
F6
F7
F8 #2
F9
F10
ROB
n Instruction Dst V R
0 LD F2, 0(Rx) F2 -- n
1 MULTD F2,F0,F2 F2 -- n
2 DIVD F8,F2,F0 F8 -- n
3
4
5
6
7
n Instruction Vj Vk Qj Qk Dst
3 MULT F2,F0,F2 F0 #0 #1
4
5
n Instruction Vj Vk Qj Qk Dst
9 LD F2, 0(Rx) Rx #0
10
11
n Instruction Vj Vk Qj Qk Dst
0
1
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #1 #2
7
8
ADD pipeline
MULT pipeline
DIV pipeline
LOAD buffer
LD F2, 0(Rx)
CDB

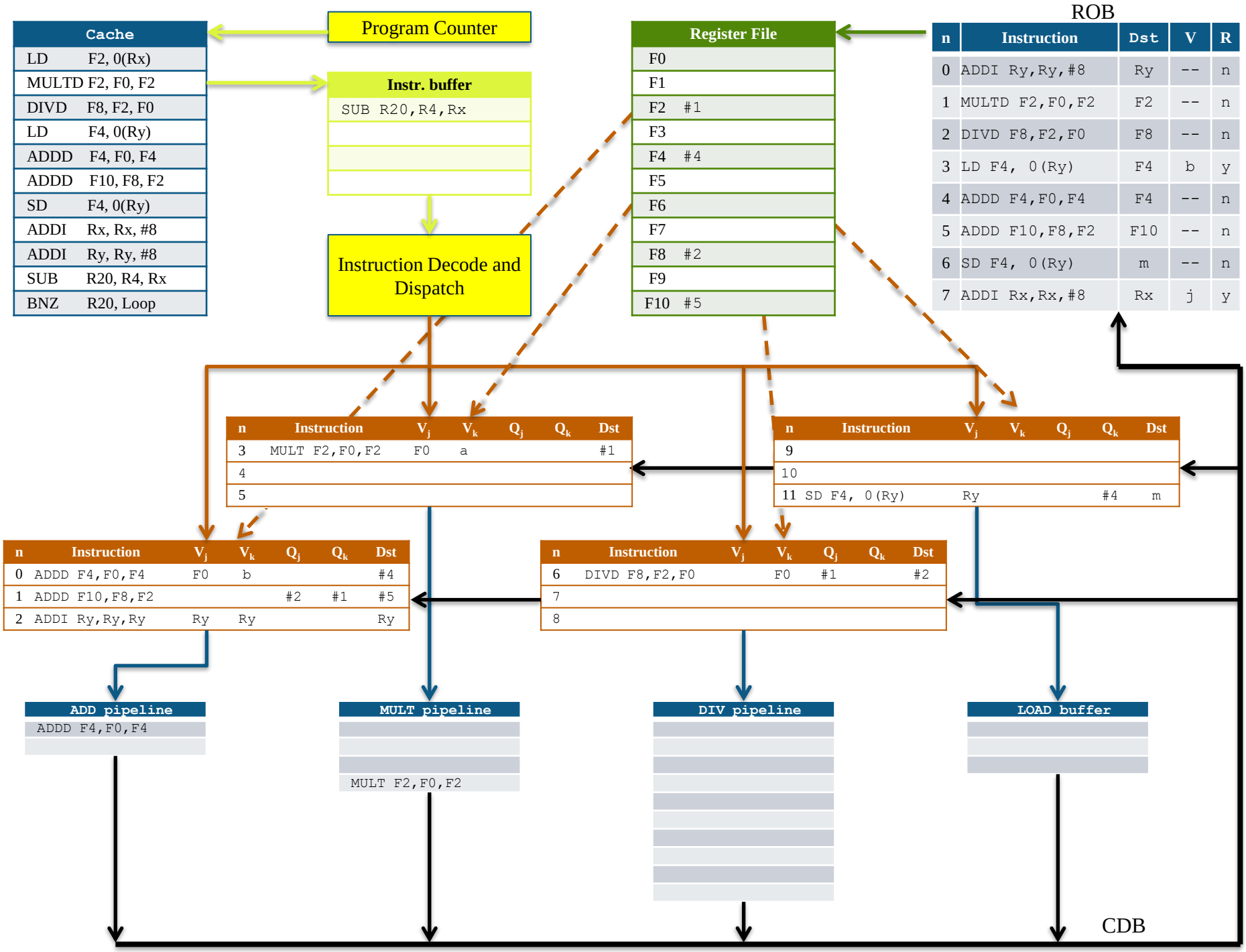

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDD F4,F0,F4
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4 #3
F5
F6
F7
F8 #2
F9
F10
ROB
n Instruction Dst V R
0 LD F2, 0(Rx) F2 -- n
1 MULTD F2,F0,F2 F2 -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 -- n
4
5
6
7
n Instruction Vj Vk Qj Qk Dst
3 MULT F2,F0,F2 F0 #0 #1
4
5
n Instruction Vj Vk Qj Qk Dst
9 LD F2, 0(Rx) Rx #0
10 LD F4, 0(Ry) Ry #3
11
n Instruction Vj Vk Qj Qk Dst
0
1
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #1 #2
7
8
ADD pipeline
MULT pipeline
DIV pipeline
LOAD buffer
LD F2, 0(Rx)
CDB

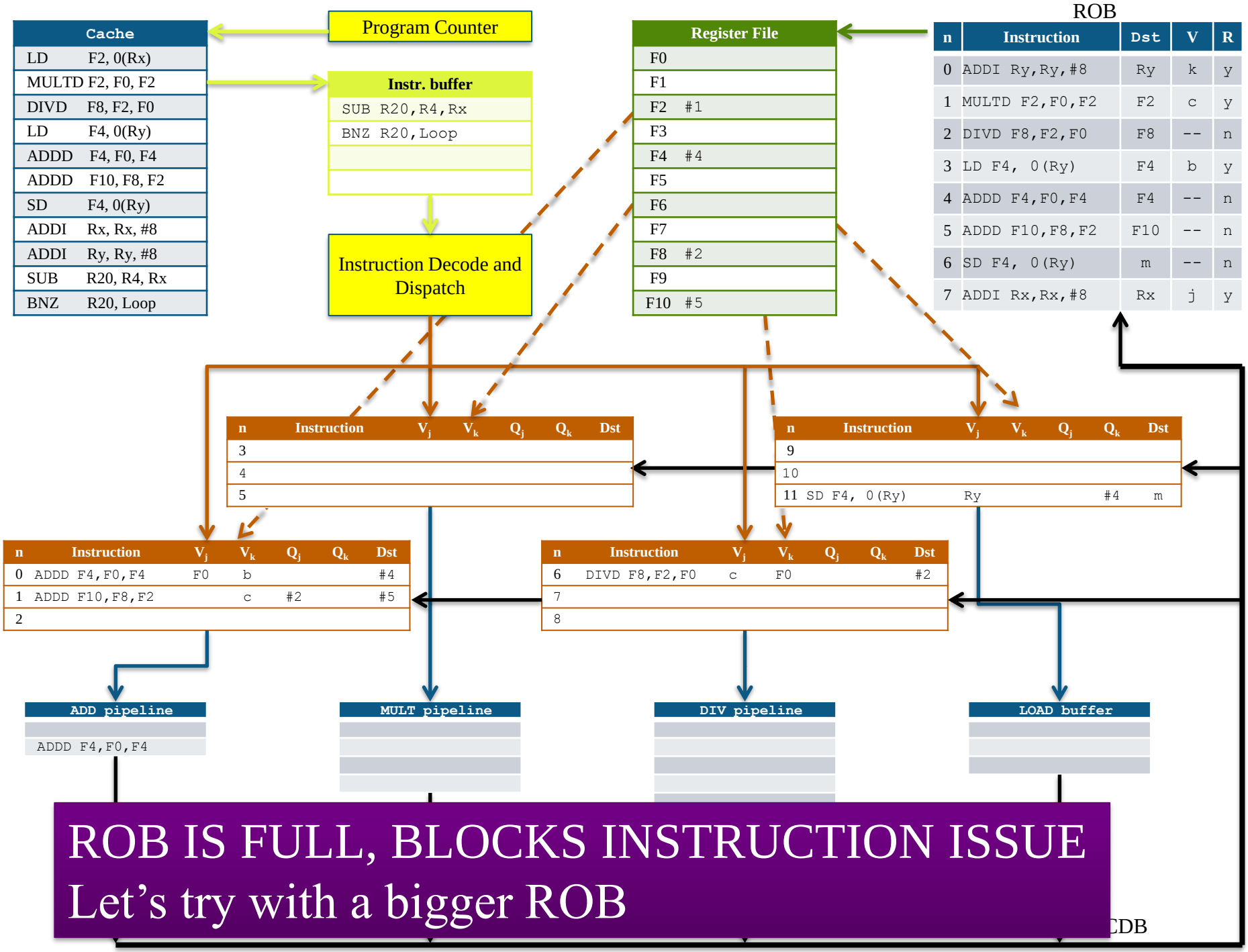

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDD F10,F8,F2
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10
ROB
n Instruction Dst V R
0 LD F2, 0(Rx) F2 a y
1 MULTD F2,F0,F2 F2 -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 -- n
4 ADDD F4,F0,F4 F4 -- n
5
6
7
n Instruction Vj Vk Qj Qk Dst
3 MULT F2,F0,F2 F0 a #1
4
5
n Instruction Vj Vk Qj Qk Dst
9
10 LD F4, 0(Ry) Ry #3
11
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #3 #4
1
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #1 #2
7
8
ADD pipeline
MULT pipeline
DIV pipeline
LOAD buffer
LD F4, 0(Ry)
CDB


Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
SD F4, 0(Ry)
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10 #5
ROB
n Instruction Dst V R
0
1 MULTD F2,F0,F2 F2 -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 -- n
4 ADDD F4,F0,F4 F4 -- n
5 ADDD F10,F8,F2 F10 -- n
6
7
n Instruction Vj Vk Qj Qk Dst
3 MULT F2,F0,F2 F0 a #1
4
5
n Instruction Vj Vk Qj Qk Dst
9
10 LD F4, 0(Ry) Ry #3
11
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #3 #4
1 ADDD F10,F8,F2 #2 #1 #5
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #1 #2
7
8
ADD pipeline
MULT pipeline
MULT F2,F0,F2
DIV pipeline
LOAD buffer
LD F4, 0(Ry)
CDB


Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Rx,Rx,#8
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10 #5
ROB
n Instruction Dst V R
0
1 MULTD F2,F0,F2 F2 -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 -- n
4 ADDD F4,F0,F4 F4 -- n
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m -- n
7
n Instruction Vj Vk Qj Qk Dst
3 MULT F2,F0,F2 F0 a #1
4
5
n Instruction Vj Vk Qj Qk Dst
9
10 LD F4, 0(Ry) Ry #3
11 SD F4, 0(Ry) Ry #4 m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #3 #4
1 ADDD F10,F8,F2 #2 #1 #5
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #1 #2
7
8
ADD pipeline
MULT pipeline
MULT F2,F0,F2
DIV pipeline
LOAD buffer
LD F4, 0(Ry)
CDB

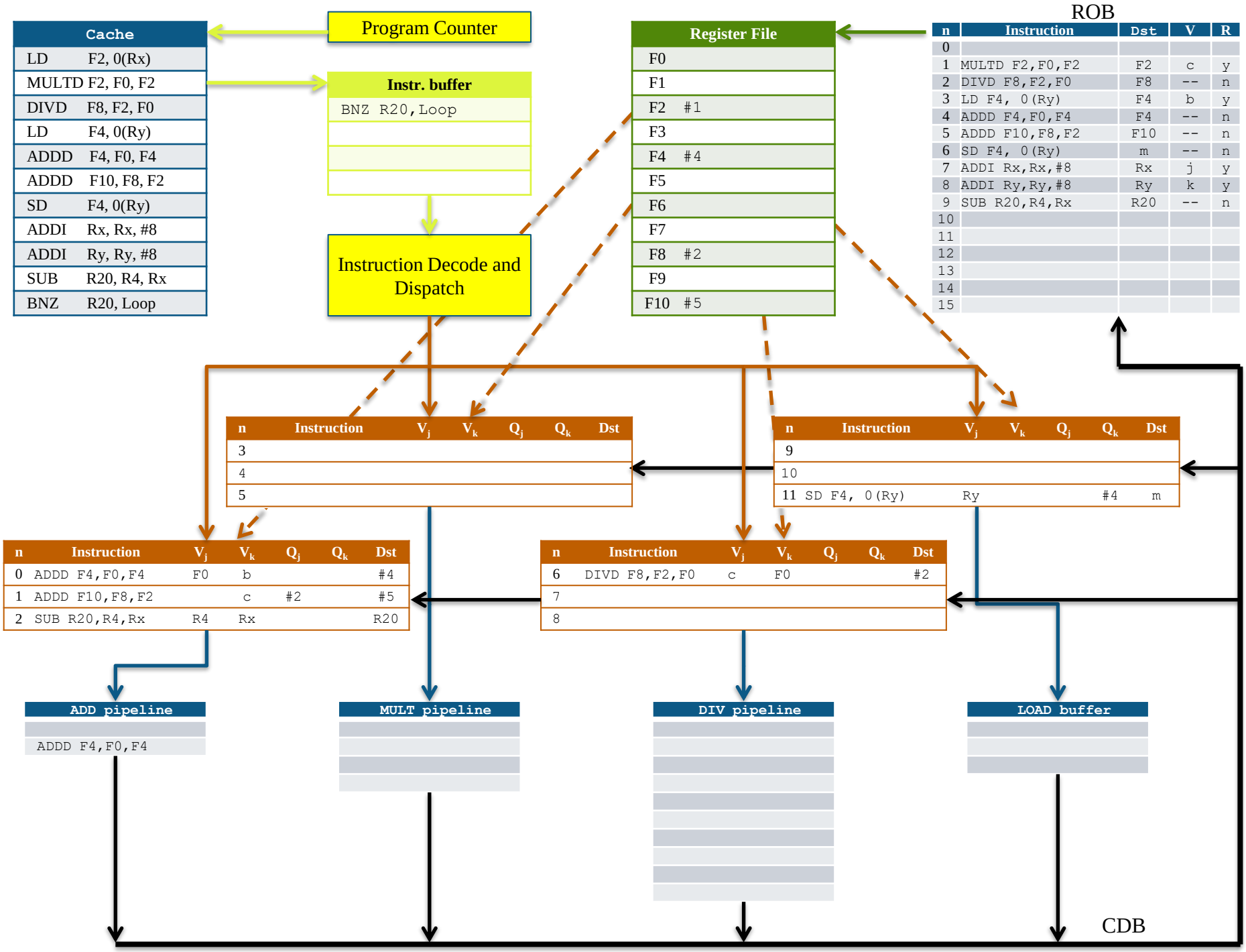

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Ry,Ry,#8
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10 #5
ROB
n Instruction Dst V R
0
1 MULTD F2,F0,F2 F2 -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 -- n
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m -- n
7 ADDI Rx,Rx,#8 Rx -- n
n Instruction Vj Vk Qj Qk Dst
3 MULT F2,F0,F2 F0 a #1
4
5
n Instruction Vj Vk Qj Qk Dst
9
10
11 SD F4, 0(Ry) Ry #4 m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 b #4
1 ADDD F10,F8,F2 #2 #1 #5
2 ADDI Rx,Rx,Rx Rx Rx Rx
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #1 #2
7
8
ADD pipeline
MULT pipeline
MULT F2,F0,F2
DIV pipeline
LOAD buffer
CDB

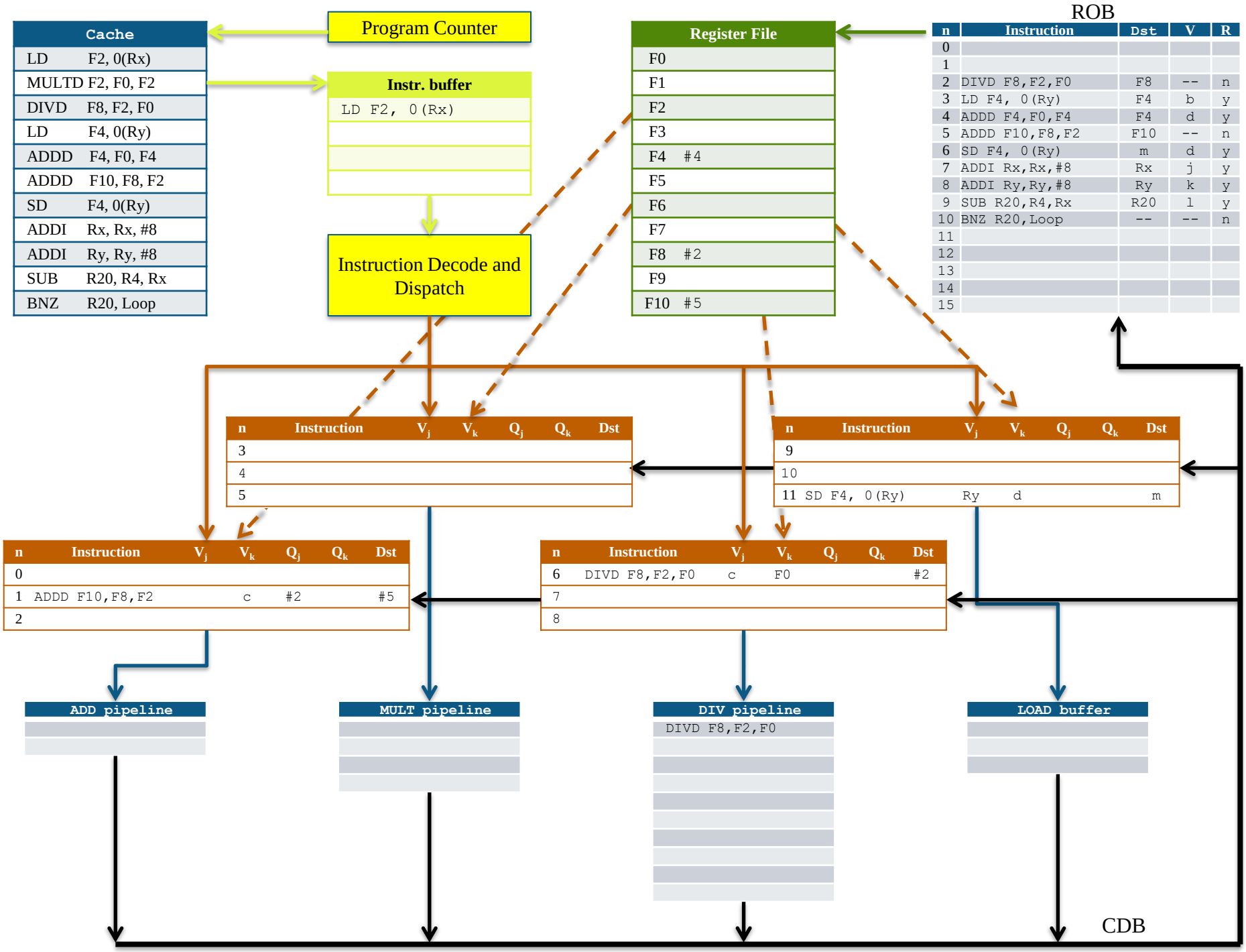

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
SUB R20,R4,Rx
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10 #5
ROB
n Instruction Dst V R
0 ADDI Ry,Ry,#8 Ry -- n
1 MULTD F2,F0,F2 F2 -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 -- n
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m -- n
7 ADDI Rx,Rx,#8 Rx j y
n Instruction Vj Vk Qj Qk Dst
3 MULT F2,F0,F2 F0 a #1
4
5
n Instruction Vj Vk Qj Qk Dst
9
10
11 SD F4, 0(Ry) Ry #4 m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 b #4
1 ADDD F10,F8,F2 #2 #1 #5
2 ADDI Ry,Ry,Ry Ry Ry Ry
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #1 #2
7
8
ADD pipeline
ADDD F4,F0,F4
MULT pipeline
MULT F2,F0,F2
DIV pipeline
LOAD buffer
CDB

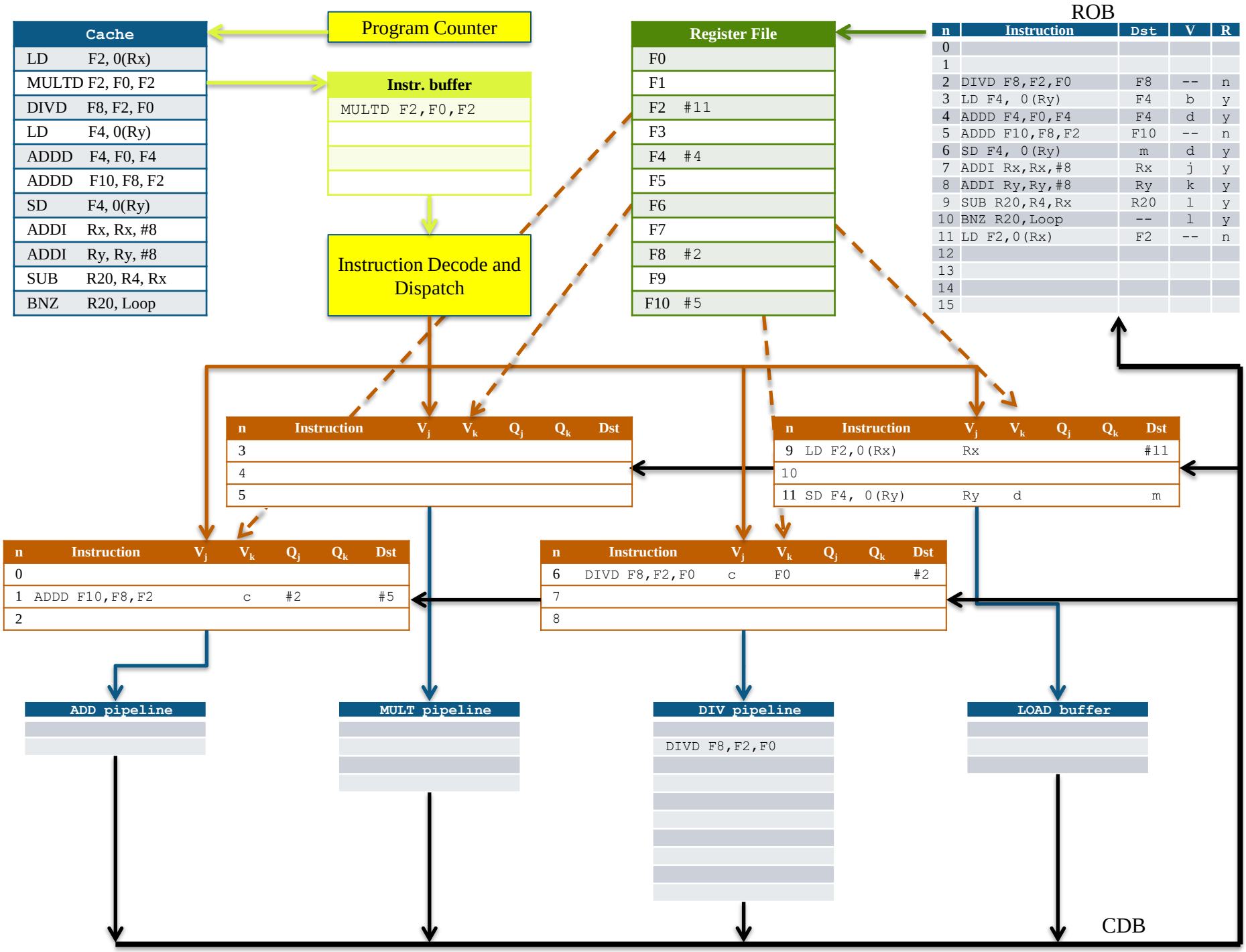


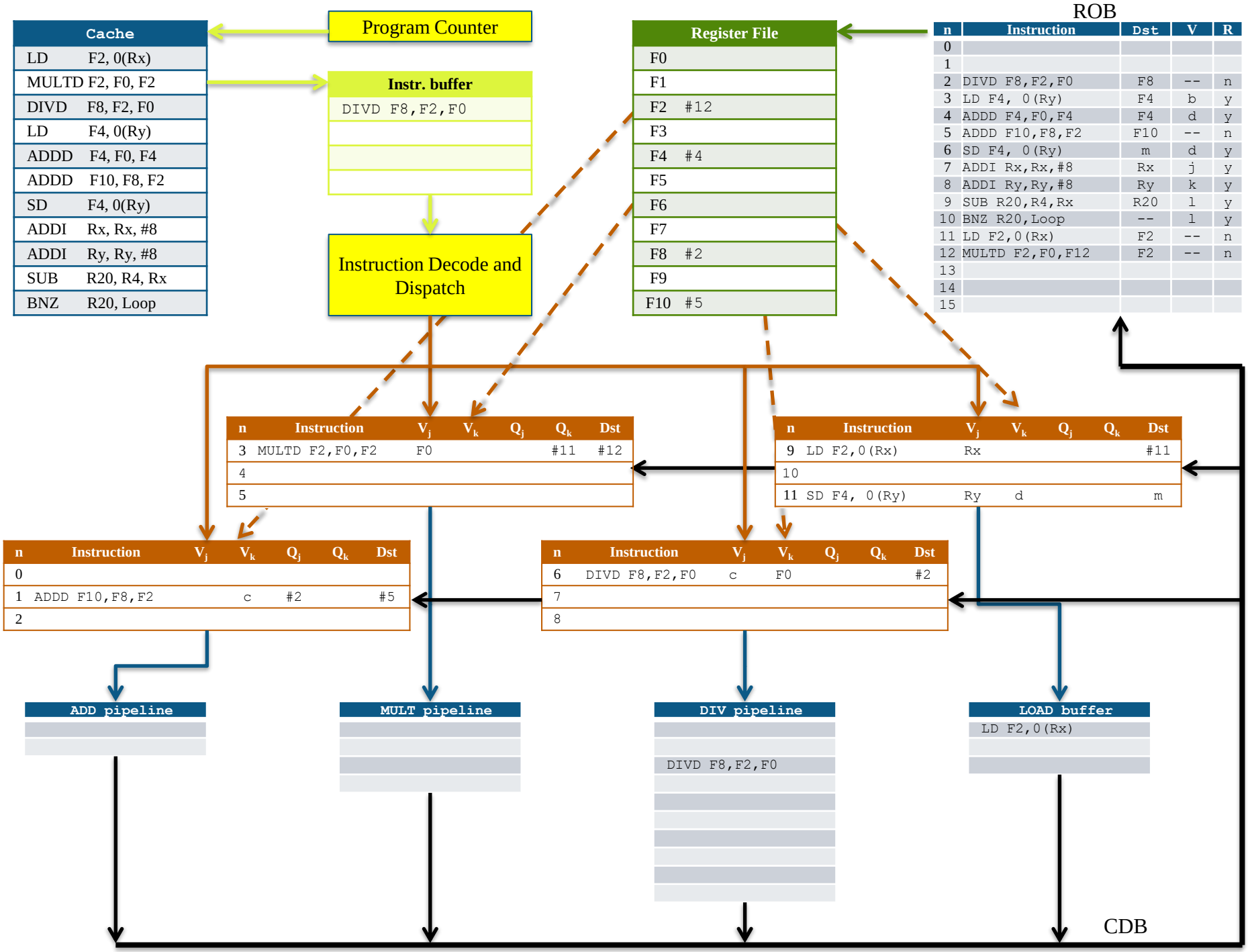
ROB IS FULL, BLOCKS INSTRUCTION ISSUE

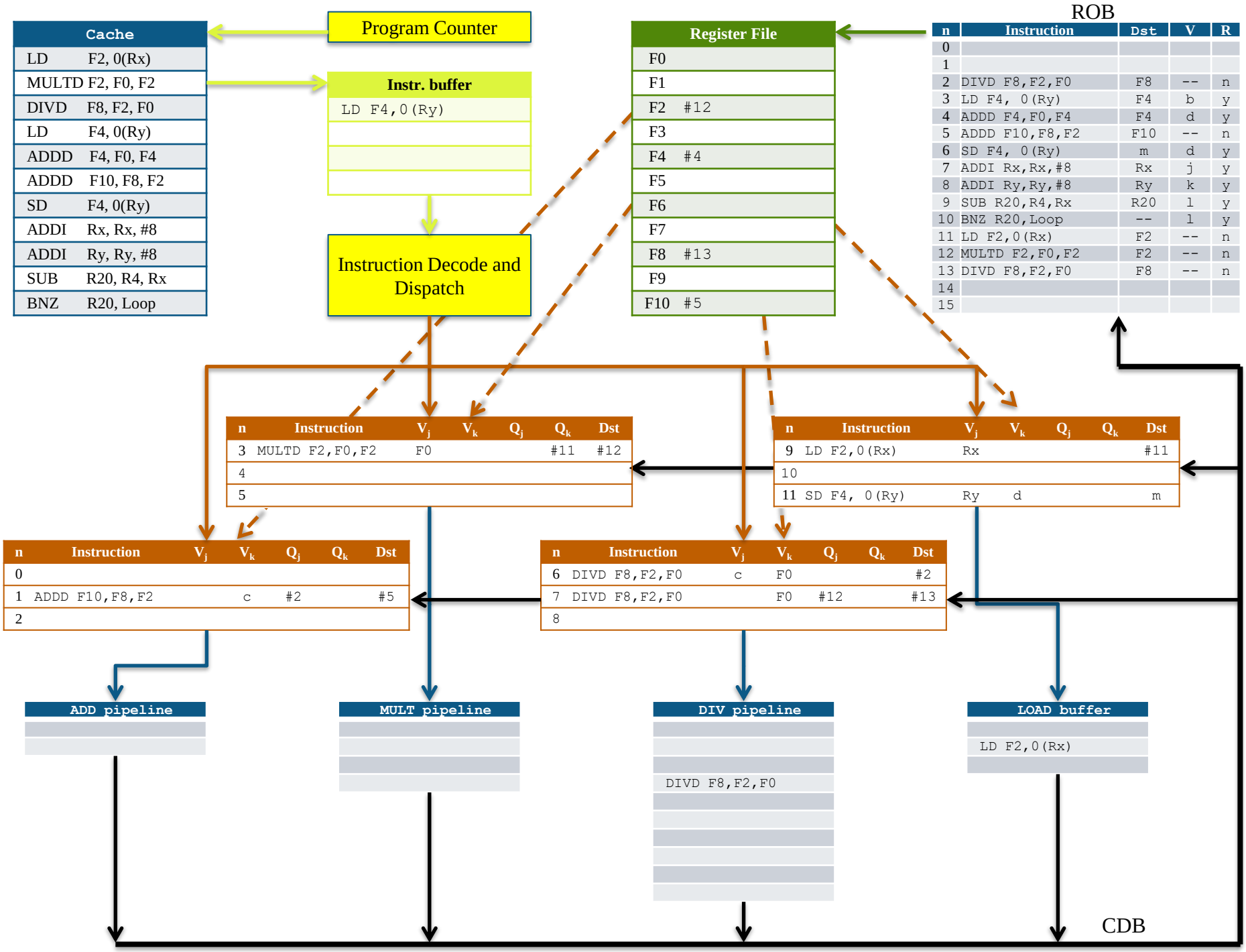
Let's try with a bigger ROB

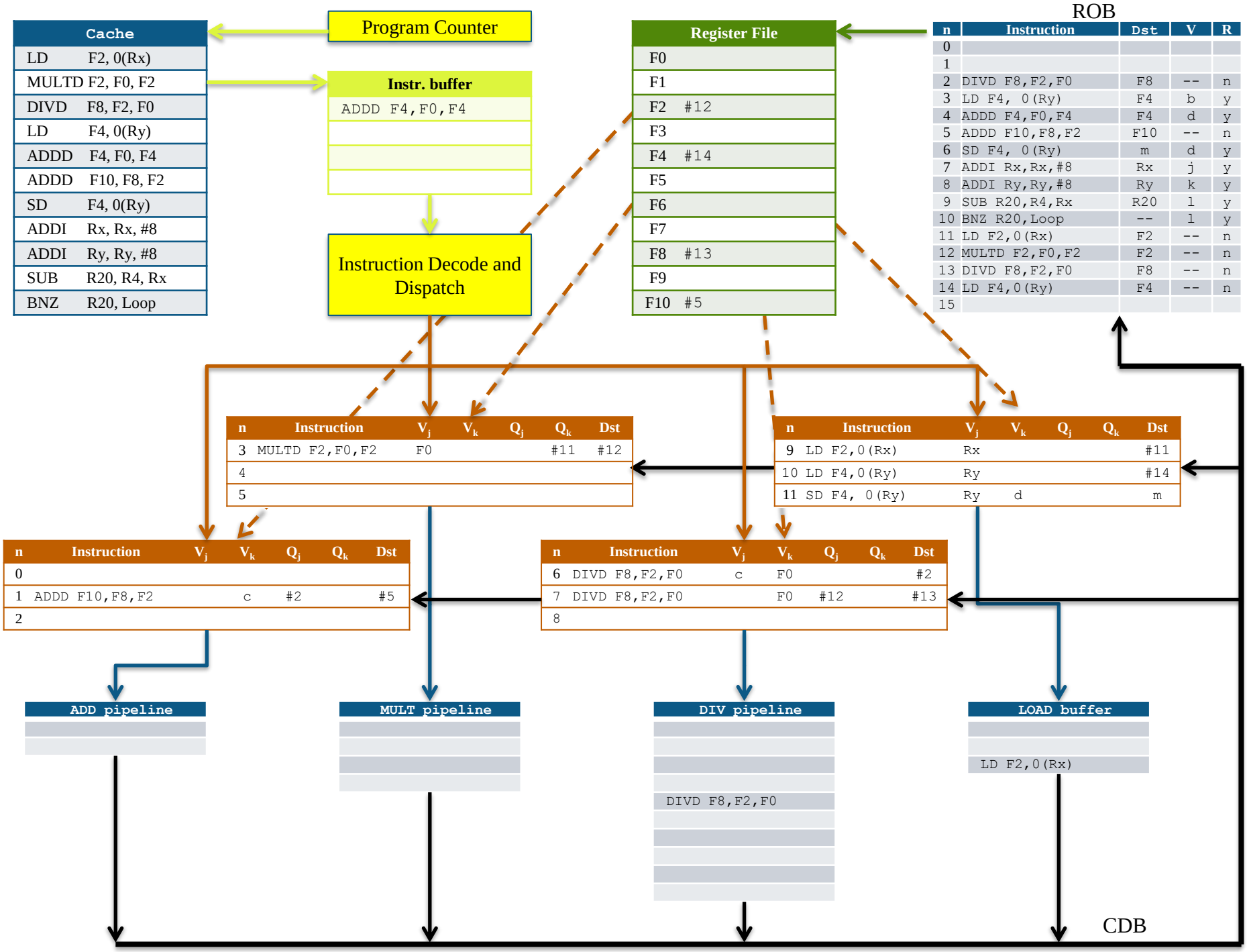

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
BNZ R20,Loop
Instruction Decode and Dispatch
Register File
F0
F1
F2 #1
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10 #5
ROB
n Instruction Dst V R
0
1 MULTD F2,F0,F2 F2 c y
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 -- n
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m -- n
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 -- n
10
11
12
13
14
15
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9
10
11 SD F4, 0(Ry) Ry #4 m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 b #4
1 ADDD F10,F8,F2 c #2 #5
2 SUB R20,R4,Rx R4 Rx R20
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7
8
ADD pipeline
ADDD F4,F0,F4
MULT pipeline
DIV pipeline
LOAD buffer
CDB

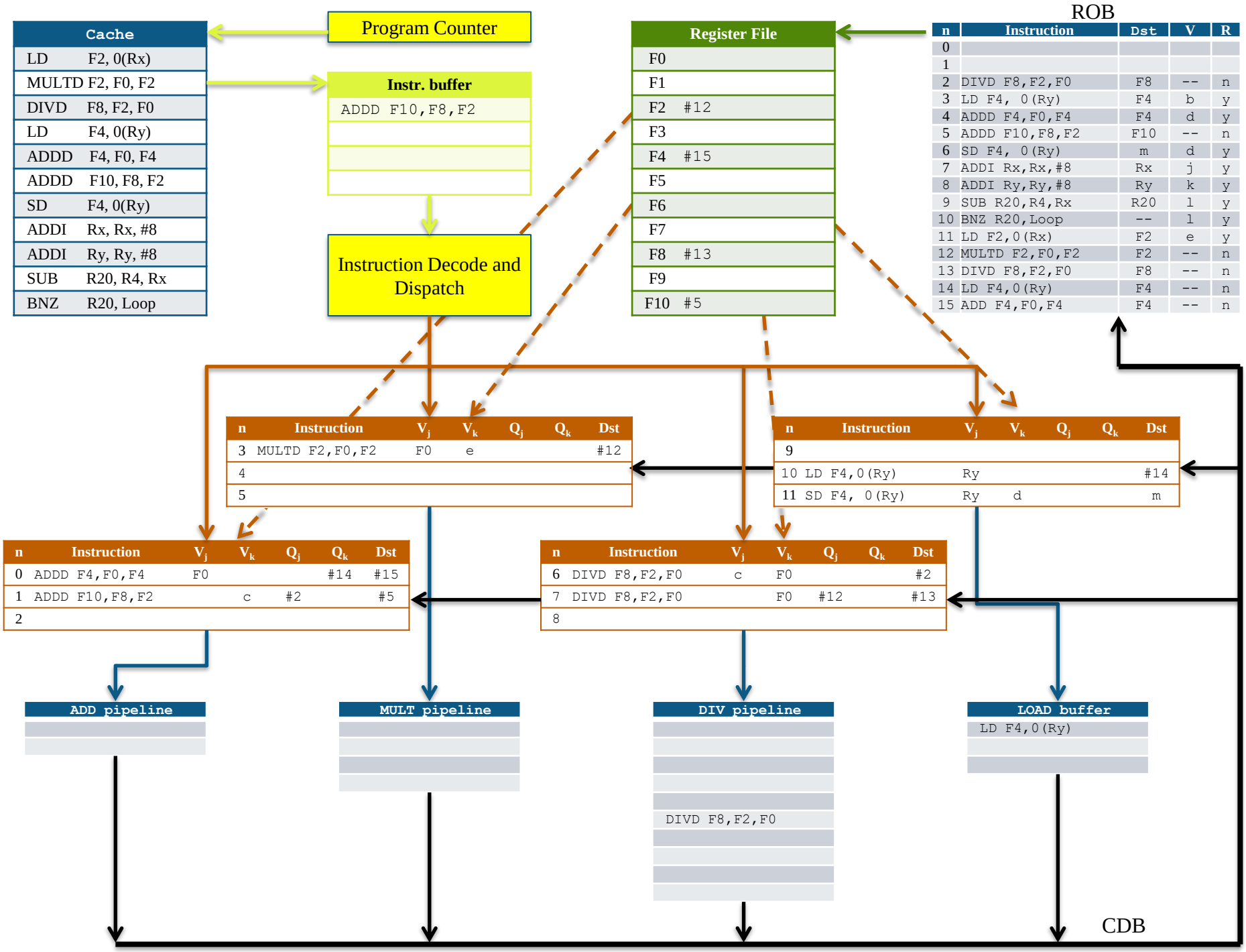

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F2, 0(Rx)
Instruction Decode and Dispatch
Register File
F0
F1
F2
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10 #5
ROB
n Instruction Dst V R
0
1
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- -- n
11
12
13
14
15
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0
1 ADDD F10,F8,F2 c #2 #5
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

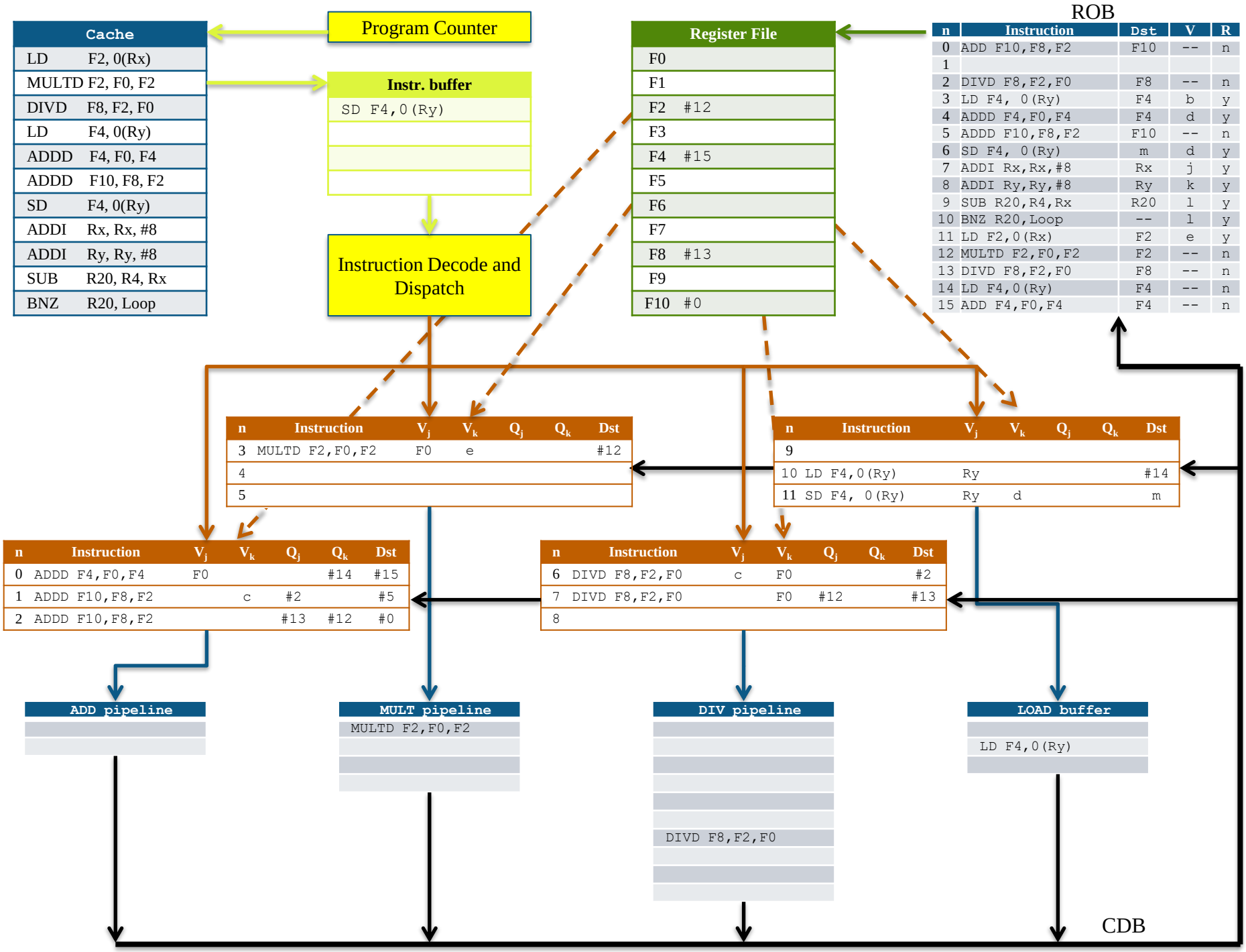

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
MULTD F2,F0,F2
Instruction Decode and Dispatch
Register File
F0
F1
F2 #11
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10 #5
ROB
n Instruction Dst V R
0
1
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 -- n
12
13
14
15
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 LD F2,0(Rx) Rx #11
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0
1 ADDD F10,F8,F2 c #2 #5
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

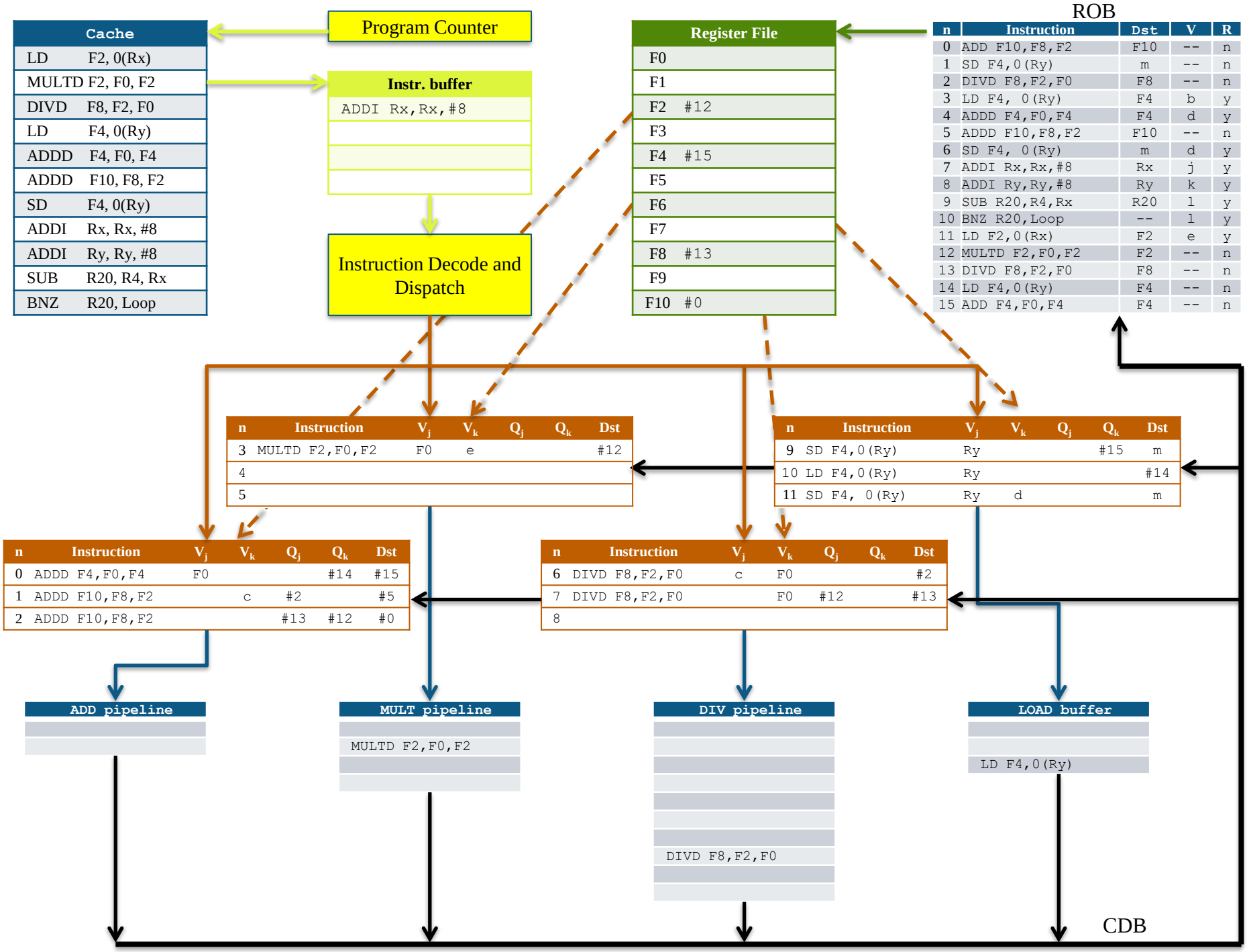

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
DIVD F8,F2,F0
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #4
F5
F6
F7
F8 #2
F9
F10 #5
ROB
n Instruction Dst V R
0
1
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 -- n
12 MULTD F2,F0,F12 F2 -- n
13
14
15
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 #11 #12
4
5
n Instruction Vj Vk Qj Qk Dst
9 LD F2,0(Rx) Rx #11
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0
1 ADDD F10,F8,F2 c #2 #5
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F2,0(Rx)
CDB

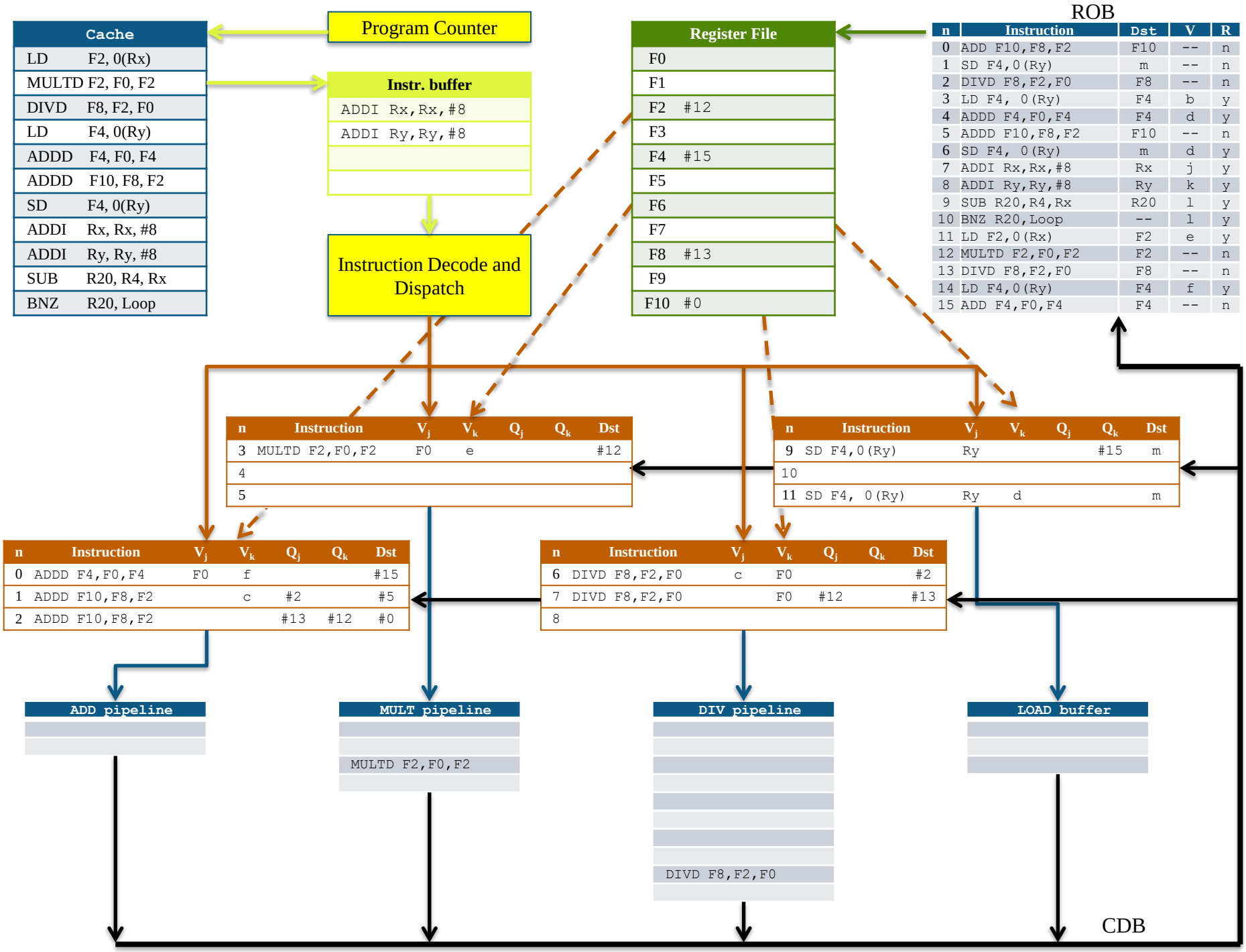

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F4,0(Ry)
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #4
F5
F6
F7
F8 #13
F9
F10 #5
ROB
n Instruction Dst V R
0
1
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 -- n
12 MULTD F2,F0,F2 F2 -- n
13 DIVD F8,F2,F0 F8 -- n
14
15
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 #11 #12
4
5
n Instruction Vj Vk Qj Qk Dst
9 LD F2,0(Rx) Rx #11
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0
1 ADDD F10,F8,F2 c #2 #5
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7 DIVD F8,F2,F0 F0 #12 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F2,0(Rx)
CDB

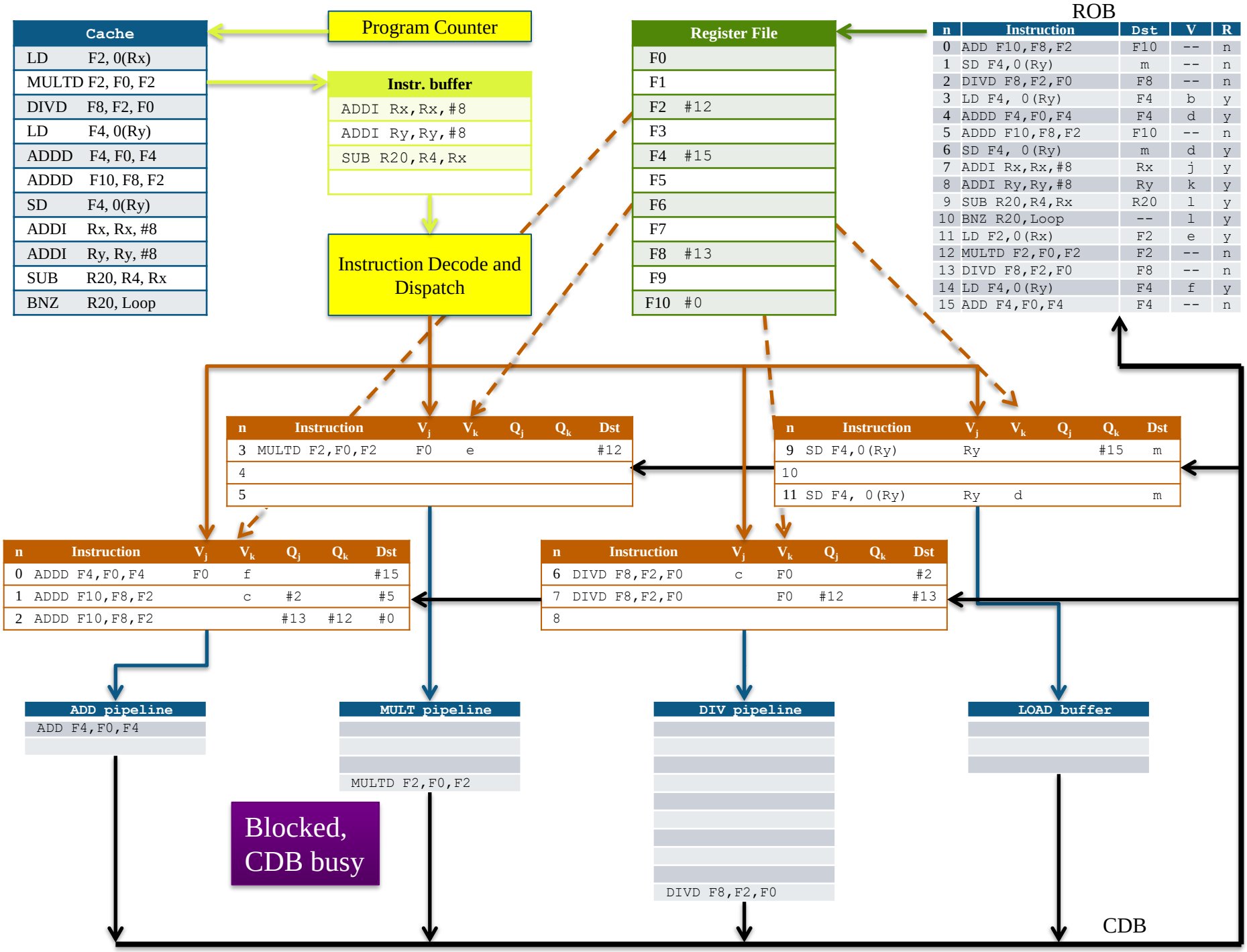

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDD F4,F0,F4
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #14
F5
F6
F7
F8 #13
F9
F10 #5
ROB
n Instruction Dst V R
0
1
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 -- n
12 MULTD F2,F0,F2 F2 -- n
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 -- n
15
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 #11 #12
4
5
n Instruction Vj Vk Qj Qk Dst
9 LD F2,0(Rx) Rx #11
10 LD F4,0(Ry) Ry #14
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0
1 ADDD F10,F8,F2 c #2 #5
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7 DIVD F8,F2,F0 F0 #12 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F2,0(Rx)
CDB

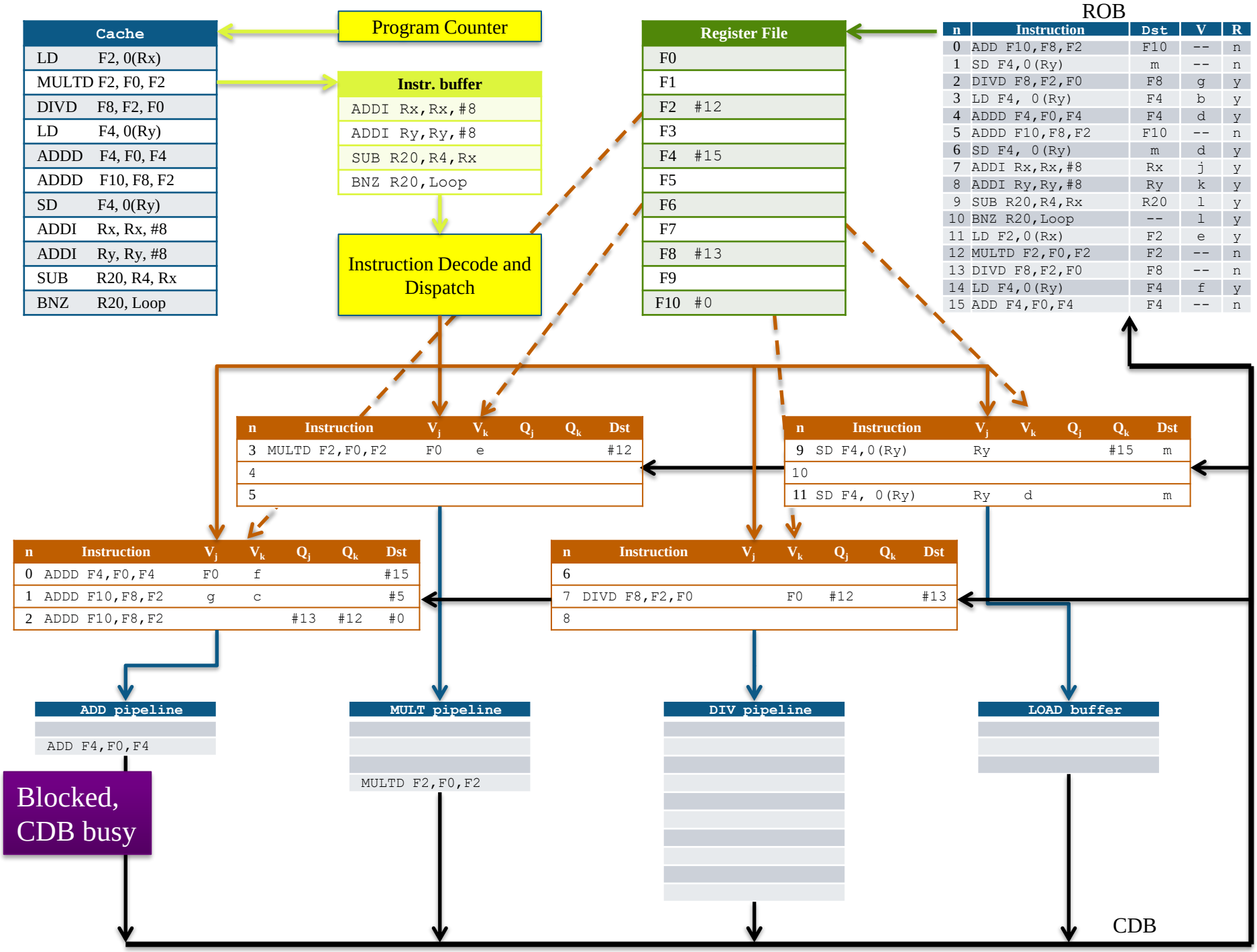

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDD F10,F8,F2
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #5
ROB
n Instruction Dst V R
0
1
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 -- n
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 -- n
15 ADD F4,F0,F4 F4 -- n
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 e #12
4
5
n Instruction Vj Vk Qj Qk Dst
9
10 LD F4,0(Ry) Ry #14
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #14 #15
1 ADDD F10,F8,F2 c #2 #5
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7 DIVD F8,F2,F0 F0 #12 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F4,0(Ry)
CDB

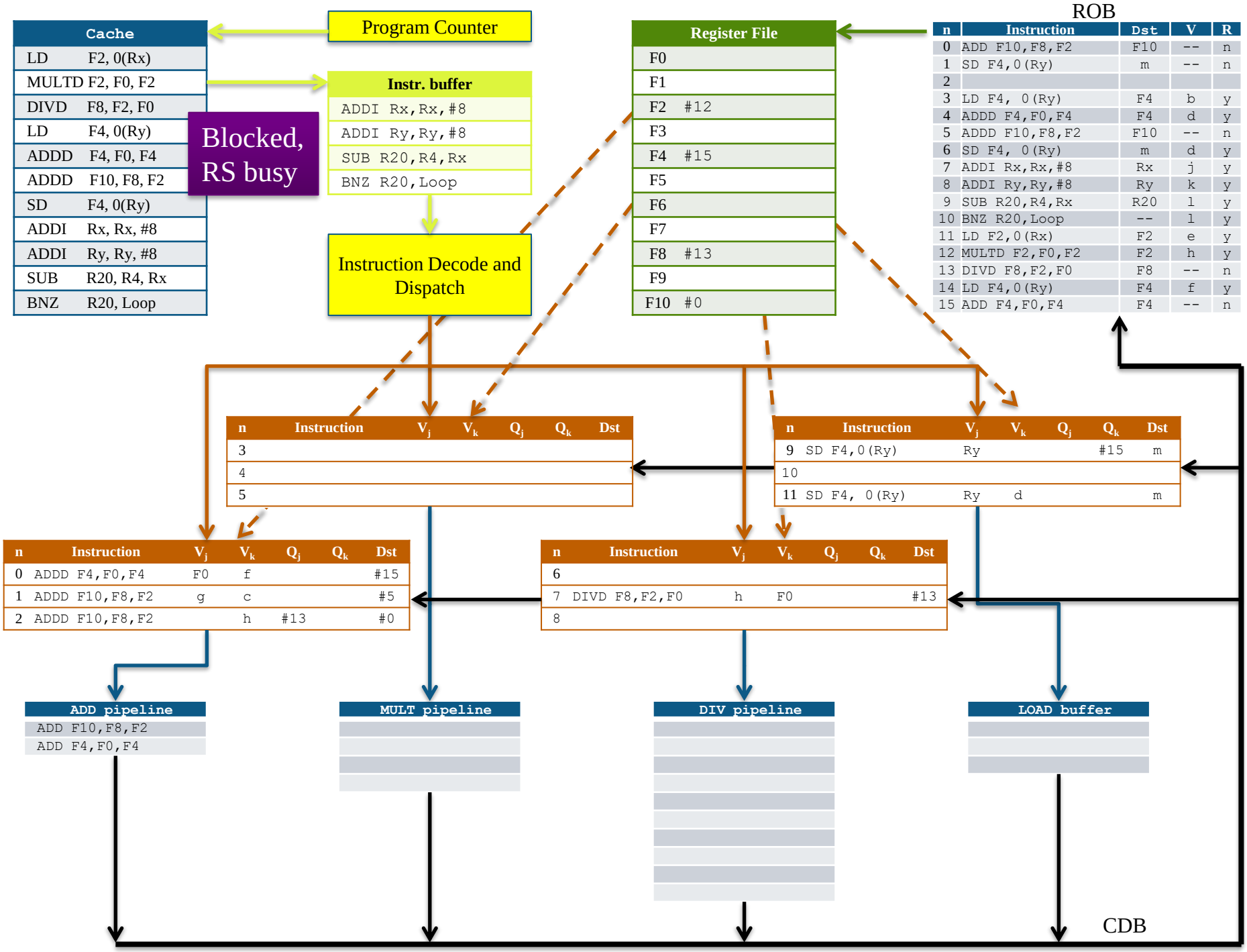

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
SD F4,0(Ry)
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 -- n
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 -- n
15 ADD F4,F0,F4 F4 -- n
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 e #12
4
5
n Instruction Vj Vk Qj Qk Dst
9
10 LD F4,0(Ry) Ry #14
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #14 #15
1 ADDD F10,F8,F2 c #2 #5
2 ADDD F10,F8,F2 #13 #12 #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7 DIVD F8,F2,F0 F0 #12 #13
8
ADD pipeline
MULT pipeline
MULTD F2,F0,F2
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F4,0(Ry)
CDB

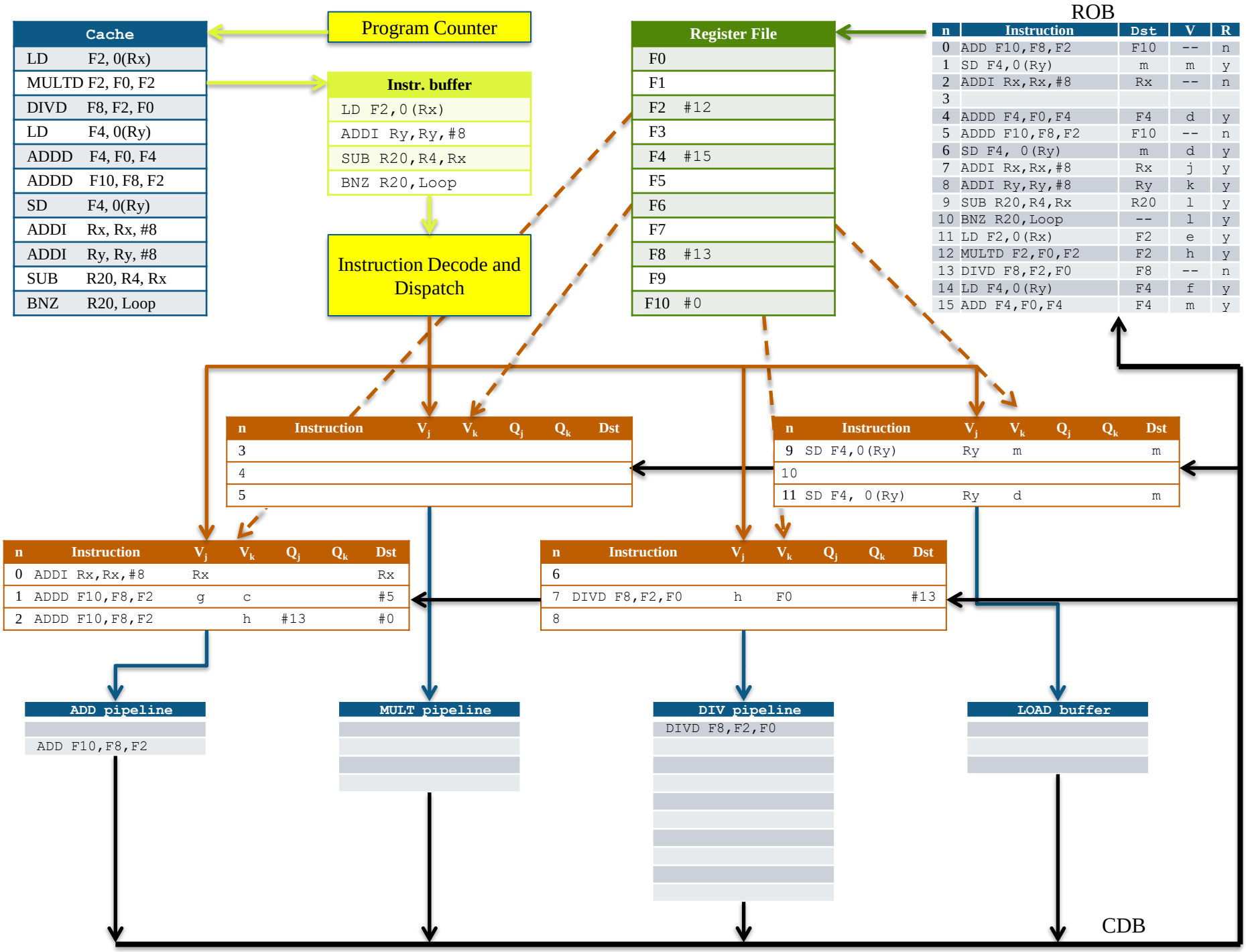

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Rx,Rx,#8
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 -- n
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 -- n
15 ADD F4,F0,F4 F4 -- n
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 e #12
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry #15 m
10 LD F4,0(Ry) Ry #14
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #14 #15
1 ADDD F10,F8,F2 c #2 #5
2 ADDD F10,F8,F2 #13 #12 #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7 DIVD F8,F2,F0 F0 #12 #13
8
ADD pipeline
MULT pipeline
MULTD F2,F0,F2
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F4,0(Ry)
CDB

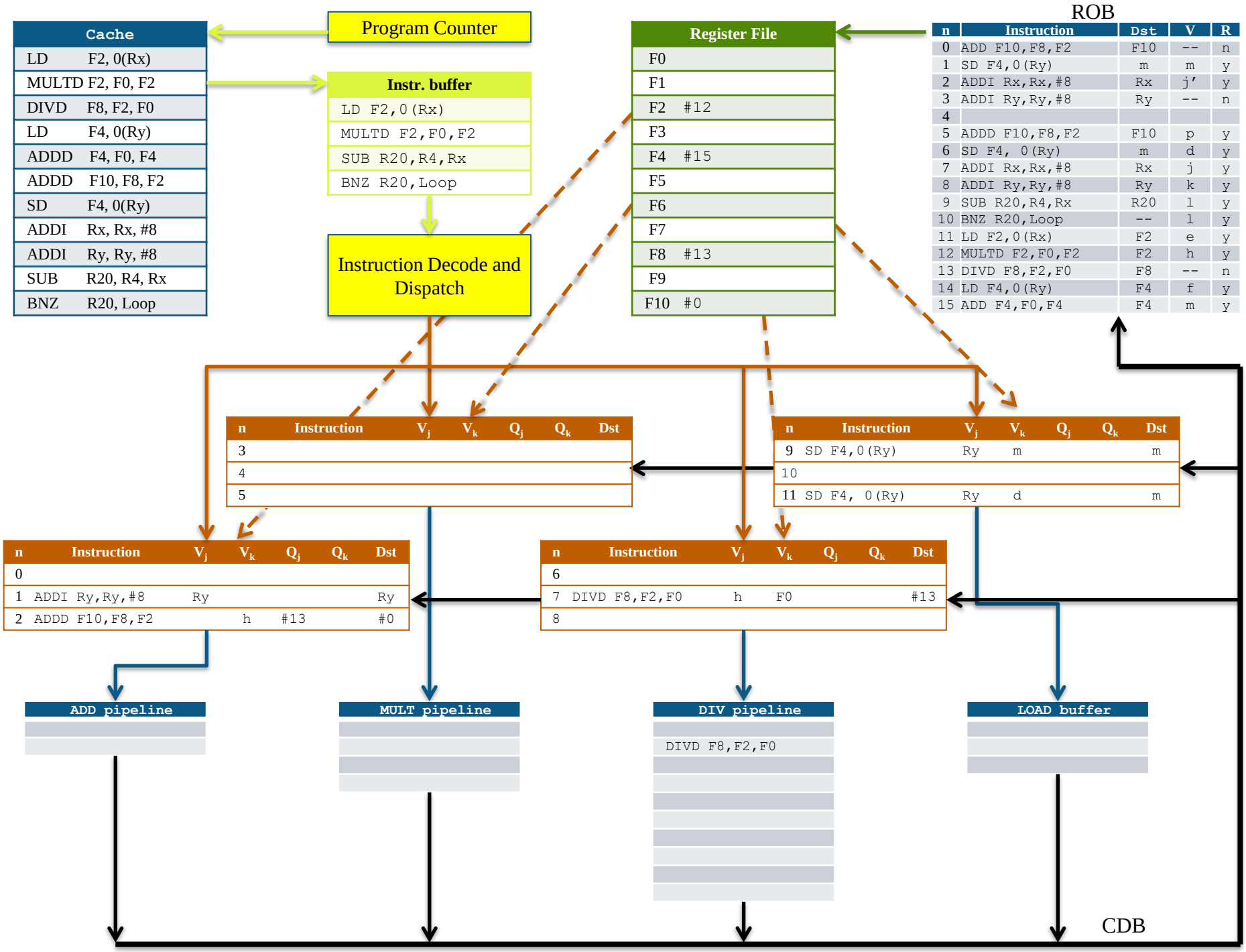

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Rx,Rx,#8
ADDI Ry,Ry,#8
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 -- n
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 -- n
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 e #12
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry #15 m
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 f #15
1 ADDD F10,F8,F2 c #2 #5
2 ADDD F10,F8,F2 #13 #12 #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7 DIVD F8,F2,F0 F0 #12 #13
8
ADD pipeline
MULT pipeline
MULTD F2,F0,F2
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

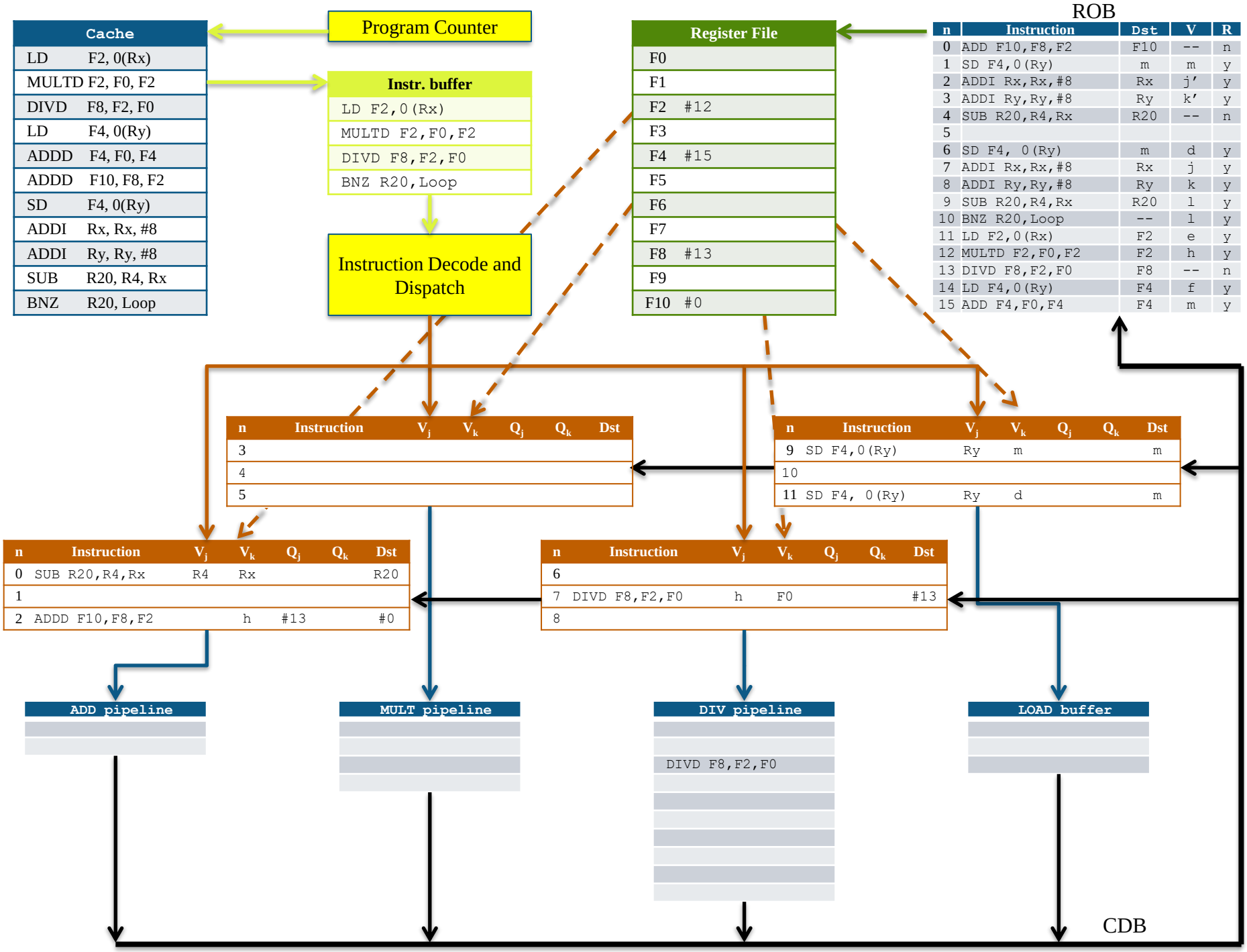

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Rx,Rx,#8
ADDI Ry,Ry,#8
SUB R20,R4,Rx
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m -- n
2 DIVD F8,F2,F0 F8 -- n
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 -- n
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 -- n
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 e #12
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry #15 m
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 f #15
1 ADDD F10,F8,F2 c #2 #5
2 ADDD F10,F8,F2 #13 #12 #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 c F0 #2
7 DIVD F8,F2,F0 F0 #12 #13
8
ADD pipeline
ADD F4,F0,F4
MULT pipeline
MULTD F2,F0,F2
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
Blocked, CDB busy
CDB

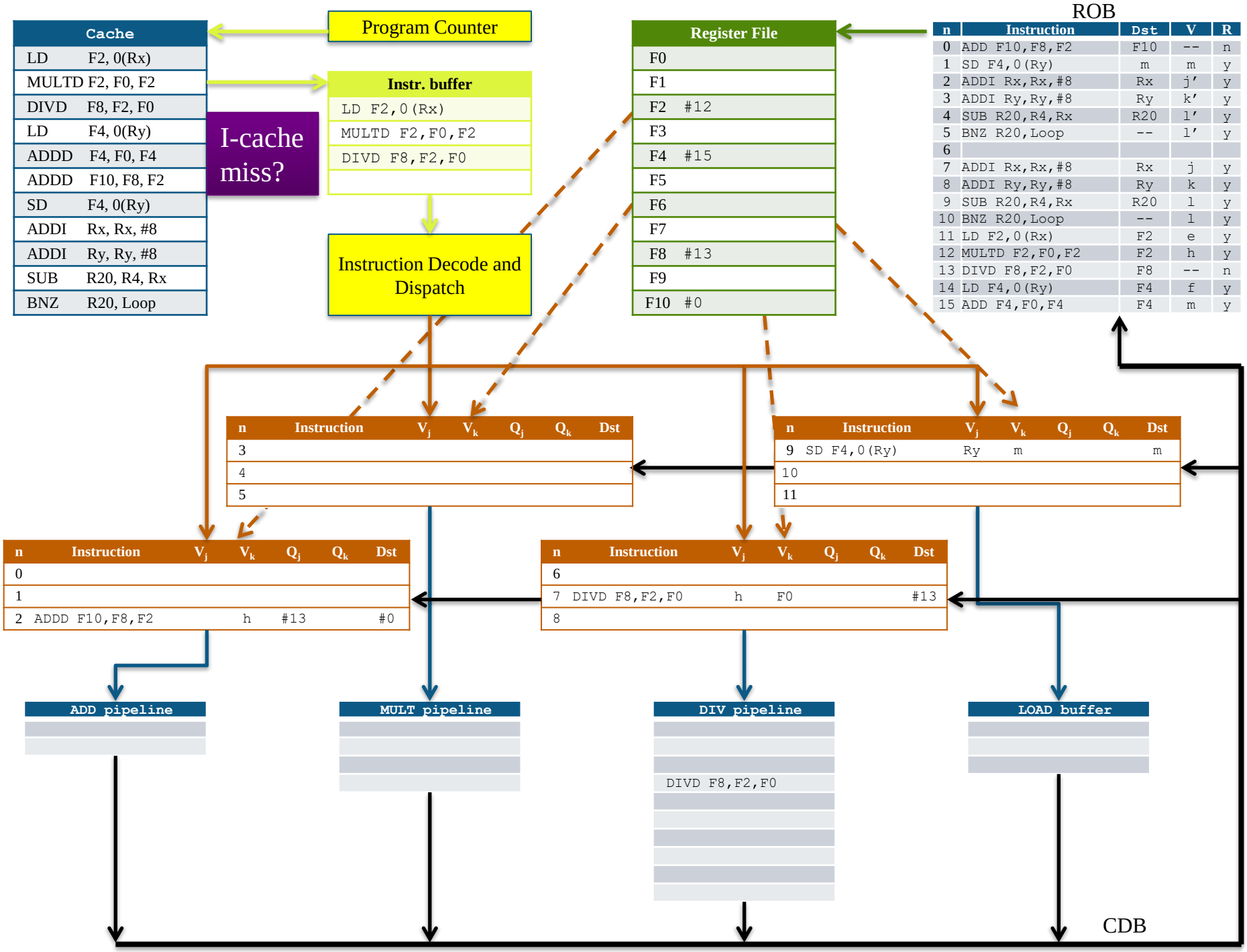

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Rx,Rx,#8
ADDI Ry,Ry,#8
SUB R20,R4,Rx
BNZ R20,Loop
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m -- n
2 DIVD F8,F2,F0 F8 g y
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 -- n
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 -- n
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 e #12
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry #15 m
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 f #15
1 ADDD F10,F8,F2 g c #5
2 ADDD F10,F8,F2 #13 #12 #0
n Instruction Vj Vk Qj Qk Dst
6
7 DIVD F8,F2,F0 F0 #12 #13
8
ADD pipeline
ADD F4,F0,F4
MULT pipeline
MULTD F2,F0,F2
DIV pipeline
LOAD buffer
Blocked, CDB busy
CDB

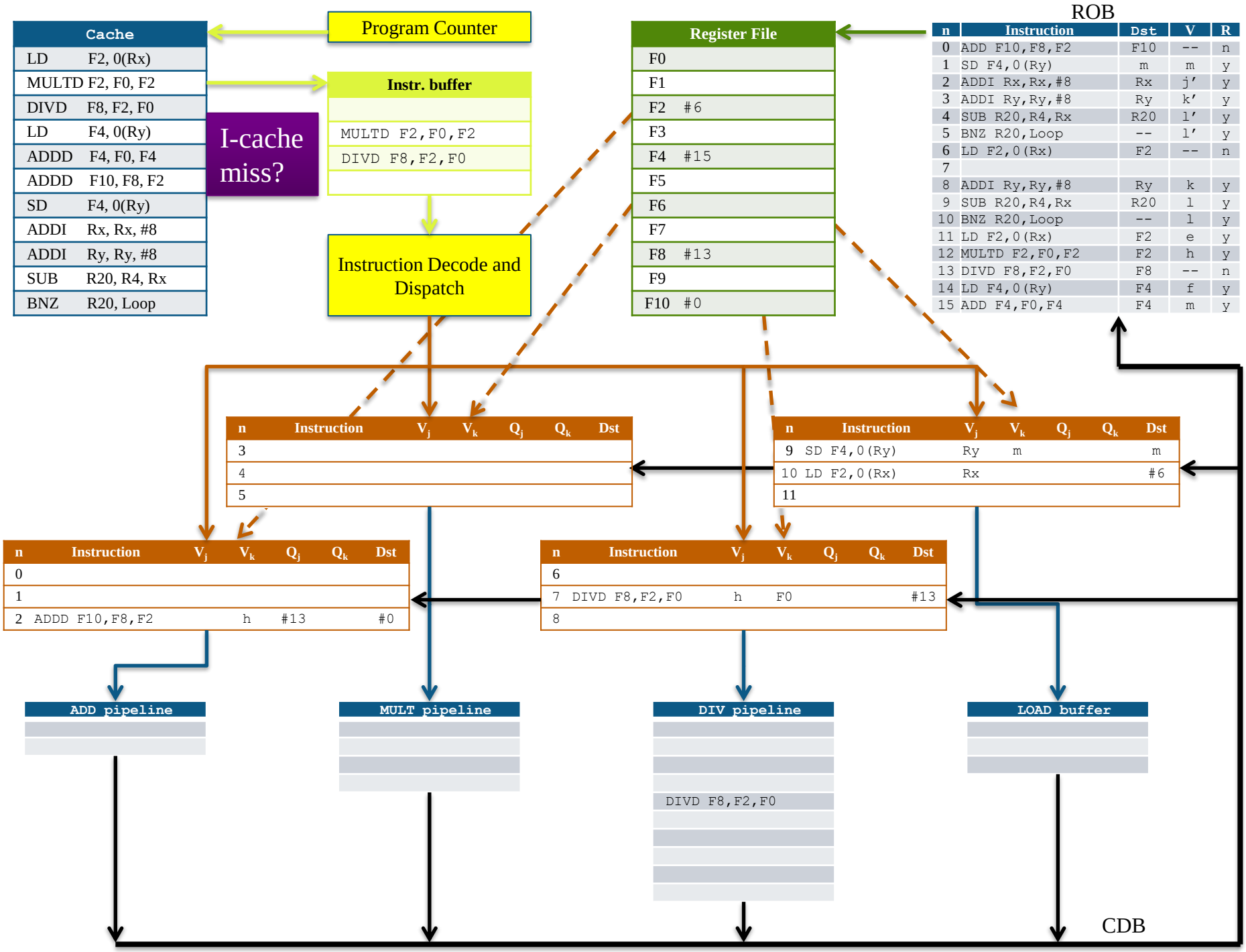

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Blocked, RS busy
Instr. buffer
ADDI Rx,Rx,#8
ADDI Ry,Ry,#8
SUB R20,R4,Rx
BNZ R20,Loop
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m -- n
2
3 LD F4, 0(Ry) F4 b y
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 -- n
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry #15 m
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 f #15
1 ADDD F10,F8,F2 g c #5
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
ADD F10,F8,F2
ADD F4,F0,F4
MULT pipeline
DIV pipeline
LOAD buffer
CDB

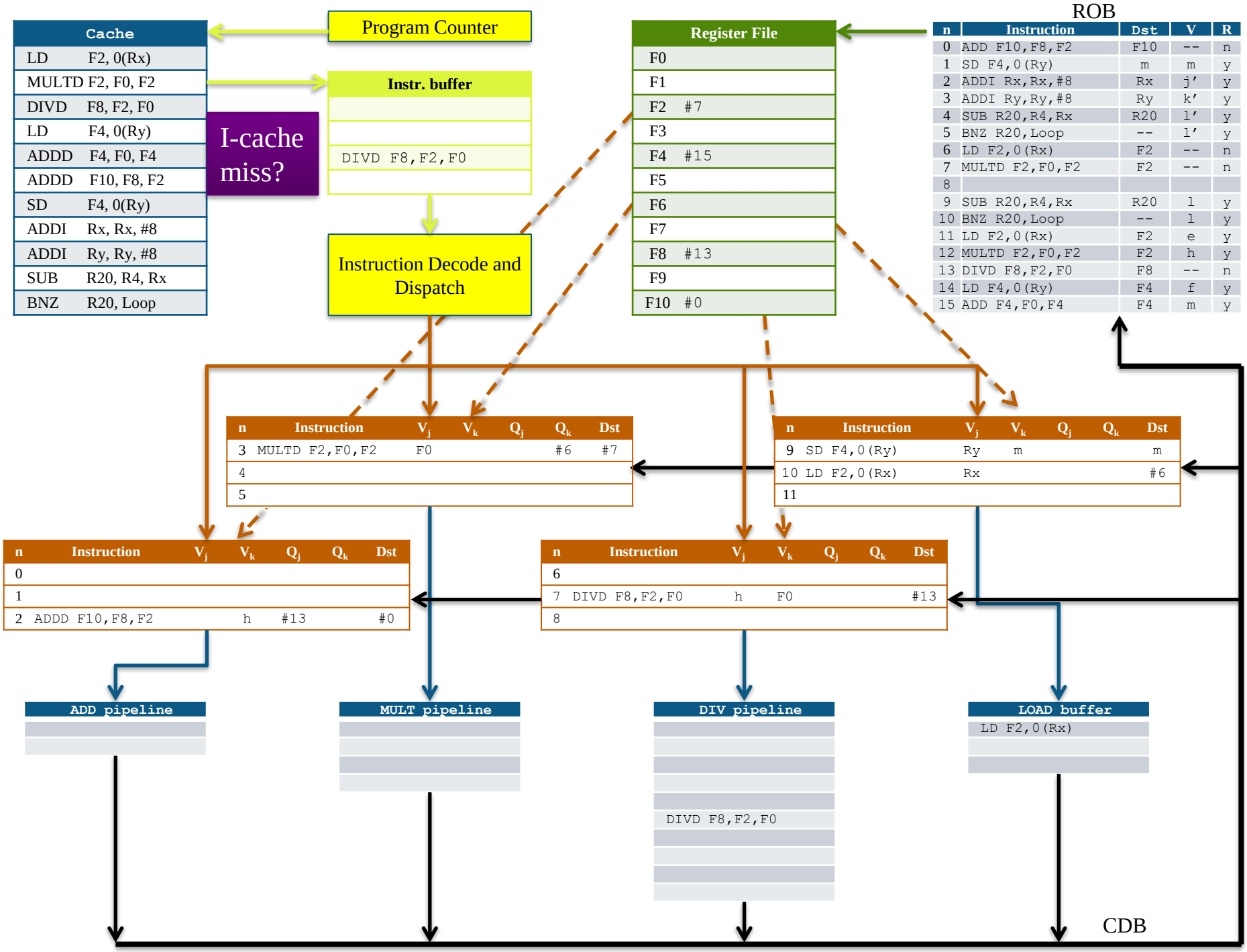

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F2,0(Rx)
ADDI Ry,Ry,#8
SUB R20,R4,Rx
BNZ R20,Loop
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx -- n
3
4 ADDD F4,F0,F4 F4 d y
5 ADDD F10,F8,F2 F10 -- n
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 ADDI Rx,Rx,#8 Rx Rx
1 ADDD F10,F8,F2 g c #5
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
ADD F10,F8,F2
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

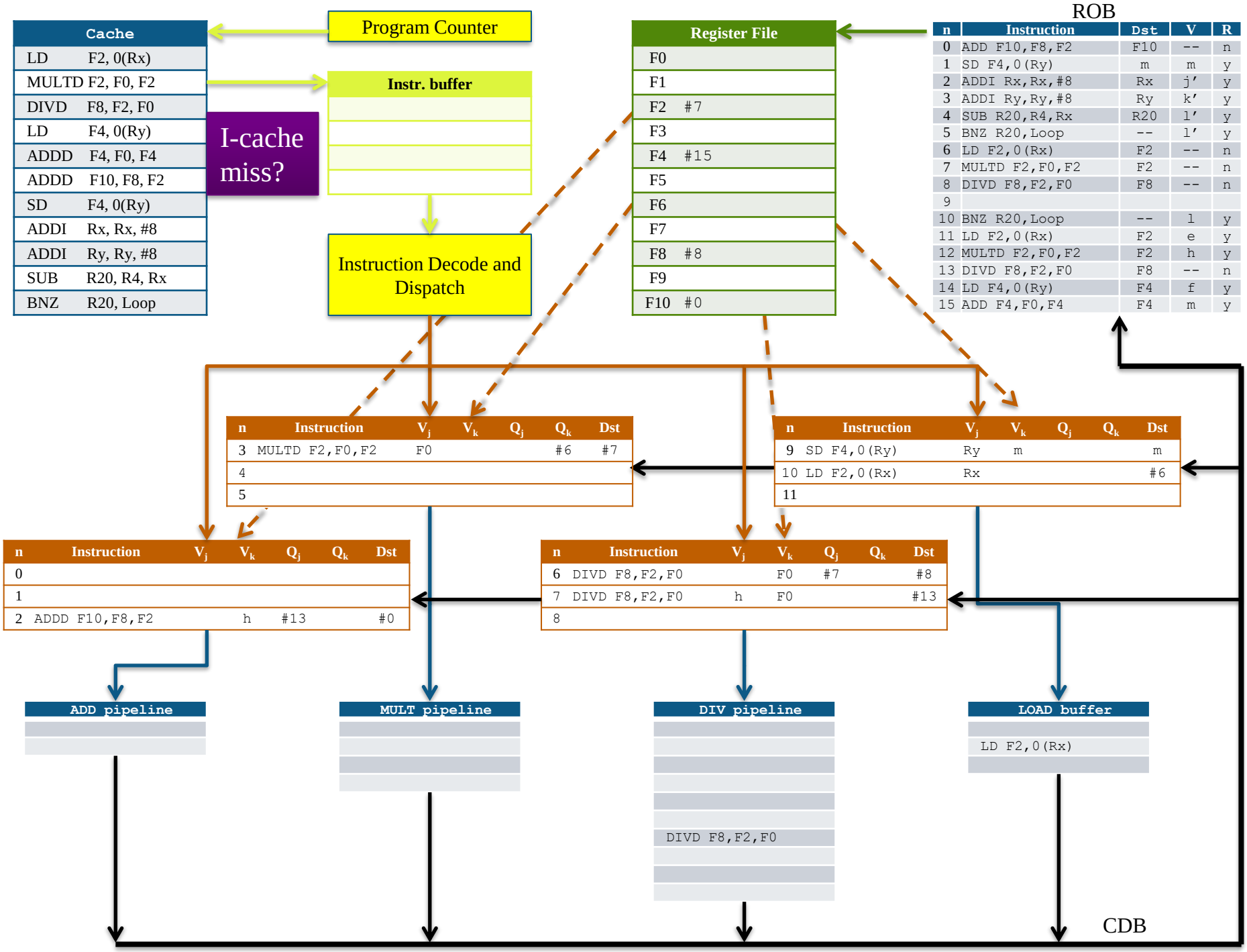

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F2,0(Rx)
MULTD F2,F0,F2
SUB R20,R4,Rx
BNZ R20,Loop
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry -- n
4
5 ADDD F10,F8,F2 F10 p y
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0
1 ADDI Ry,Ry,#8 Ry Ry
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

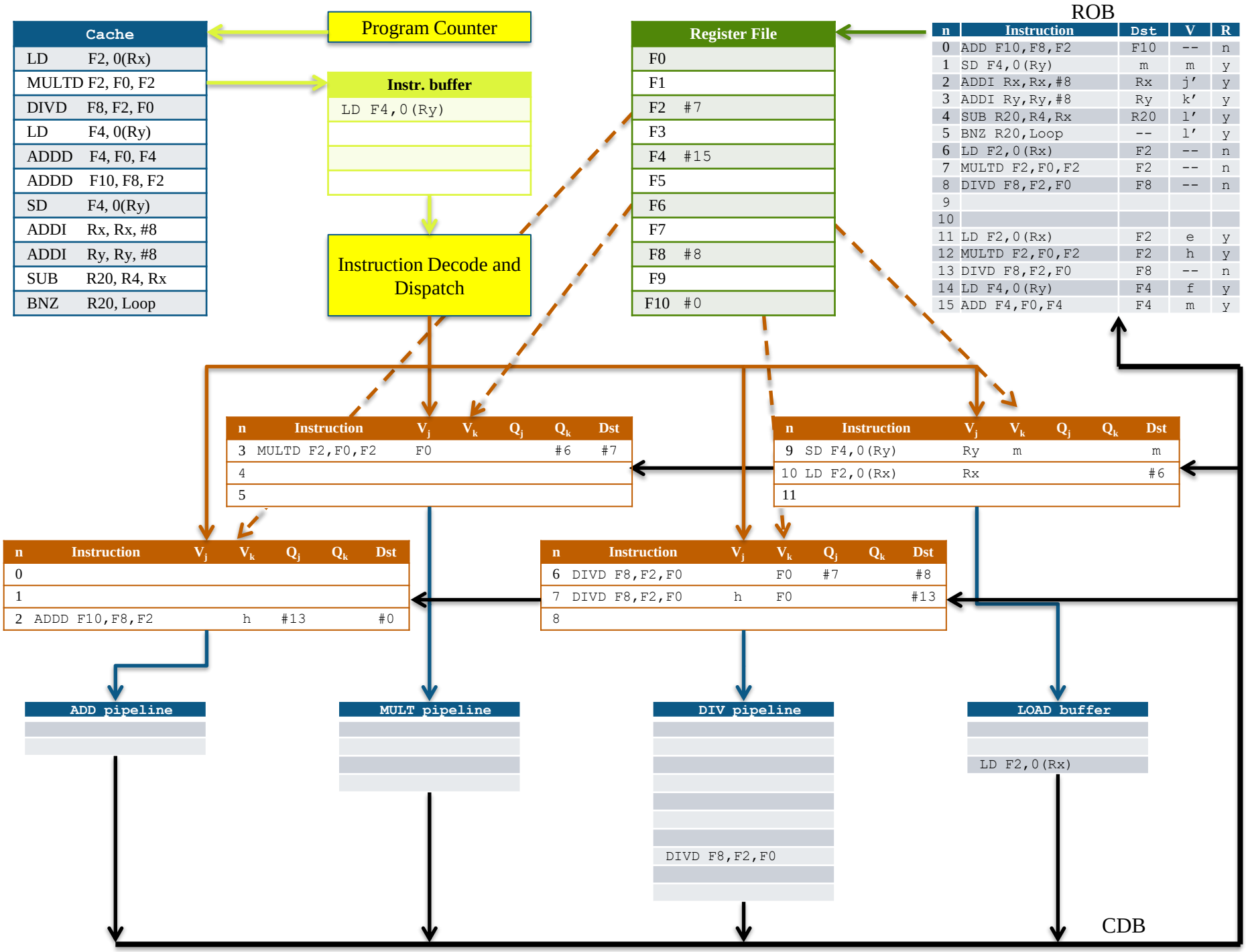

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F2,0(Rx)
MULTD F2,F0,F2
DIVD F8,F2,F0
BNZ R20,Loop
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 -- n
5
6 SD F4, 0(Ry) m d y
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10
11 SD F4, 0(Ry) Ry d m
n Instruction Vj Vk Qj Qk Dst
0 SUB R20,R4,Rx R4 Rx R20
1
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

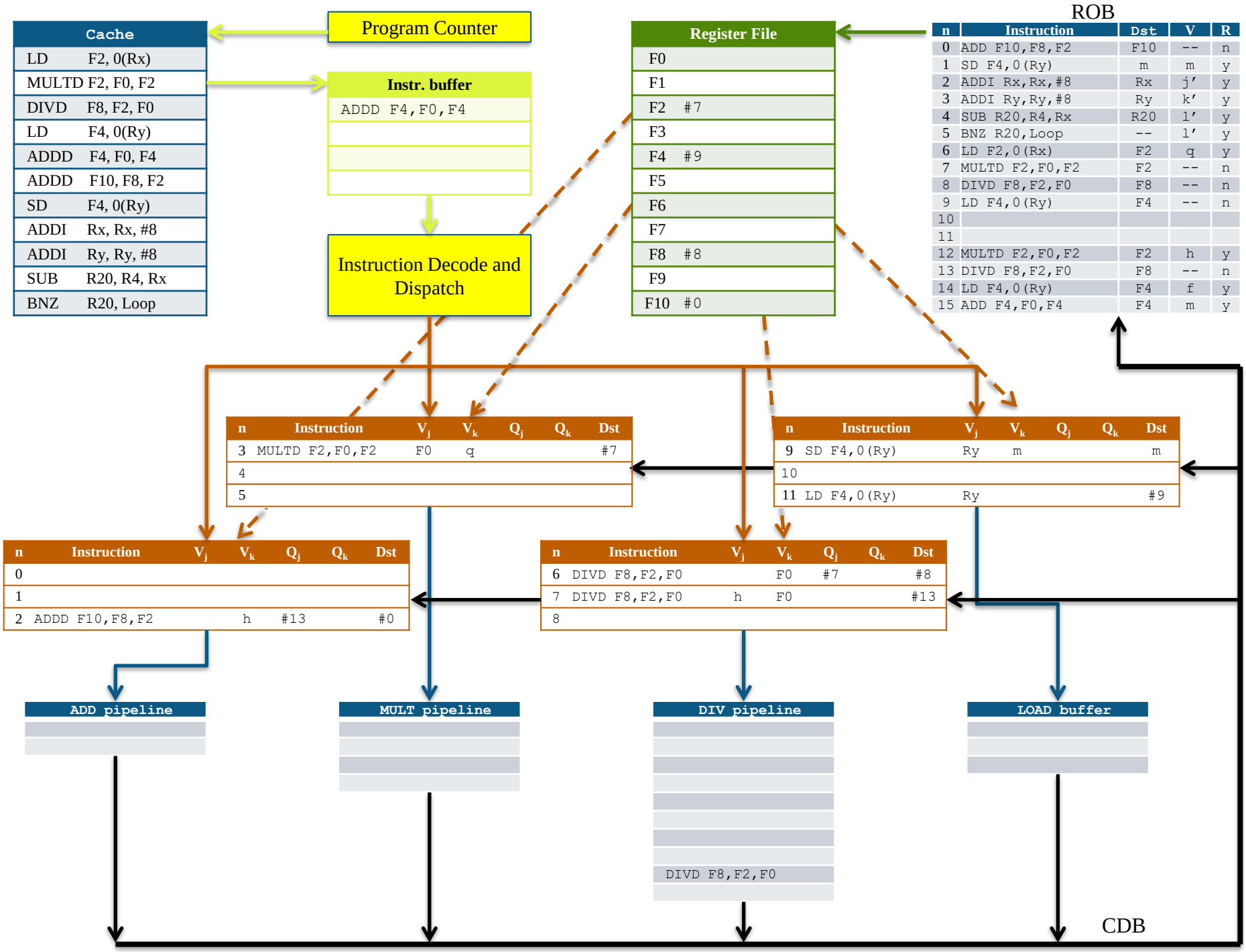

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
I-cache miss?
Instr. buffer
LD F2,0(Rx)
MULTD F2,F0,F2
DIVD F8,F2,F0
Instruction Decode and Dispatch
Register File
F0
F1
F2 #12
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6
7 ADDI Rx,Rx,#8 Rx j y
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10
11
n Instruction Vj Vk Qj Qk Dst
0
1
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

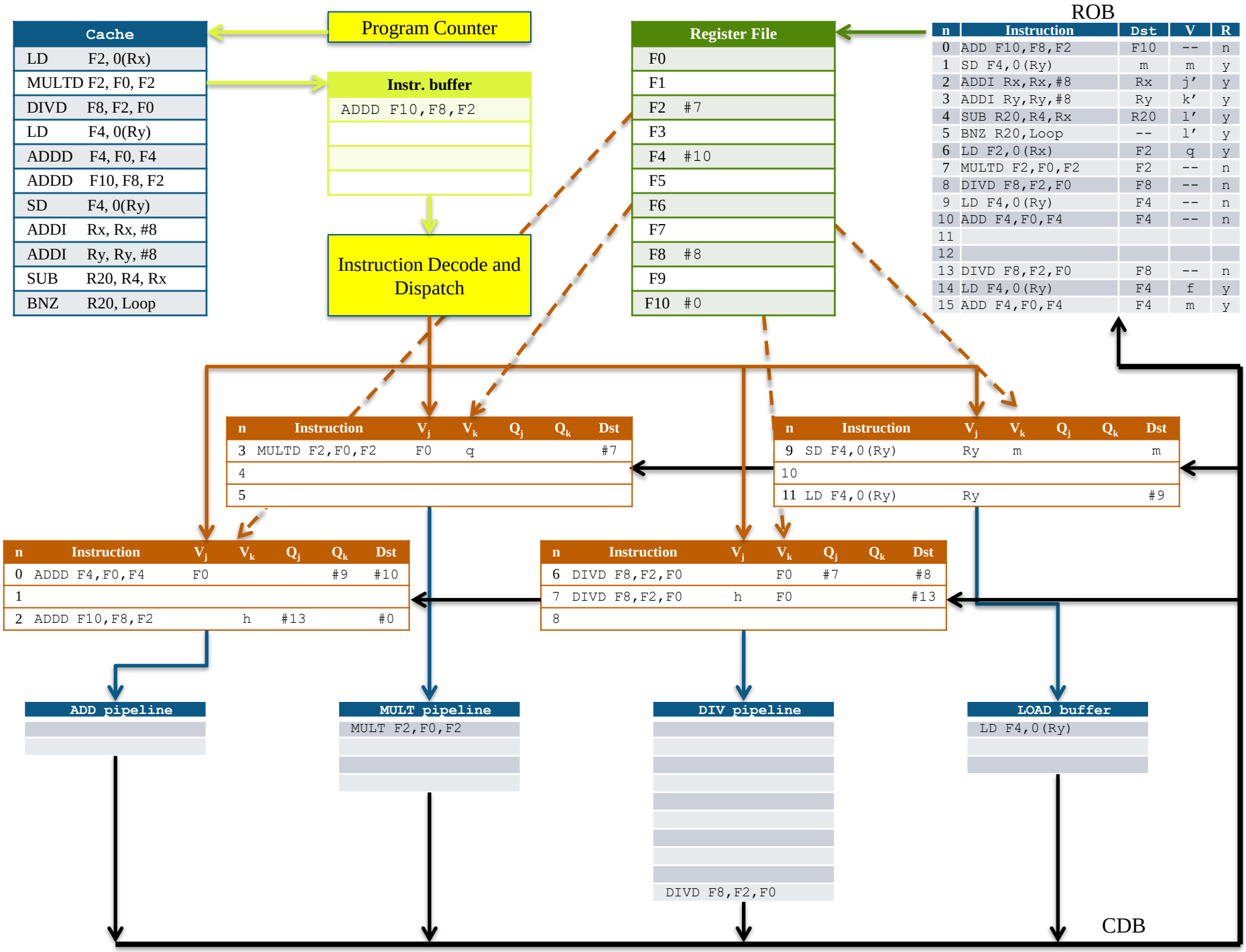

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
I-cache miss?
Instr. buffer
MULTD F2,F0,F2
DIVD F8,F2,F0
Instruction Decode and Dispatch
Register File
F0
F1
F2 #6
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 -- n
7
8 ADDI Ry,Ry,#8 Ry k y
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10 LD F2,0(Rx) Rx #6
11
n Instruction Vj Vk Qj Qk Dst
0
1
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

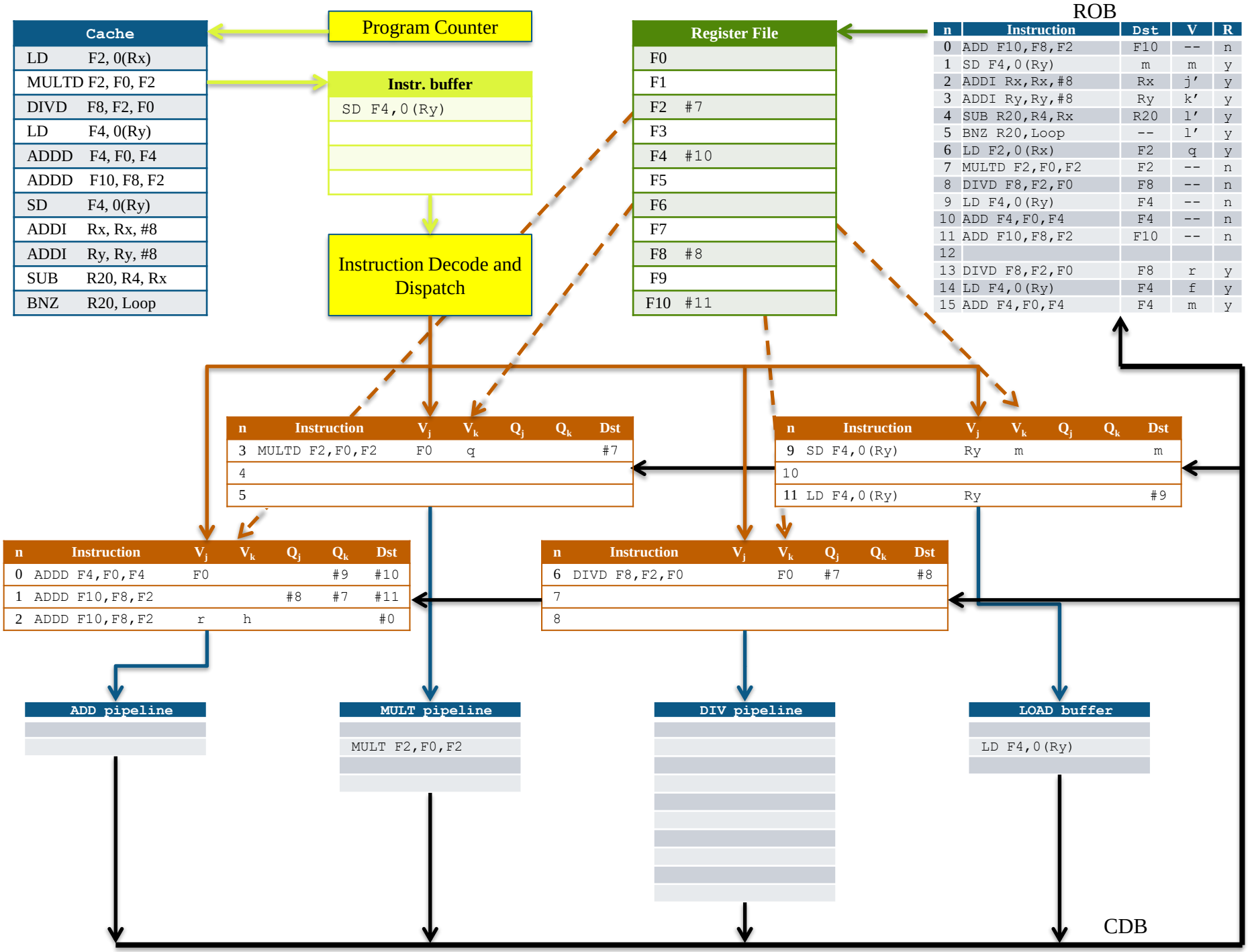

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
I-cache miss?
Instr. buffer
DIVD F8,F2,F0
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #15
F5
F6
F7
F8 #13
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 -- n
7 MULTD F2,F0,F2 F2 -- n
8
9 SUB R20,R4,Rx R20 l y
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 #6 #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10 LD F2,0(Rx) Rx #6
11
n Instruction Vj Vk Qj Qk Dst
0
1
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F2,0(Rx)
CDB

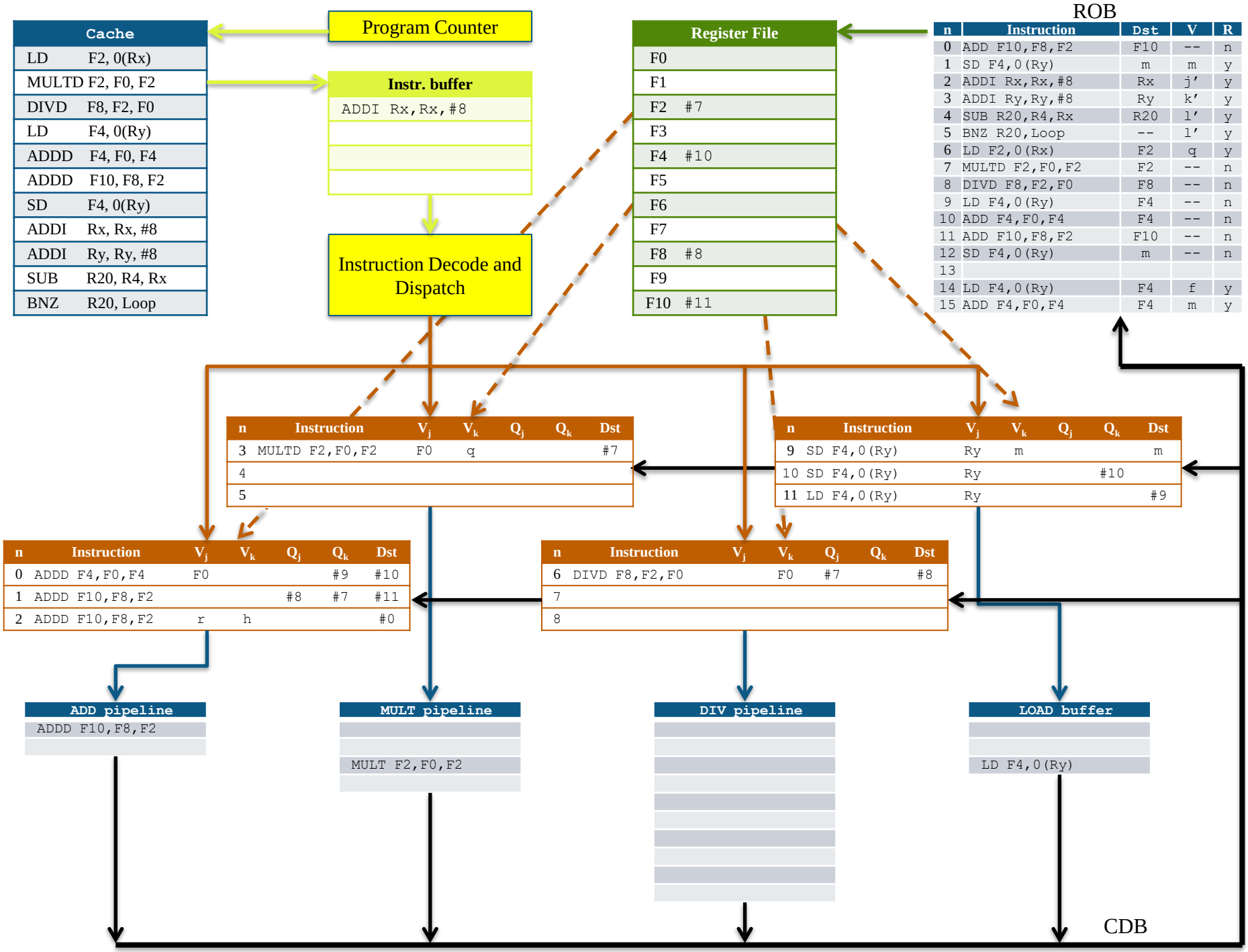

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
I-cache miss?
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #15
F5
F6
F7
F8 #8
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 -- n
7 MULTD F2,F0,F2 F2 -- n
8 DIVD F8,F2,F0 F8 -- n
9
10 BNZ R20,Loop -- l y
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 #6 #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10 LD F2,0(Rx) Rx #6
11
n Instruction Vj Vk Qj Qk Dst
0
1
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #7 #8
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F2,0(Rx)
CDB

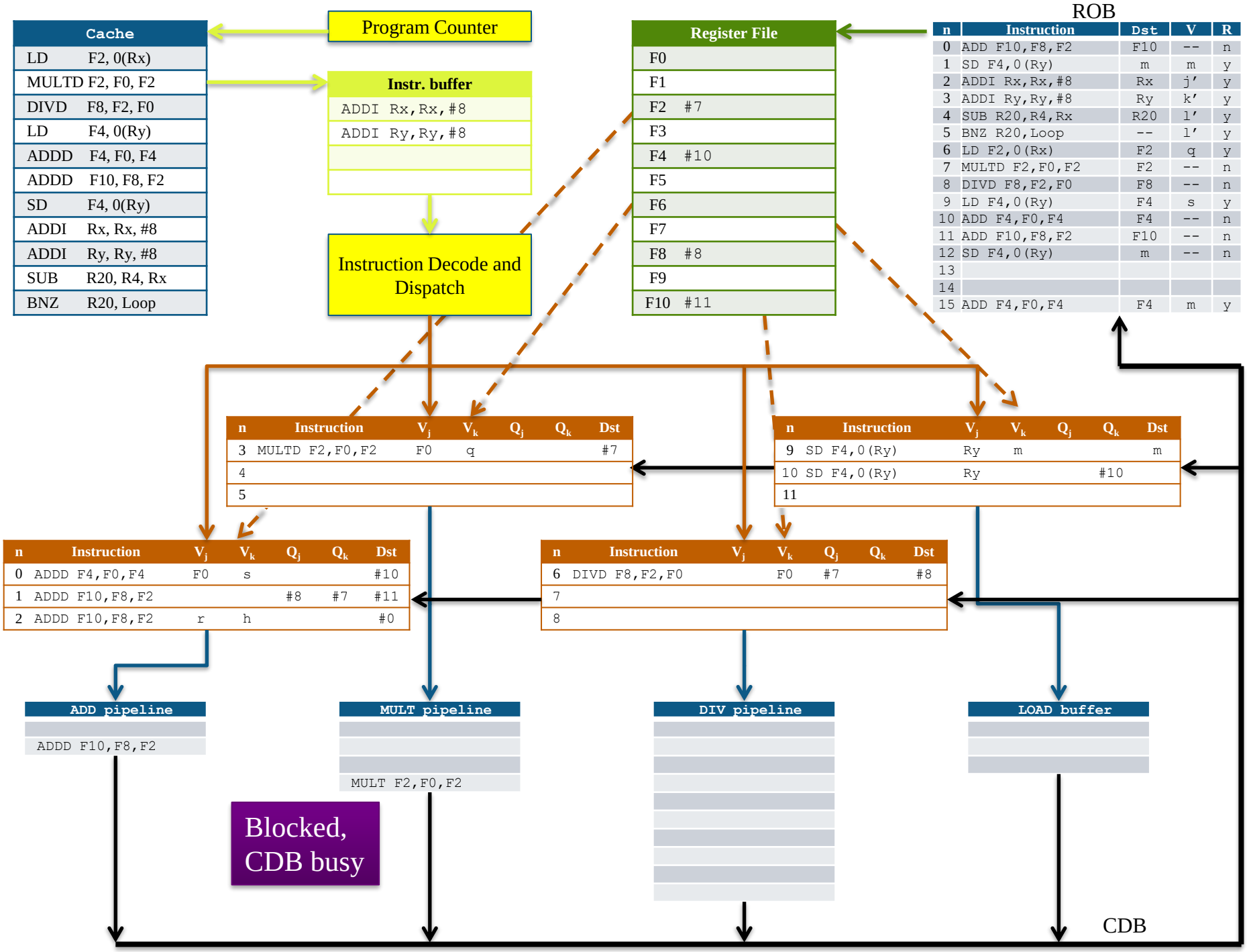

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F4,0(Ry)
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #15
F5
F6
F7
F8 #8
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 -- n
7 MULTD F2,F0,F2 F2 -- n
8 DIVD F8,F2,F0 F8 -- n
9
10
11 LD F2,0(Rx) F2 e y
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 #6 #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10 LD F2,0(Rx) Rx #6
11
n Instruction Vj Vk Qj Qk Dst
0
1
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #7 #8
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F2,0(Rx)
CDB

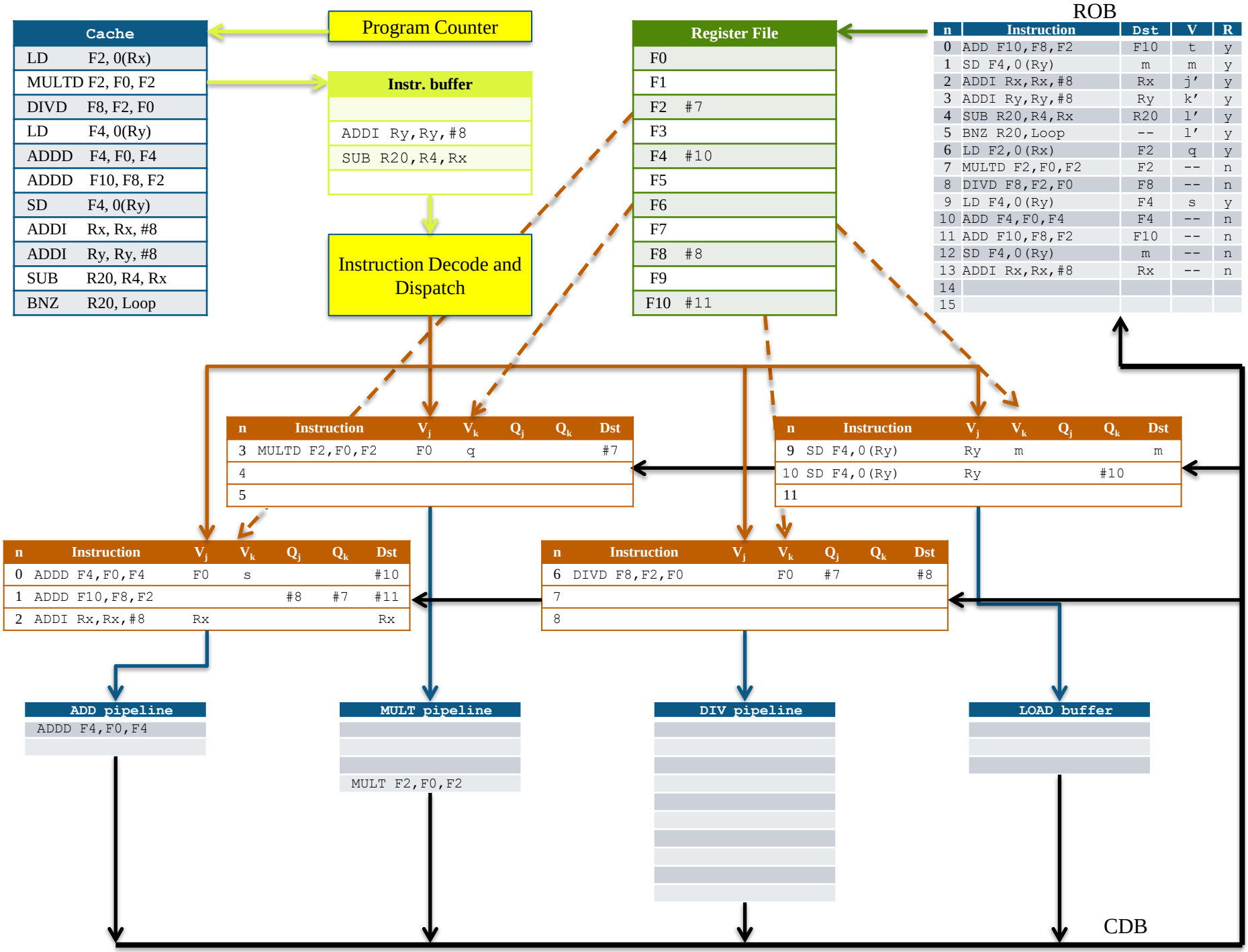

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDD F4,F0,F4
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #9
F5
F6
F7
F8 #8
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 q y
7 MULTD F2,F0,F2 F2 -- n
8 DIVD F8,F2,F0 F8 -- n
9 LD F4,0(Ry) F4 -- n
10
11
12 MULTD F2,F0,F2 F2 h y
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 q #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10
11 LD F4,0(Ry) Ry #9
n Instruction Vj Vk Qj Qk Dst
0
1
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #7 #8
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB

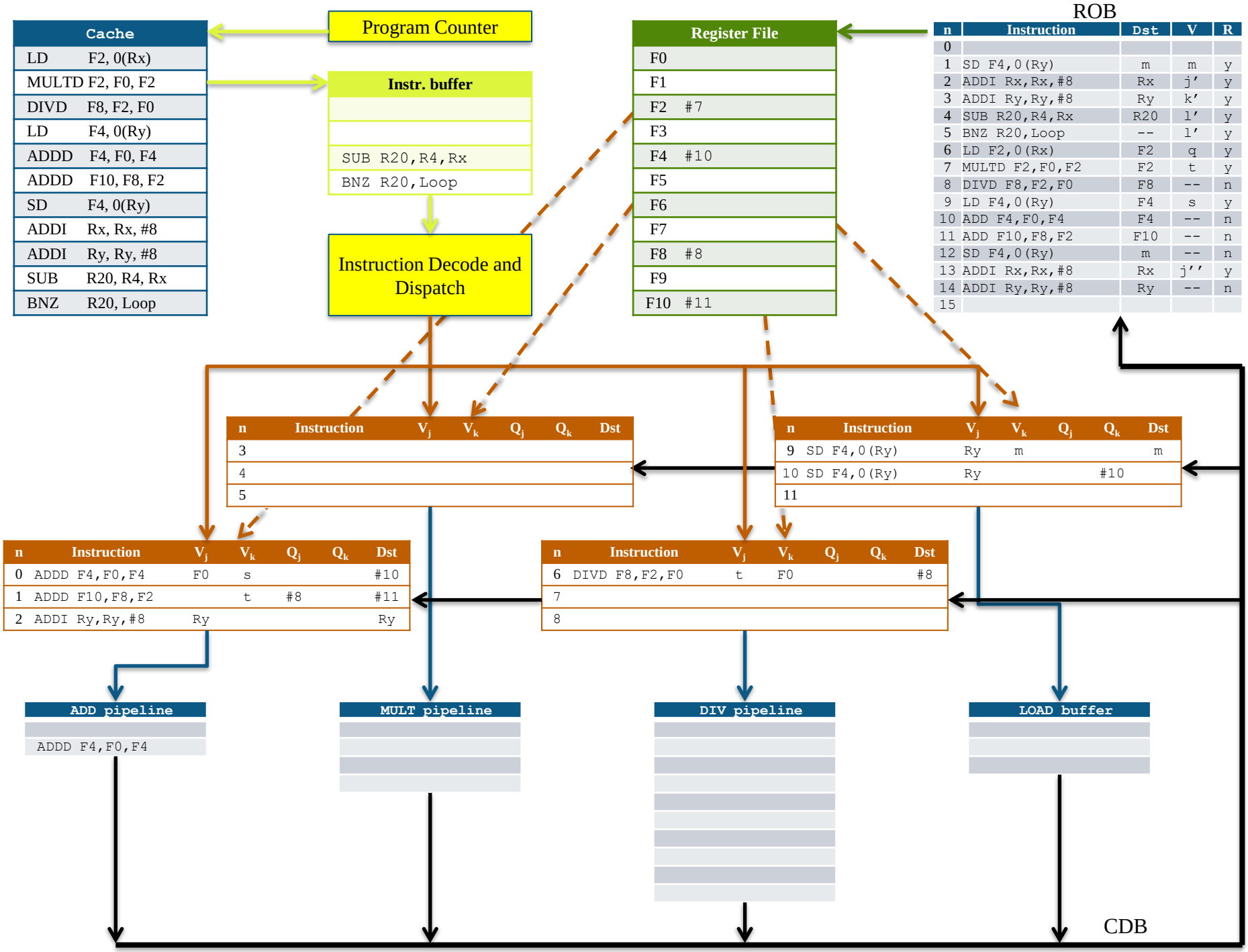

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDD F10,F8,F2
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #10
F5
F6
F7
F8 #8
F9
F10 #0
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 q y
7 MULTD F2,F0,F2 F2 -- n
8 DIVD F8,F2,F0 F8 -- n
9 LD F4,0(Ry) F4 -- n
10 ADD F4,F0,F4 F4 -- n
11
12
13 DIVD F8,F2,F0 F8 -- n
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 q #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10
11 LD F4,0(Ry) Ry #9
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #9 #10
1
2 ADDD F10,F8,F2 h #13 #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #7 #8
7 DIVD F8,F2,F0 h F0 #13
8
ADD pipeline
MULT pipeline
MULT F2,F0,F2
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
LD F4,0(Ry)
CDB

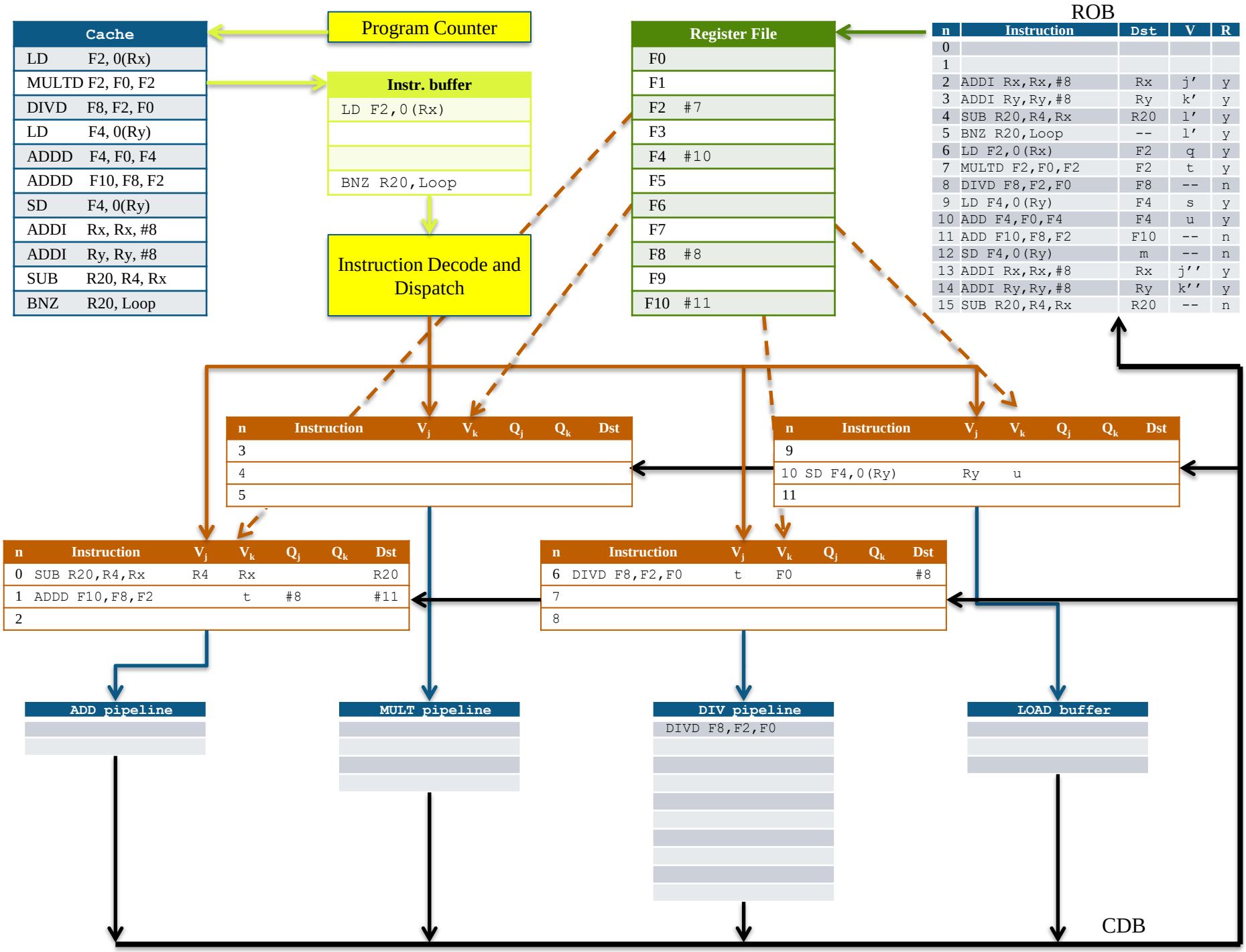

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
SD F4,0(Ry)
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #10
F5
F6
F7
F8 #8
F9
F10 #11
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 q y
7 MULTD F2,F0,F2 F2 -- n
8 DIVD F8,F2,F0 F8 -- n
9 LD F4,0(Ry) F4 -- n
10 ADD F4,F0,F4 F4 -- n
11 ADD F10,F8,F2 F10 -- n
12
13 DIVD F8,F2,F0 F8 r y
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 q #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10
11 LD F4,0(Ry) Ry #9
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #9 #10
1 ADDD F10,F8,F2 #8 #7 #11
2 ADDD F10,F8,F2 r h #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #7 #8
7
8
ADD pipeline
MULT pipeline
MULT F2,F0,F2
DIV pipeline
LOAD buffer
LD F4,0(Ry)
CDB

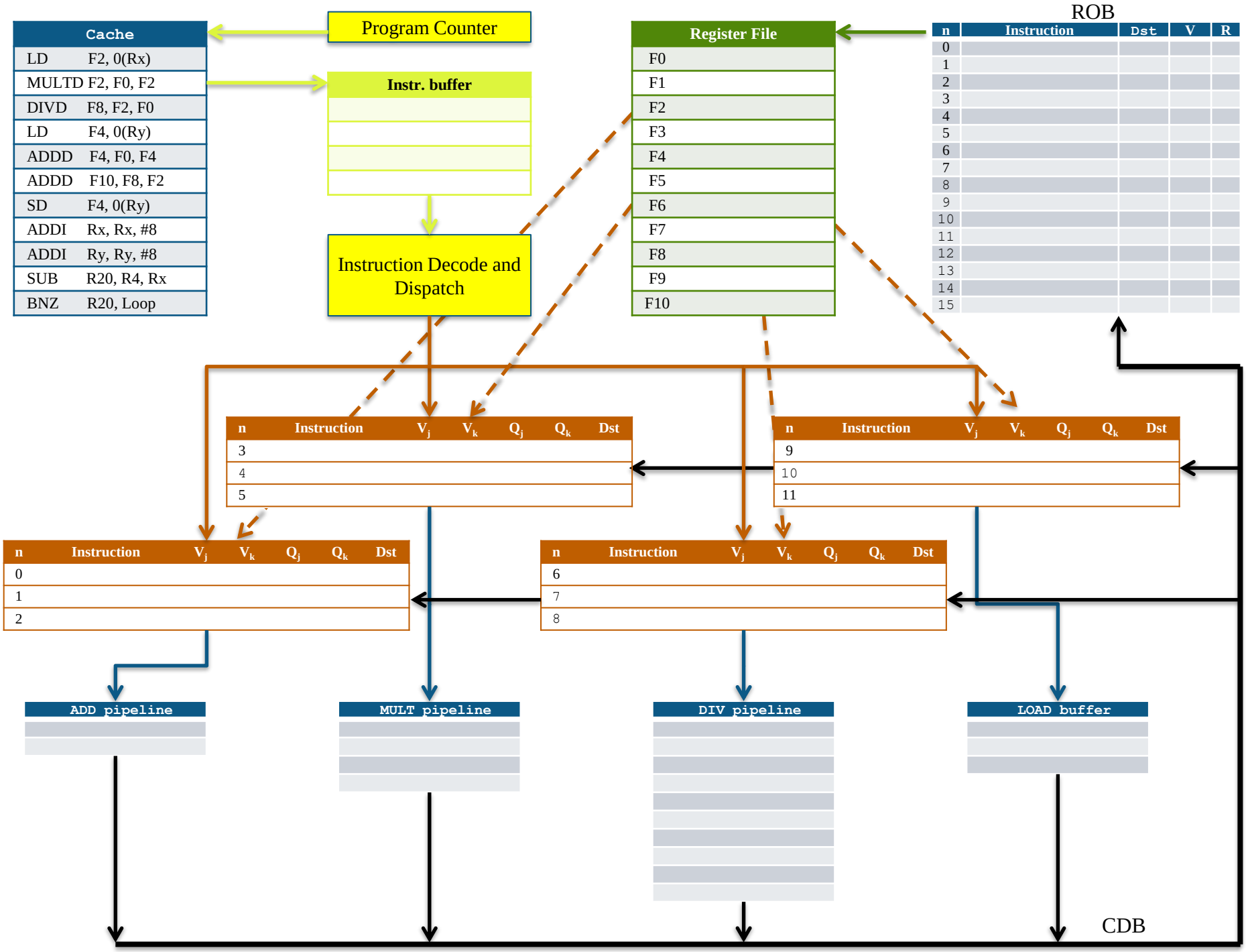

Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Rx,Rx,#8
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #10
F5
F6
F7
F8 #8
F9
F10 #11
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 q y
7 MULTD F2,F0,F2 F2 -- n
8 DIVD F8,F2,F0 F8 -- n
9 LD F4,0(Ry) F4 -- n
10 ADD F4,F0,F4 F4 -- n
11 ADD F10,F8,F2 F10 -- n
12 SD F4,0(Ry) m -- n
13
14 LD F4,0(Ry) F4 f y
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 q #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10 SD F4,0(Ry) Ry #10
11 LD F4,0(Ry) Ry #9
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 #9 #10
1 ADDD F10,F8,F2 #8 #7 #11
2 ADDD F10,F8,F2 r h #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #7 #8
7
8
ADD pipeline
ADDD F10,F8,F2
MULT pipeline
MULT F2,F0,F2
DIV pipeline
LOAD buffer
LD F4,0(Ry)
CDB


Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Rx,Rx,#8
ADDI Ry,Ry,#8
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #10
F5
F6
F7
F8 #8
F9
F10 #11
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 -- n
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 q y
7 MULTD F2,F0,F2 F2 -- n
8 DIVD F8,F2,F0 F8 -- n
9 LD F4,0(Ry) F4 s y
10 ADD F4,F0,F4 F4 -- n
11 ADD F10,F8,F2 F10 -- n
12 SD F4,0(Ry) m -- n
13
14
15 ADD F4,F0,F4 F4 m y
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 q #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10 SD F4,0(Ry) Ry #10
11
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 s #10
1 ADDD F10,F8,F2 #8 #7 #11
2 ADDD F10,F8,F2 r h #0
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #7 #8
7
8
ADD pipeline
ADDD F10,F8,F2
MULT pipeline
MULT F2,F0,F2
DIV pipeline
LOAD buffer
Blocked, CDB busy
CDB


Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
ADDI Ry,Ry,#8
SUB R20,R4,Rx
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #10
F5
F6
F7
F8 #8
F9
F10 #11
ROB
n Instruction Dst V R
0 ADD F10,F8,F2 F10 t y
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 q y
7 MULTD F2,F0,F2 F2 -- n
8 DIVD F8,F2,F0 F8 -- n
9 LD F4,0(Ry) F4 s y
10 ADD F4,F0,F4 F4 -- n
11 ADD F10,F8,F2 F10 -- n
12 SD F4,0(Ry) m -- n
13 ADDI Rx,Rx,#8 Rx -- n
14
15
n Instruction Vj Vk Qj Qk Dst
3 MULTD F2,F0,F2 F0 q #7
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10 SD F4,0(Ry) Ry #10
11
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 s #10
1 ADDD F10,F8,F2 #8 #7 #11
2 ADDI Rx,Rx,#8 Rx Rx
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 F0 #7 #8
7
8
ADD pipeline
ADDD F4,F0,F4
MULT pipeline
MULT F2,F0,F2
DIV pipeline
LOAD buffer
CDB


Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
SUB R20,R4,Rx
BNZ R20,Loop
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #10
F5
F6
F7
F8 #8
F9
F10 #11
ROB
n Instruction Dst V R
0
1 SD F4,0(Ry) m m y
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 q y
7 MULTD F2,F0,F2 F2 t y
8 DIVD F8,F2,F0 F8 -- n
9 LD F4,0(Ry) F4 s y
10 ADD F4,F0,F4 F4 -- n
11 ADD F10,F8,F2 F10 -- n
12 SD F4,0(Ry) m -- n
13 ADDI Rx,Rx,#8 Rx j'' y
14 ADDI Ry,Ry,#8 Ry -- n
15
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9 SD F4,0(Ry) Ry m m
10 SD F4,0(Ry) Ry #10
11
n Instruction Vj Vk Qj Qk Dst
0 ADDD F4,F0,F4 F0 s #10
1 ADDD F10,F8,F2 t #8 #11
2 ADDI Ry,Ry,#8 Ry Ry
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 t F0 #8
7
8
ADD pipeline
ADDD F4,F0,F4
MULT pipeline
DIV pipeline
LOAD buffer
CDB


Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
LD F2,0(Rx)
BNZ R20,Loop
Instruction Decode and Dispatch
Register File
F0
F1
F2 #7
F3
F4 #10
F5
F6
F7
F8 #8
F9
F10 #11
ROB
n Instruction Dst V R
0
1
2 ADDI Rx,Rx,#8 Rx j' y
3 ADDI Ry,Ry,#8 Ry k' y
4 SUB R20,R4,Rx R20 l' y
5 BNZ R20,Loop -- l' y
6 LD F2,0(Rx) F2 q y
7 MULTD F2,F0,F2 F2 t y
8 DIVD F8,F2,F0 F8 -- n
9 LD F4,0(Ry) F4 s y
10 ADD F4,F0,F4 F4 u y
11 ADD F10,F8,F2 F10 -- n
12 SD F4,0(Ry) m -- n
13 ADDI Rx,Rx,#8 Rx j'' y
14 ADDI Ry,Ry,#8 Ry k'' y
15 SUB R20,R4,Rx R20 -- n
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9
10 SD F4,0(Ry) Ry u
11
n Instruction Vj Vk Qj Qk Dst
0 SUB R20,R4,Rx R4 Rx R20
1 ADDD F10,F8,F2 t #8 #11
2
n Instruction Vj Vk Qj Qk Dst
6 DIVD F8,F2,F0 t F0 #8
7
8
ADD pipeline
MULT pipeline
DIV pipeline
DIVD F8,F2,F0
LOAD buffer
CDB


Cache
LD F2, 0(Rx)
MULTD F2, F0, F2
DIVD F8, F2, F0
LD F4, 0(Ry)
ADDD F4, F0, F4
ADDD F10, F8, F2
SD F4, 0(Ry)
ADDI Rx, Rx, #8
ADDI Ry, Ry, #8
SUB R20, R4, Rx
BNZ R20, Loop
Program Counter
Instr. buffer
Instruction Decode and Dispatch
Register File
F0
F1
F2
F3
F4
F5
F6
F7
F8
F9
F10
ROB
n Instruction Dst V R
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
n Instruction Vj Vk Qj Qk Dst
3
4
5
n Instruction Vj Vk Qj Qk Dst
9
10
11
n Instruction Vj Vk Qj Qk Dst
0
1
2
n Instruction Vj Vk Qj Qk Dst
6
7
8
ADD pipeline
MULT pipeline
DIV pipeline
LOAD buffer
CDB

Multiple issue

- Suppose you have two execution pipelines
  - each capable of beginning execution of one instruction per cycle
  - also have enough fetch/decode bandwidth so that IF does not stall
  - results can be immediately forwarded from one execution unit to another (or to itself)
  - pipeline only stalls for data dependencies
- Run the original code in the two-issue pipeline
  - **NOTE: do not reschedule the operations yet** (will do that later)
  - Ensure **in-order issue**

Loop:	LD	F2,0(Rx)	1 + 3
	MULTD	F2,F0,F2	1 + 4
	DIVD	F8,F2,F0	1 + 10
	LD	F4,0(Ry)	1 + 3
	ADDD	F4,F0,F4	1 + 2
	ADDD	F10,F8,F2	1 + 2
	SD	F4,0(Ry)	1 + 1
	ADDI	Rx,Rx,#8	1
	ADDI	Ry,Ry,#8	1
	SUB	R20,R4,Rx	1
	BNZ	R20,Loop	1 + 1
	cycles per loop iter		37

	Execution pipe 0		Execution pipe 1	
Loop:	LD	F2,0(Rx)	; <nop>	
	<stall for LD latency>		; <nop>	
	<stall for LD latency>		; <nop>	
	<stall for LD latency>		; <nop>	
	MULTD	F2,F0,F2	; <nop>	
	<stall for MULTD latency>		; <nop>	
	<stall for MULTD latency>		; <nop>	
	<stall for MULTD latency>		; <nop>	
	<stall for MULTD latency>		; <nop>	
	DIVD	F8,F2,F0	; LD	F4,0(Ry)
	<stall for LD latency>		; <nop>	
	<stall for LD latency>		; <nop>	
	<stall for LD latency>		; <nop>	
	ADDD	F4,F0,F4	; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	ADDD	F10,F8,F2	; SD	F4,0(Ry)
	ADDI	Rx,Rx,#8	; ADDI	Ry,Ry,#8
	SUB	R20,R4,Rx	; BNZ	R20,Loop
	<stall due to BNZ>		; <nop>	
	cycles per loop iter 24			

- LD goes first
- MULTD must wait because of data dependency
- Also DIVD must wait
- But the following LD can execute in 2nd pipe because it does not depend on DIVD
- More opportunities for concurrency at the end with loop handling code

Out of order

- The two pipelines are not quite identical
  - There is an implicit ordering: first pipeline has earlier instruction
- What if Pipe 1 completes before Pipe 0?
  - Could there be problems in this case?
  - Does our program have problems?
- Answers: problems
  - Interrupt occurs between N and N+1 instruction, then N+1 should not write result in any permanent way
  - N and N+1 may target the same register or same memory cell
  - N might be long floating point and eventually trap, after N+1 completes
  - In our example, DIVD will complete long after concurrent LD

Instruction scheduling

- Reorder the instructions to improve performance of the code
- Assume two-pipe machine as before
  - Assume out-of-order completion is dealt with already through a reorder buffer
  - Make sure you respect data dependency and functional latencies

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

	Execution pipe 0		Execution pipe 1
Loop:	**LD F2,0(Rx)**	;	**LD F4,0(Ry)**
	<stall for LD latency>	;	<stall for LD latency>
	<stall for LD latency>	;	<stall for LD latency>
	<stall for LD latency>	;	<stall for LD latency>
	MULTD F2,F0,F2	;	**ADDD F4,F0,F4**
	<stall for MULTD latency>	;	<stall for ADDD latency>
	<stall for MULTD latency>	;	<stall for ADDD latency>
	<stall for MULTD latency>	;	**SD F4,0(Ry)**
	<stall for MULTD latency>	;	<nop>
	DIVD F8,F2,F0	;	<nop>
	<stall for DIVD latency>	;	<nop>
	<stall for DIVD latency>	;	<nop>
	<stall for DIVD latency>	;	<nop>
	<stall for DIVD latency>	;	<nop>
	<stall for DIVD latency>	;	<nop>
	<stall for DIVD latency>	;	<nop>
	<stall for DIVD latency>	;	<nop>
	<stall for DIVD latency>	;	<nop>
	ADDI Rx,Rx,#8	;	**ADDI Ry,Ry,#8**
	SUB R20,R4,Rx	;	<nop>
	ADDD F10,F8,F2	;	**BNZ R20,Loop**
	<stall due to BNZ>	;	<stall due to BNZ>

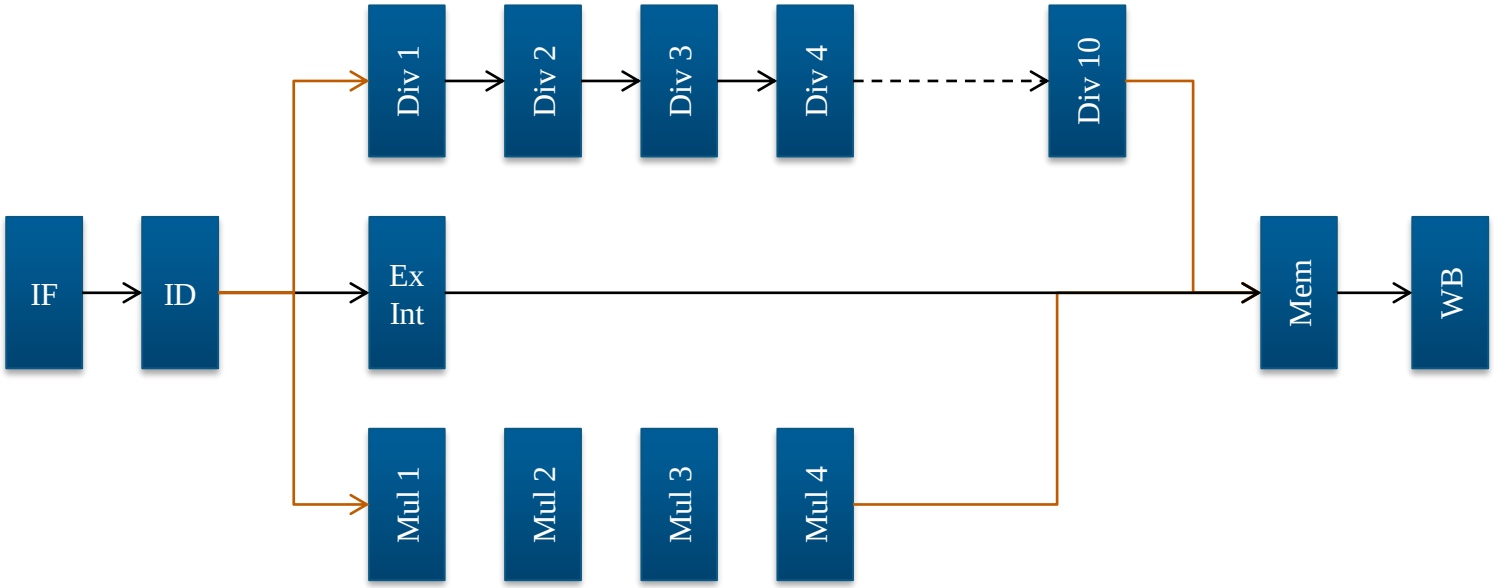
cycles per loop iter 22

Wasted resources

- What fraction of all cycles, counting both pipes, were wasted (did not initiate a new operation)?
  - 33 nops out of 2 * 22 cycles
  - 33 / 44 = 0.75 wasted cycles (or 75%)

Loop unrolling

- Hand-unroll two iterations of the loop in the reordered code
  - Reschedule the instructions after unrolling
  - No need to rename registers for this exercise (but would need to if doing it for real)
  - Also, assume that all functional units are pipelined. For instance, the divider has 10 pipeline stages, and we can start a new division the clock cycle after the first



	Execution pipe 0		Execution pipe 1	
Loop:	LD	F2,0(Rx)	; LD	F4,0(Ry)
	LD	F2,0(Rx)	; LD	F4,0(Ry)
	<stall for LD latency>		; <nop>	
	<stall for LD latency>		; <nop>	
	MULTD	F2,F0,F2	; ADDD	F4,F0,F4
	MULTD	F2,F0,F2	; ADDD	F4,F0,F4
	<stall for MULTD latency>		; <nop>	
	<stall for MULTD latency>		; SD	F4,0(Ry)
	<stall for MULTD latency>		; SD	F4,0(Ry)
	DIVD	F8,F2,F0	; <nop>	
	DIVD	F8,F2,F0	; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	<stall due to DIVD latency>		; <nop>	
	ADDI	Rx,Rx,#16	; ADDI	Ry,Ry,#16
	SUB	R20,R4,Rx	; <nop>	
	ADDD	F10,F8,F2	; <nop>	
	ADDD	F10,F8,F2	; BNZ	R20,Loop
	<stall due to BNZ>		; <nop>	

- Cycle per loop iteration: 23
- But iteration does effectively two original iterations
  - 23/2 = 11.5 cycles per loop
- Speedup
  - 22 / 11.5 = 1.91

Register renaming

- Assume your hardware has 64 extra registers, named T0 to T63
- Starting from register T9, rename the following fragment of code
  - Rename each time you see a new destination
  - Don't forget to rename the ensuing references to the same register

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F5,F0,F2
I1:   DIVD   F8,F0,F2
I2:   LD     F4,0(Ry)
I3:   ADDD   F6,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
```

Register renaming

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F5,F0,F2
I1:   DIVD   F8,F0,F2
I2:   LD     F4,0(Ry)
I3:   ADDD   F6,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
```

```
Loop:   LD      T9,0(Rx)
I0:     MULTD   T10,F0,T9
I1:     DIVD    T11,F0,T9
I2:     LD      T12,0(Ry)
I3:     ADDD    T13,F0,T12
I4:     ADDD    T14,T11,T9
I5:     SD      T12,0(Ry)
```

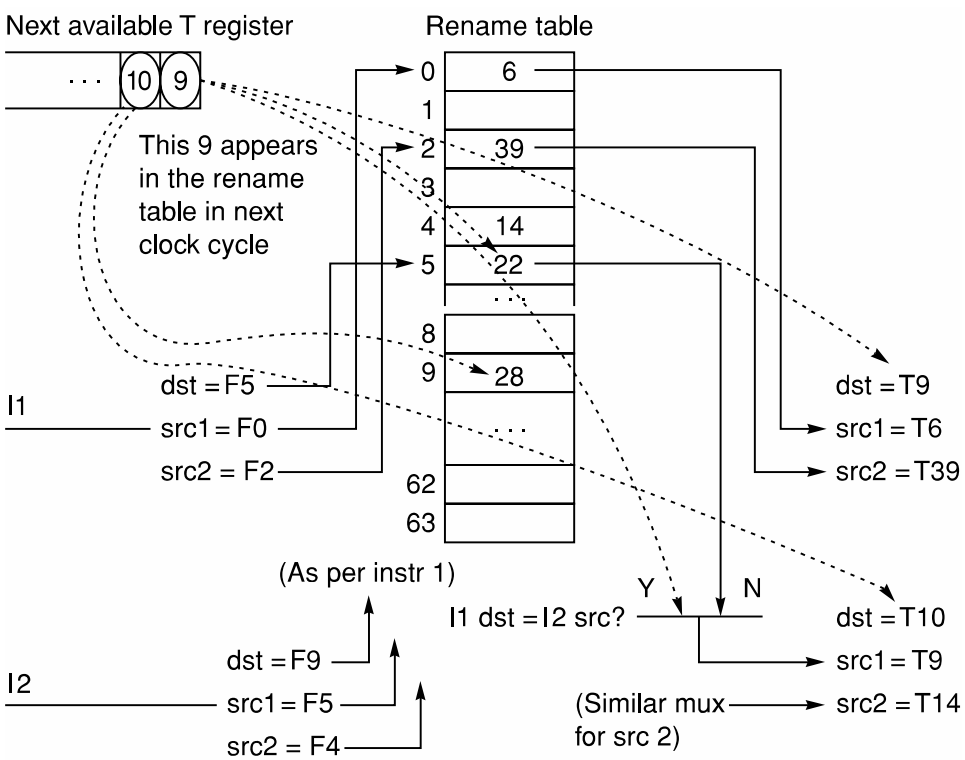
Superscalar renaming table

- Superscalar processors must rename multiple instructions in the same clock cycle
- They must also keep track of the dependencies between instructions issued in the same clock cycle
- Consider the code shown and rename the first two instructions simultaneously
  - First available register is T9
  - I0 must be renamed first
  - I1 must be renamed second
  - But all during the same clock cycle
  - Can’t update the rename table before we rename I1

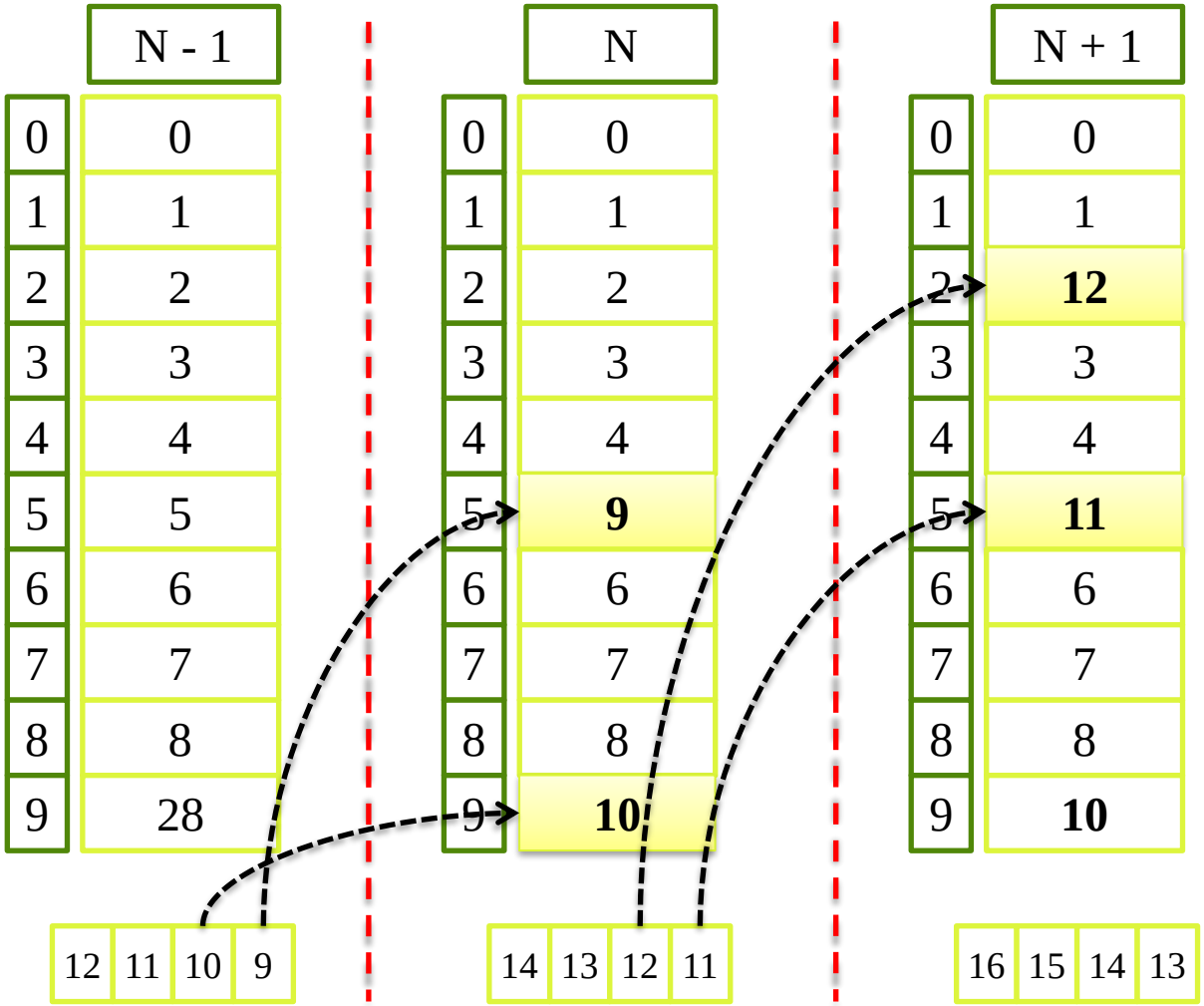
```
I0:  MULTD  F5,F0,F2
I1:  ADDD   F9,F5,F4
I2:  ADDD   F5,F5,F2
I3:  DIVD   F2,F9,F0
```

Superscalar renaming table

- Consider the circuit below
  - Show the cycle by cycle state of the rename table for every instruction, assuming initially every register is mapped to an entry equal to its index



I0:	MULTD	F5,F0,F2
I1:	ADDD	F9,F5,F4
I2:	ADDD	F5,F5,F2
I3:	DIVD	F2,F9,F0




```
I0:  MULTD  F5,F0,F2
I1:  ADDD   F9,F5,F4
I2:  ADDD   F5,F5,F2
I3:  DIVD   F2,F9,F0
```

I0: MULTD T9, T0, T2
I1: ADDD T10, T9, T4
I2: ADDD T11, T9, T2
I3: DIVD T12, T10, T0

I0.F5 renamed to T9
I1.F5 also renamed to T9 before rename table is updated
(I0 dst = I1 src)

I2.src.F5 first renamed to T9
I2.dst.F5 then renamed to T11
I2.src.F2 first renamed to T2
I3.dst.F2 then renamed to T12

Rename the original code

- Is there any part of the code that improves because of register renaming?
  - Are there any WAR or WAW hazards?
  - Hint: look at dependencies across iterations

```
Loop: LD     F2,0(Rx)
I0:   MULTD  F2,F0,F2
I1:   DIVD   F8,F2,F0
I2:   LD     F4,0(Ry)
I3:   ADDD   F4,F0,F4
I4:   ADDD   F10,F8,F2
I5:   SD     F4,0(Ry)
I6:   ADDI   Rx,Rx,#8
I7:   ADDI   Ry,Ry,#8
I8:   SUB    R20,R4,Rx
I9:   BNZ    R20,Loop
```

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
ADDD	+2
MULTD	+4
DIVD	+10

Five-stage pipeline

- Assume a five-stage pipeline (fetch, decode, execute, memory, write back)
  - All ops execute in 1 cycle, except LWs which are 1 + 2 cycles, branches are 1 + 1
  - No forwarding (data available when generating result in the WB stage)
- Show phases of execution for one iteration of the loop
  - How many clock cycles are lost to branch overhead?
  - Assuming a static branch predictor, how many clock cycles are wasted on branch? (must decode branch first!)
  - Assuming a dynamic branch predictor? (simply look at program counter to detect branches)

```
Loop: LW     R1,0(R2)
      ADDI   R1,R1,#1
      SW     R1,0(R2)
      ADDI   R2,R2,#4
      SUB    R4,R3,R2
      BNZ    R4,Loop
```

Five-stage pipeline

Loop length

Loop:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
LW R1,0(R2)	F	D	E	M	–	–	W												
ADDI R1,R1,#1		F	D	–	–	–	E	M	W										
SW R1,0(R2)			F	–	–	–	D	E	M	W									
ADDI R2,R2,#4							F	D	E	M	W								
SUB R4,R3,R2								F	–	–	D	E	M	W					
BNZ R4, Loop											F	–	–	D	E	M	W		
LW R1,0(R2)																	F	D	

(2.11 a.) 5 cycles lost to branch overhead

(2.11 b.) 2 cycles lost with static predictor

(2.11 c.) No cycles lost with correct dynamic prediction

```
Loop: LW      R1,0(R2)
      ADDI    R1,R1,#1
      SW      R1,0(R2)
      ADDI    R2,R2,#4
      SUB     R4,R3,R2
      BNZ     R4,Loop
```

1. Two-level CPU cache architecture

- Discuss the architecture of a two-level memory system. Show how to compute the Average Memory Access Time, and identify the local and the global miss rates.

- Assume that for every 1000 CPU-to-memory references (references, not instructions), 40 of them miss in level 1, and 20 of them miss in level 2. What are the local miss rates for the two caches? What is the global miss rate?

- Given the parameters for the L1 and L2 caches shown below, together with the local miss rate for different sizes of the L1 cache shown in the table, compute the average memory access time for the L1-L2 system for each L1 sizes.

  - L1 hit time = 1 cycle
  - L2 hit time = 4 cycles
  - L2 miss penalty = 100 cycles
  - L2 local miss rate = 50%

Cache size	L1 local miss rate
4 kB	16%
8 kB	10%
16 kB	6%
32 kB	3%
64 kB	2%

- For the two extreme cases above (4 kB and 64 kB), compute:

  - The global miss rate
  - The AMAT assuming there is no L2 cache using the same 100-cycle miss penalty.

1. Two-level CPU cache architecture

- Discuss the architecture of a two-level memory system. Show how to compute the Average Memory Access Time, and identify the local and the global miss rates.
  - AMAT = hittime_1 + miss_rate1 * miss_penalty_1
  - = hittime_1 + miss_rate1 * (hittime_2 + miss_rate2 * miss_penalty2)
  - Local miss rate: miss_rate1 and miss_rate2
  - Global miss rate: miss_rate1 and miss_rate1 * miss_rate2

- Assume that for every 1000 CPU-to-memory references (references, not instructions), 40 of them miss in level 1, and 20 of them miss in level 2. What are the local miss rates for the two caches? What is the global miss rate?
  - Local_1 = 40 / 1000 = 4/100 = 4%, Local_2 = 20 / 40 = 1/2 = 50%
  - Global_1 = 4%, Global_2 = 4/100 * 1/2 = 2/100 = 2%

- Given the parameters for the L1 and L2 caches shown below, together with the local miss rate for different sizes of the L1 cache shown in the table, compute the average memory access time for the L1-L2 system for each L1 sizes.

  - L1 hit time = 1 cycle, L2 hit time = 4 cycles, L2 miss penalty = 100 cycles
  - AMAT_4k = 1 + .16*(4 + .5*100) = 9.64 cycles, AMAT_8k = 1 + .10*(4 + .5*100) = 6.4 cycles
  - AMAT_16k = 1 + .06*(4 + .5*100) = 4.24 cycles, AMAT_32k = 1 + .03*(4 + .5*100) = 2.62 cycles
  - AMAT_64k = 1 + .02*(4 + .5*100) = 2.08 cycles

- For the two extreme cases above (4 kB and 64 kB), compute:

  - The global miss rate
  - Global_4k = .16 * .5 = 0.08 = 8%, Global_64k = .02 * .5 = 0.01 = 1%
  - The AMAT assuming there is no L2 cache using the same 100-cycle miss penalty.
  - AMAT_4k = 1 + .16 * 100 = 17 cycles, AMAT_64k = 1 + .02 * 100 = 3 cycles

2a. Multiprocessing

- Discuss the implementation of multiprocessor systems. Discuss in particular the different types of multiprocessor (symmetric and distributed shared memory), highlighting the main differences.
- Illustrate the Snoopy Coherence Protocol, using the write invalidate strategy and write back policy.

2b. Cache coherence

- For the following sequence of operations, assuming the access is always to the same address, show the state of the cache block for two CPUs sharing the memory. Assume a symmetric multiprocessor system using the coherence protocol described above, and describe the transactions that are sent on the bus.

time	Cache 1 state	Cache 2 state	Operation
0	Invalid	Invalid	
1			CPU 1 read miss
2			CPU 2 read miss
3			CPU 1 read hit
4			CPU 2 write hit
5			CPU 2 read hit
6			CPU 1 read miss

2a. Multiprocessing

- Discuss the implementation of multiprocessor systems. Discuss in particular the different types of multiprocessor (symmetric and distributed shared memory), highlighting the main differences.
- Illustrate the Snoopy Coherence Protocol, using the write invalidate strategy and write back policy.

2b. Cache coherence

- For the following sequence of operations, assuming the access is always to the same address, show the state of the cache block for two CPUs sharing the memory. Assume a symmetric multiprocessor system using the coherence protocol described above, and describe the transactions that are sent on the bus.

time	Cache 1 state	Cache 2 state	Operation
0	Invalid	Invalid	
1	Shared	Invalid	CPU 1 read miss
2	Shared	Shared	CPU 2 read miss
3	Shared	Shared	CPU 1 read hit
4	Invalid	Modified	CPU 2 write hit
5	Invalid	Modified	CPU 2 read hit
6	Shared	Shared	CPU 1 read miss

3b. TLP execution

Schedule the following two threads on a 2-way superscalar processor using the table below. Fetch one instruction from each thread per clock cycle and store them in an instruction buffer, ready to be scheduled. There is a single LD/ST unit, one FP adder, one FP multiplier, and one unit for the other integer operations. MULTD has a latency of 2 clock cycles, and ADDD has a latency of 1 clock cycles. The units are *not* pipelined. All other instructions execute in one clock cycle. At each clock cycle, you can execute an instruction if the functional unit is available and there are no data dependencies.

In the table, indicate the thread number and the instruction index to make it easier to identify which instruction is executing in each unit. Show the results for 8 clock cycles (not all instructions will be executed!). Suggestion: keep track of which instructions are in the instruction buffer for every clock cycle.

Thread #1

1: **LD** F2, 0(Rx)
2: **MULTD** F2, F0, F2
3: **ADDD** F4, F4, F2
4: **LD** F6, 0(Ry)
5: **ADDD** F8, F6, F6
6: **MULTD** F10, F6, F6
7: **ADDD** F10, F8, F10
8: **ADDI** Rx, Rx, #8
9: **ADDI** Ry, Ry, #8

Thread #2

1: **LD** F6, 0(Rz)
2: **ADDD** F2, F0, F6
3: **ADDI** Rz, Rz, #8
4: **JMP** -3

(the last JMP goes back to the first instruction)

Clock	Thr.#	LD/ST (lat: 0)	Thr. #	MULT (lat: 2)	Thr. #	ADDD (lat: 1)	Thr. #	Integer (lat: 0)
1								
2								
3								
4								
5								
6								
7								
8								

Thread #1

1: **LD** F2, 0(Rx)
2: **MULTD** F2, F0, F2
3: **ADDD** F4, F4, F2
4: **LD** F6, 0(Ry)
5: **ADDD** F8, F6, F6
6: **MULTD** F10, F6, F6
7: **ADDD** F10, F8, F10
8: **ADDI** Rx, Rx, #8
9: **ADDI** Ry, Ry, #8

Thread #2

1: **LD** F6, 0(Rz)
2: **ADDD** F2, F0, F6
3: **ADDI** Rz, Rz, #8
4: **JMP** -3

(the last JMP goes back to the first instruction)

Clock	Instr. Thread #1	Instr. Thread #2
1	1: LD	1: LD
2	2: MULTD	1: LD, 2: ADDD
3	3: ADDD	2: ADDD, 3: ADDI
4	3: ADDD, 4: LD	4: JMP
5	3: ADDD, 5: ADDD	1: LD
6	5: ADDD, 6: MULTD	2: ADDD
7	5: ADDD, 7: ADDD	2: ADDD, 3: ADDI
8	7: ADDD, 8: ADDI	2: ADDD, 4: JMP

Clock	Thr .#	LD/ST (lat: 0)	Thr . #	MULT (lat: 2)	Thr . #	ADDD (lat: 1)	Thr . #	Integer (lat: 0)
1	1	1						
2	2	1	1	2				
3			1	2	2	2	2	3
4	1	4	1	2	2	2	2	4
5	2	1			1	3		
6			1	6	1	3		
7			1	6	1	5	2	3
8			1	6	1	5	1	8

Tournament predictor

- Discuss the principle of a tournament predictor, and a possible state machine implementation.

- A tournament predictor uses an underlying 2-bit predictor and a (2, 2)-correlating predictor. The tournament predictor is initialized to use the 2-bit predictor. The table shows a sequence of 12 predictions, where value **1** indicates a **correct** prediction of the respective underlying predictor, while **0** indicates an **incorrect** prediction. Complete the table indicating the state of the tournament predictor (as illustrated in the previous point) and specifying the underlying predictor in use.

n	2-bit	(2,2) correl.	State	In use
0				2-bit
1	1	1		
2	1	0		
3	0	0		
4	0	1		
5	0	1		
6	1	1		
7	0	1		
8	0	0		
9	1	0		
10	1	0		
11	1	1		
12	1	1		